



UNIVERSIDAD CENTRAL DE VENEZUELA
FACULTAD DE CIENCIAS
ESCUELA DE COMPUTACIÓN

Trabajo Especial de Grado
**Implementación de una Metodología
de Mantenimiento de Software para la
Solución y Prevención de Fallas del
Sistema *PortalAsig2***

Trabajo Especial de Grado presentado ante la Ilustre
Universidad Central de Venezuela
Por el Bachiller

Rafael Arturo Vasquez Balza
C.I. 20.116.813

Para optar al título de
Licenciado en Computación

Tutor:
Prof. Walter Hernández

Caracas, Noviembre de 2020



Universidad Central de Venezuela
Facultad de Ciencias
Escuela de Computación

ACTA DEL VEREDICTO

Quienes suscriben, miembros del Jurado designado por el Consejo de Escuela de Computación, para examinar el Trabajo Especial de Grado presentado por el Bachiller Rafael Arturo Vasquez Balza, C.I.: 20.116.813, titulado: “Implementación de una Metodología de Mantenimiento de Software para la Solución y Prevención de Fallas del Sistema *PortalAsig2*”, a los fines de cumplir con el requisito legal para optar por el título de Licenciado en Computación, dejan constancia de lo siguiente:

Leído el trabajo por cada uno de los Miembros del Jurado, se fijó el día 19 de Enero de 2021 a las 10:20 horas para que su autor lo defendiera en forma pública y a distancia utilizando la plataforma *Discord*, mediante una presentación oral de su contenido, luego de lo cual respondió las preguntas formuladas, todo ello conforme a lo dispuesto en la Ley de Universidades y demás normativas vigentes de la Universidad Central de Venezuela. Finalizada la defensa pública del Trabajo Especial de Grado, el jurado decidió aprobarlo.

En fe de lo cual se levanta la presente Acta, en Caracas a los diecinueve (19) días del mes de Enero del año dos mil veintiuno (2021) dejándose también constancia de que actuó como coordinador del Jurado el Profesor Tutor Walter Hernández.

Prof. Walter Hernández
Tutor

Prof. Eugenio Escalise
Jurado Principal

Profa. Maria Paula Herrero
Jurado Principal

Agradecimientos

A mi padre, Jaime Vasquez, por ser un grato ejemplo de profesionalismo, dedicación y paciencia.

A mi madre, Maleisy Balza, por todo su sacrificio para garantizar el futuro de mi hermana y el mío así como los cariños, ánimos y cuidados que me ha dado.

A mi futura esposa, Raquel Escalante, cuyo invaluable apoyo, honestidad y sonrisa me permitieron dar los pasos finales y me brindaron la motivación necesaria en momentos oscuros.

A mi amigo, Rabindranath Ferreira, por motivarme a aprender nuevas habilidades y enfrentar desafíos técnicos para ganar experiencia. A mis amigos y compañeros de carrera, por presentarme nuevas perspectivas de la vida y hacer de esta larga maratón un paseo más ameno.

A mi tutor, Walter Hernández, por su entusiasmo, enfoque directo y práctico para la enseñanza, por su orientación y guía tanto académica como personal y finalmente por darme la oportunidad de participar en este proyecto.

Finalmente, a la Ilustre Universidad Central de Venezuela por las segundas oportunidades, por haberme dado una segunda casa a la que puedo llamar mi Alma Mater.

Resumen

En el mundo de los sistemas de información y desarrollo de aplicaciones web, los avances tecnológicos ocurren a pasos tan grandes y acelerados que las tecnologías y técnicas consideradas en el presente como confiables y efectivas, pueden volverse obsoletas en menos de un año. Debido a esto, gran parte de los esfuerzos de una organización que utiliza alguna pieza de software como componente importante de sus operaciones, son dirigidos a su mantenimiento. Este no solo abarca la solución de fallas o la adaptación del sistema a cambios en su ambiente de operación, sino también, y en mayor medida, la integración de nuevas funcionalidades para satisfacer las cambiantes necesidades de los usuarios. *PortalAsig2* es un sistema de software que se utiliza para crear portales de las asignaturas dictadas en la Facultad de Ciencias de la Universidad Central de Venezuela, con el fin de facilitar la administración de contenidos de las materias así como la comunicación entre grupos docentes y estudiantes. Este sistema, por falta de mantenimiento y un equipo que incluya al menos un desarrollador desde su puesta en operación, ha quedado relegado como un recurso estático, incapaz de adaptarse a los nuevos requerimientos sugeridos por sus propios usuarios o de aprovechar las mejoras en las tecnologías para el desarrollo de aplicaciones web. En este Trabajo Especial de Grado se define e implementa una metodología de mantenimiento de software con el fin de resolver la degradación presente en el sistema *PortalAsig2*. Luego, se define e implementa una metodología de reingeniería del software como parte de las actividades de mantenimiento. Además, se escoge una serie de métricas del software para medir y comparar los efectos del mantenimiento y la reingeniería de forma empírica. Como resultado se obtuvo una versión de *PortalAsig2* con nuevas funcionalidades y con mejores valores en las métricas definidas, comparado con su versión original.

Palabras clave: *PortalAsig2*, Mantenimiento de Software, Reingeniería del Software, Niveles de Abstracción, Documentación del Software, Aplicaciones *FrontEnd*, Aplicaciones *BackEnd*.

Índice general

Índice general	I
Índice de figuras	V
Índice de tablas	VIII
Códigos	IX
Introducción	X
1. Planteamiento del Problema	1
1.1. Justificación	2
1.2. Objetivo General	2
1.3. Objetivos Específicos	2
2. Marco Conceptual	4
2.1. Mantenimiento de Software	4
2.1.1. Tipos de Mantenimiento de Software	4
2.1.2. Problemas Asociados al Mantenimiento de Software	5
2.1.3. Procesos de Mantenimiento de Software	6
2.1.3.1. Modelo de Boehm	7
2.1.3.2. Proceso de Mejora Iterativo	7
2.1.3.3. Modelo Orientado al Reuso	8
2.1.4. Refactorización del Código	9

2.1.5.	Facilidad de Mantenimiento	9
2.1.6.	Complejidad del Software	10
2.1.6.1.	Complejidad Ciclomática de McCabe	10
2.1.6.2.	Métricas de Halstead	11
2.2.	Reingeniería de Software	12
2.2.1.	Objetivos de la Reingeniería de Software	12
2.2.2.	Niveles de Abstracción del Desarrollo de Software	13
2.2.3.	Modelo General para la Reingeniería de Software	14
2.2.4.	Ingeniería Inversa	15
2.2.5.	Ingeniería Directa	15
2.2.6.	Enfoques de Reingeniería	15
2.2.6.1.	Enfoque <i>Big Bang</i>	16
2.2.6.2.	Enfoque Incremental	17
2.2.6.3.	Enfoque Evolutivo	17
2.2.7.	Tareas y Fases de la Reingeniería del Software	18
2.2.7.1.	Formación del Equipo para la Reingeniería	18
2.2.7.2.	Estudio de Viabilidad del Proyecto	19
2.2.7.3.	Análisis y Planificación	19
2.2.7.4.	Implementación de la Reingeniería	20
2.2.7.5.	Transición y Pruebas	20
2.3.	Estado del Arte de Tecnologías Web <i>FrontEnd</i>	21
2.3.1.	<i>AngularJS</i>	21
2.3.2.	<i>Angular 2</i>	21
2.3.3.	<i>React</i>	21
2.3.4.	<i>VueJS</i>	21
2.3.5.	<i>Vuex</i>	22
2.3.6.	<i>Vue Router</i>	23
2.3.7.	<i>Bootstrap</i>	23

2.3.8.	<i>BootstrapVue</i>	24
2.4.	Tecnologías Web <i>BackEnd</i> Relevantes	24
2.4.1.	<i>Node.js</i>	24
2.4.2.	<i>NPM (Node Package Manager)</i>	24
2.4.3.	<i>Express.js</i>	25
2.4.4.	<i>Knex.js</i>	25
2.4.5.	<i>Bookshelf.js</i>	25
3.	Marco Aplicativo	26
3.1.	Metodología de Mantenimiento	26
3.2.	Iteración 1	27
3.2.1.	Análisis del Sistema Actual	27
3.2.2.	Clasificación de las Modificaciones Propuestas	32
3.2.2.1.	Metodología de Reingeniería	33
3.2.3.	Rediseño e Implementación	34
3.2.3.1.	Estudio de Viabilidad del Proyecto	34
3.2.3.2.	Análisis y Planificación	35
3.2.3.3.	Implementación de la Reingeniería	37
3.2.3.4.	Transición y Pruebas	43
3.3.	Iteración 2	45
3.3.1.	Análisis del Sistema Actual	45
3.3.2.	Rediseño e Implementación	48
3.4.	Iteración 3	55
3.4.1.	Análisis del Sistema Actual	55
3.4.2.	Rediseño e Implementación	57
3.5.	Iteración 4	59
3.5.1.	Análisis del Sistema Actual	59
3.5.2.	Rediseño e Implementación	60

<i>ÍNDICE GENERAL</i>	IV
4. Pruebas y Resultados	64
5. Conclusiones y Recomendaciones	69
Apéndice	71
Diagramas Actualizados de <i>PortalAsig2</i>	71
Diagramas de <i>PortalAsig2</i> Post Mantenimiento	77
Bibliografía	84

Índice de figuras

2.1. Modelo de Boehm	7
2.2. Fases del Proceso de Mejora Iterativo	8
2.3. Niveles de Abstracción	13
2.4. Modelo General para la Reingeniería del Software	14
2.5. Proceso de Ingeniería Inversa	16
2.6. Enfoque <i>Big Bang</i>	16
2.7. Enfoque Incremental	17
2.8. Enfoque Evolutivo	18
2.9. Estructura de Aplicación de <i>Vue</i> con <i>Vuex</i>	23
3.1. Descomposición en Componentes de <i>Vue</i> de la Página Principal	38
3.2. Diagrama de Componentes del Nuevo Sistema	39
3.3. <i>UI</i> del <i>Widget</i> de <i>Twitter</i> en el Nuevo Sistema	42
3.4. <i>UI</i> de Administración de Calificaciones del Sistema <i>Legacy</i>	43
3.5. Nueva <i>UI</i> de Administración de Calificaciones	44
3.6. Diagrama de Despliegue Actualizado de <i>PortalAsig2</i>	45
3.7. Nuevos Casos de Uso de Encuestas para el Rol Estudiante	46
3.8. Nuevos Casos de Uso de Encuestas del Grupo Docente	46
3.9. Nuevas Entidades y Relaciones de Encuestas de Sitios de Asignaturas	49
3.10. <i>UI</i> Para Grupo Docente de Lista de Encuestas Creadas	49
3.11. <i>UI</i> Para Grupo Docente de Detalles y Preguntas de un Quiz	50

3.12. UI Para Grupo Docente de Participantes de una encuesta/quiz	51
3.13. Modal para Abrir Encuesta/Quiz	51
3.14. UI Para Grupo Docente de Respuestas de una Encuesta/Quiz	52
3.15. UI Para Estudiantes de Lista de Encuestas o Quices Disponibles	52
3.16. UI Para Estudiantes de Detalle de un Quiz sin Empezar	53
3.17. Modal para Estudiantes de Inicio de Encuesta/Quiz	53
3.18. UI para que Estudiantes Respondan Quiz	54
3.19. UI para que Estudiantes Revisen Respuestas de Quiz	54
3.20. UI para que Grupo Docente Revise Respuestas de Quiz	55
3.21. Ejemplo de Horarios con Múltiples Secciones de Estudiantes	56
3.22. Formulario de Horarios con Campo <i>schedule_only</i>	58
3.23. Nueva UI de Lista de Cursos en Página Principal	58
3.24. Botón para Agregar Asignaturas	59
3.25. UI para Agregar Profesores a Sitio de Asignaturas	61
3.26. UI de Lista de Noticias	62
3.27. Formulario para Noticias con Editor <i>WYSIWYG</i>	62
3.28. Texto Enriquecido en Lista de Noticias	62
3.29. Contenido de Correos Electrónicos de Noticias	63
4.1. Variaciones de Métricas de Aplicación <i>FrontEnd</i>	66
4.2. Variaciones de Métricas de Aplicación <i>BackEnd</i>	66
4.3. Diferencia de desempeño entre distintas versiones de <i>PortalAsig</i> y <i>PortalAsig2</i>	68
A.1. Diagrama de Casos de Uso Actualizado - Usuario Anónimo	71
A.2. Diagrama de Casos de Uso Actualizado - Usuario Autenticado	72
A.3. Diagrama de Casos de Uso Actualizado - Estudiante	72
A.4. Diagrama de Casos de Uso Actualizado - Grupo Docente	73
A.5. Diagrama de Casos de Uso Actualizado - Docente	73
A.6. Diagrama de Casos de Uso Actualizado - Coordinador	74

A.7. Diagrama de Casos de Uso Actualizado - Administrador	74
A.8. Diagrama Entidad Relación Actualizado	75
A.9. Diagrama de Componentes de Aplicación <i>FrontEnd</i> Actualizado	76
A.10. Diagrama de Componentes de Aplicación <i>BackEnd</i> Actualizado	76
A.11. Diagrama de Casos de Uso- Usuario Anónimo	77
A.12. Diagrama de Casos de Uso - Usuario Autenticado	78
A.13. Diagrama de Casos de Uso - Estudiante	78
A.14. Diagrama de Casos de Uso - Grupo Docente	79
A.15. Diagrama de Casos de Uso - Docente	80
A.16. Diagrama de Casos de Uso - Coordinador	80
A.17. Diagrama de Casos de Uso - Administrador	81
A.18. Diagrama Entidad Relación	82
A.19. Diagrama de Componentes de Aplicación <i>FrontEnd</i>	83
A.20. Diagrama de Componentes de Aplicación <i>BackEnd</i>	83

Índice de tablas

2.1. Costos del Mantenimiento del Software	5
3.1. Clasificación de Modificaciones Propuestas	33
3.2. Versiones de Tecnologías del Sistema <i>Legacy</i>	35
3.3. Versiones de Tecnologías del Nuevo Sistema	36
3.4. Requerimientos Funcionales de Encuestas/Quices de Sitios de Asignatura	48
4.1. Resultados de Versiones de Aplicación <i>FrontEnd</i>	65
4.2. Resultados de Versiones de Aplicación <i>BackEnd</i>	65
4.3. Resultados de Aplicación <i>FrontEnd</i> en Porcentajes	65
4.4. Resultados de Aplicación <i>BackEnd</i> en Porcentajes	66
4.5. Resultados de Prueba de Desempeño de Velocidad de Carga	67

Códigos

2.1. Ejemplo de Componente de <i>Vue</i>	22
3.1. Método <i>Getter</i> de Objetivos Ordenados por Fecha de Creación	39
3.2. Función Auxiliar para dar Formato a Fechas	40
3.3. Ejemplo de Uso de Función Auxiliar Para Formato de Fechas	40
3.4. Implementación de Widget de Twitter en el Nuevo Sistema	41
3.5. <i>Middleware isProfessorSite</i> Modificado	57
3.6. Migración para Alterar Tabla <i>schedule</i> de la Base de Datos	57
3.7. Búsqueda de Profesores por Cédula, Nombre o Apellido	60

Introducción

En el mundo de los sistemas de información y desarrollo de aplicaciones web, los avances tecnológicos ocurren a pasos tan grandes y acelerados que las tecnologías y técnicas consideradas en el presente como confiables y efectivas, pueden volverse obsoletas en menos de un año. Debido a esto, gran parte de los esfuerzos de una organización que utiliza alguna pieza de software como componente importante de sus operaciones, son dirigidos a su mantenimiento. Este no solo abarca la solución de fallas o la adaptación del sistema a cambios en su ambiente de operación, sino también, y en mayor medida, la integración de nuevas funcionalidades para satisfacer las cambiantes necesidades de los usuarios.

En la actualidad, el desarrollo de nuevo software es el foco de atención principal en las organizaciones, delegando las actividades de mantenimiento a un segundo plano. Esto es atribuido a la necesidad de aprovechar las oportunidades emergentes en el mercado. Sin embargo, una vez que un producto de software encuentra éxito en el presente, este debe ser respaldado por un debido proceso de mantenimiento, que aumente su valor y robustez ante los usuarios, para garantizar su éxito a futuro.

Entre las actividades más notables del mantenimiento de software se encuentra la reingeniería, método empleado para recuperar los requerimientos y las reglas de negocio de sistemas de software anticuados o heredados que aún se encuentran en uso. Estos requerimientos y reglas, que fueron originalmente introducidos a lo largo de una amplia serie de modificaciones, son depurados, rediseñados e implementados en un nuevo sistema. Este suele poseer un mayor desempeño y una documentación actualizada, aspectos que otorgan un mayor valor agregado para sus usuarios y desarrolladores (o mantenedores) que el sistema original.

Un caso de estudio que se acopla a los criterios necesarios para que un sistema sea objeto de un proceso de reingeniería es la segunda versión del sistema generador de portales de asignaturas dictadas en la Facultad de Ciencias de la Universidad Central de Venezuela, mejor conocido como *PortalAsig2* en la comunidad de la Facultad. Este sistema, por falta de mantenimiento y un equipo que incluya al menos un desarrollador desde su puesta en operación, ha quedado relegado como un recurso estático, incapaz de adaptarse a los nuevos requerimientos sugeridos por sus propios usuarios o de aprovechar las mejoras en las tecnologías para el desarrollo de aplicaciones web.

En este Trabajo Especial de Grado se define e implementa una metodología de mantenimiento de software con el fin de resolver la degradación presente en el sistema *PortalAsig2*. Luego, se define e implementa una metodología de reingeniería del software como parte de las

actividades de mantenimiento. Además, se escoge una serie de métricas del software para medir y comparar los efectos del mantenimiento y la reingeniería de forma empírica. Como resultado se obtuvo una versión de *PortalAsig2* con nuevas funcionalidades y con mejores valores en las métricas definidas, comparado con su versión original.

El presente Trabajo Especial de Grado está compuesto por cinco capítulos. El primer capítulo, titulado “Planteamiento del Problema” describe el problema a solventar, que implica la falta de mantenimiento actual en el sistema *PortalAsig2* y la descripción de los objetivos de la solución a dicho problema. En el segundo capítulo, “Marco Conceptual” se definen los conceptos asociados al mantenimiento de software, reingeniería de software y las diferentes tecnologías asociadas al sistema *PortalAsig2* y la solución al problema. El tercer capítulo, “Marco Aplicativo” describe la metodología de mantenimiento, así como las actividades realizadas entre las que se encuentra la implementación de una metodología de reingeniería de software como componente fundamental del mantenimiento. El cuarto capítulo, “Pruebas y resultados” describe las pruebas realizadas al sistema y los resultados obtenidos con el fin de demostrar los beneficios tangibles obtenidos como consecuencia del desarrollo del presente Trabajo Especial de Grado. Finalmente, en el quinto capítulo, “Conclusiones y Recomendaciones”, se presentan las conclusiones obtenidas y una serie de recomendaciones con el objetivo de preservar y mejorar el trabajo realizado.

Capítulo 1

Planteamiento del Problema

PortalAsig fue un sistema de software propuesto, desarrollado y colocado en operación en el año 2013, para generar sitios o portales web de las asignaturas dictadas en la Facultad de Ciencias de la Universidad Central de Venezuela de manera simple, cómoda y moderna. Su objetivo principal era mantener una vía de comunicación constante entre estudiantes y docentes para complementar el contenido temático de las asignaturas [1]. Este sistema no recibió ningún tipo de mantenimiento durante todo su tiempo de operación, como consecuencia de esto, las fallas encontradas no fueron solucionadas y sus limitaciones, impuestas durante la fase de desarrollo, causaron varias inconveniencias y problemas a sus usuarios.

Para resolver los problemas presentes en el sistema *PortalAsig*, una nueva versión fue propuesta, desarrollada y puesta en operación en el año 2016. Los objetivos principales de esta nueva versión fueron preservar, mejorar y aumentar las funcionalidades existentes del sistema y alterar drásticamente su arquitectura, separando el sistema en dos aplicaciones web, una *BackEnd* y otra *FrontEnd*, esto permitió ofrecer una Interfaz de Programación de Aplicaciones (*API* por sus siglas en inglés) para proveer los servicios del sistema a otros componentes o aplicaciones externas, como por ejemplo, aplicaciones móviles [2]. Para su desarrollo fue necesario descartar el código fuente del sistema original y comenzar desde cero utilizando nuevas tecnologías que favorecieran la nueva arquitectura.

Entre los trabajos futuros y recomendaciones hechas por los equipos de desarrollo del sistema *PortalAsig* y su segunda versión *PortalAsig2*, se incluyeron nuevas funcionalidades a desarrollar e integrar. Sin embargo, entre estas, no fue considerada la formulación y consolidación de un plan efectivo de mantenimiento de software. De hecho, desde su puesta en operación en el año 2016, el sistema *PortalAsig2* tampoco ha recibido ningún tipo de mantenimiento. Solo ha contado con una única persona como personal administrativo, el tutor principal de este Trabajo Especial de Grado, quien ha recopilado una lista de nuevas funcionalidades recomendadas y fallas reportadas por los usuarios. Esta ausencia de personal y de una metodología de mantenimiento de software han resultado en una falta de adaptación a las nuevas necesidades de los usuarios y la corrección de fallas, provocando, en última instancia una degradación del sistema *PortalAsig2* semejante a la que sufrió *PortalAsig*.

1.1. Justificación

El mantenimiento de un sistema de software es igual o aún más importante que su desarrollo, pues se encarga de mantener la solución implementada sana y lista para lidiar con los cambios que se presentan en su ambiente de operación. En el caso del sistema *PortalAsig2*, es necesario definir e implementar una metodología de mantenimiento de software para, en primera instancia y con mayor prioridad, atacar las fallas presentes en el sistema, permitiendo su ejecución sin problemas, aumentando su usabilidad ante los usuarios y disminuyendo el número errores reportados por estos.

En segunda instancia, debido a los constantes cambios en las necesidades de los usuarios, los cuales sugieren nuevas funcionalidades para el sistema *PortalAsig2*, es necesario ampliar la capacidad inherente del sistema de adaptarse a mejoras, por lo que una metodología de mantenimiento que defina pautas para incorporar nuevas funcionalidades o mejorar las existentes, hará que el sistema responda positivamente a variaciones en su entorno de operación, como cambios en la lógica del negocio u otros aspectos que afecten a futuro su usabilidad por parte de la comunidad de la facultad. A partir de este hecho, también se infiere que una metodología correctamente adaptada a un sistema de software de este tipo incluye una actualización extensiva de dependencias que, como consecuencia directa, aumentará la seguridad, eficiencia y estabilidad del sistema, contribuyendo a la solución de fallas preexistentes y haciendo el sistema robusto, flexible y compatible con las tecnologías modernas de desarrollo web.

El proceso de mantenimiento, que engloba las tareas antes mencionadas, se realiza en aras de mejorar el rendimiento general del sistema, para garantizar la ejecución eficiente de una solución informática que aporta valor a sus usuarios, que a su vez sea lo suficientemente flexible para adaptarse a cambios futuros y que facilite un proceso de acoplamiento y crecimiento del equipo de mantenimiento lo más transparente y consistentemente posible.

1.2. Objetivo General

Implementar una metodología de mantenimiento de software para la solución y prevención de fallas del sistema *PortalAsig2* que incluya la sucesiva implementación de nuevas funcionalidades.

1.3. Objetivos Específicos

1. Crear una lista de modificaciones que incluya correcciones de fallas, nuevas funcionalidades y adaptaciones al ambiente de operación actual del sistema *PortalAsig2*.
2. Definir una metodología de mantenimiento de software adaptada al sistema *PortalAsig2* que guíe las actividades de mantenimiento.
3. Implementar la metodología definida para llevar a cabo las modificaciones propuestas.

4. Definir una serie de métricas de software que permitan medir y limitar o reducir la complejidad del sistema y el esfuerzo necesario para comprenderlo y modificarlo.
5. Realizar pruebas relevantes para asegurar que las modificaciones fueron implementadas correctamente.

Capítulo 2

Marco Conceptual

En este capítulo se presentan de forma estructurada los conceptos y definiciones asociadas a las metodologías, procesos y tecnologías empleadas en el desarrollo de la solución, y que contribuyen ampliamente a la comprensión de la misma.

2.1. Mantenimiento de Software

El mantenimiento de software es la última etapa en el ciclo de vida del desarrollo del software y se puede definir como el proceso de cambio que se aplica a un sistema de software después de su entrega y despliegue. Este término suele ser aplicado al software personalizado cuyos equipos de desarrollo difieren antes y después de su entrega. Los cambios realizados al software pueden ser cambios simples para corregir errores en el código, cambios más extensos para corregir errores en el diseño o incluso cambios significativos para corregir errores en las especificaciones del sistema o para agregar nuevos requerimientos. Los cambios son implementados mediante la modificación de componentes existentes del sistema o la adición de nuevos componentes [3].

El mantenimiento del software puede requerir una inversión de un 60 % hasta 80 % del costo total de todo el ciclo de vida del desarrollo del software dependiendo de la literatura consultada, como se ilustra en la Tabla 2.1, lo que da lugar a que la mayor parte del presupuesto total de los centros de procesamiento de datos se destine a mantener los sistemas existentes, por ello, se puede asegurar que el mantenimiento es la fase dominante, en tiempo y costo, del ciclo de vida [4].

2.1.1. Tipos de Mantenimiento de Software

En general existen tres tipos de mantenimiento de software [3]:

1. **Solución de Fallas:** Es el mantenimiento enfocado en reparar cualquier tipo de fallas descubiertas en el software, las fallas de código suelen ser las más baratas, rápidas y fáciles de reparar. Las fallas de diseño requieren más esfuerzo y recursos dado que requieren la

Referencia	% del Costo Total del Mantenimiento
Lientz & Swanson (1980)	60 %
Rock-Evans & Hales (1980)	67 %
Frazer (1992)	80 %
Pigoski (1997)	75 %
Pressman (2002)	60 %
Sommerville (2002)	65-75 %

Tabla 2.1: Costos del Mantenimiento del Software [4]

reescritura de varios componentes del sistema para ser reparadas. Finalmente, las fallas de requerimientos son las más costosas de reparar debido a que su solución puede requerir un rediseño extensivo de la estructura del sistema.

2. **Adaptación al Ambiente:** Este tipo de mantenimiento es requerido cuando ocurre un cambio en algunos aspectos del ambiente del sistema como el hardware, la plataforma de operación del sistema o cualquier otro software o herramienta de apoyo. El sistema debe ser modificado para adaptarlo a lidiar con estos cambios en su ambiente.
3. **Adición de Funcionalidades:** Este tipo de mantenimiento es necesario cuando los requerimientos del sistema cambian en respuesta a una alteración en las reglas del negocio o cuando algunos requerimientos no fueron contemplados al diseñar e implementar el sistema. La escala de los cambios requeridos del software para llevar a cabo este tipo de mantenimiento suele ser mayor que los de otros tipos de mantenimiento.

En la práctica no existe una distinción clara entre estos tipos de mantenimiento. Cuando se adapta un sistema a un nuevo ambiente, se le pueden añadir nuevas funcionalidades que tomen ventajas de las virtudes del nuevo ambiente. Las fallas del software pueden ser descubiertas cuando los usuarios utilizan el software de formas inesperadas, por lo que cambiar el sistema para adaptarse a la forma de trabajo de los usuarios es la mejor forma de resolver estas fallas.

Estos tres tipos de mantenimiento son generalmente reconocidos, pero varias fuentes pueden llamarlos por nombres diferentes. “Mantenimiento Correctivo” es usado universalmente para referirse al mantenimiento que repara fallas de un sistema. Sin embargo “Mantenimiento Adaptativo” a veces significa adaptar un sistema a un nuevo ambiente y otras veces significa adaptar el software a nuevos requerimientos. “Mantenimiento Perfectivo” algunas veces significa el perfeccionamiento de un software mediante la adición de nuevas funcionalidades y a veces significa mantener la misma funcionalidad de un sistema, pero mejorando su estructura y desempeño. Para evitar confusiones en este Trabajo Especial de Grado se utilizarán los tres tipos de mantenimiento descritos anteriormente.

2.1.2. Problemas Asociados al Mantenimiento de Software

Idealmente los tipos de mantenimiento definidos anteriormente son implementados a medida que sean necesarios. En la práctica, sin embargo, esto no es siempre posible debido a varias razones enumeradas a continuación [5]:

1. **Limitaciones de Recursos:** Este es el principal impedimento para la calidad y productividad de todas las actividades de mantenimiento. La falta de profesionales con las habilidades y experiencia necesarias, la falta de herramientas y el ambiente adecuado para llevar a cabo su trabajo así como la falta del presupuesto necesario son parte de los problemas que detienen el mantenimiento.
2. **Calidad del Sistema Existente:** En algunos sistemas “anticuados” la calidad puede ser tan pobre que cualquier cambio realizado, puede disparar un efecto dominó impredecible que pudiese colapsar el sistema. En la industria de las tecnologías de información se invierten grandes cantidades de recursos en el mantenimiento de código obsoleto, incluyendo el mantenimiento de hardware obsoleto, debido a los altos riesgos de efectos dominó impredecibles como consecuencia de cambios del código.

La calidad puede ser tan pobre que los cambios pueden ser prácticamente imposibles. Esto significa que las fallas no pueden ser resueltas y el sistema no puede evolucionar al ritmo de la tecnología. Esto puede causar que los usuarios estén atados a sistemas anticuados (tanto software como hardware), con altos costos de mantenimiento y que proveen menos funcionalidades que los sistemas más baratos de sus competidores. Existe un punto en el que mantener un sistema anticuado deja de ser viable.
3. **Estrategia Organizacional:** El deseo de una organización de mantenerse competitiva, especialmente en contra de sus rivales, puede ser un factor determinante del tamaño del presupuesto reservado para el mantenimiento. Estas decisiones estratégicas poseen un peso mayor que el análisis objetivo del problema a resolver a la hora de determinar el tamaño del presupuesto para las tareas de mantenimiento. Esta no es la forma ideal para garantizar que las labores de mantenimiento se realicen de la mejor manera.
4. **Resistencia al Cambio:** La resistencia al cambio por parte de los usuarios puede prevenir modificaciones de un producto de software, no importa que tan importante o potencialmente rentables dichas modificaciones puedan ser. Esta resistencia no siempre se origina de una obstinación irracional por parte de los usuarios, sino también por varias promesas incumplidas por parte de los desarrolladores de software a lo largo de las décadas, sin importar el motivo de dichas faltas.
5. **Atracción y Retención de Personal Calificado:** Los altos costos de mantener los sistemas operacionales y asegurarse que las necesidades de los usuarios son satisfechas se deben en parte al problema de atraer y retener personal talentoso y altamente capacitado para el mantenimiento del software.

2.1.3. Procesos de Mantenimiento de Software

Del mismo modo que existen actualmente varios modelos definidos para el desarrollo de software, también se han concebido varios modelos para el mantenimiento de software explicados brevemente a continuación [5]:

2.1.3.1. Modelo de Boehm

En 1983, Boehm propuso un modelo para el proceso de mantenimiento de software basado en los principios y modelos económicos. La idea de Boehm era que los principios y modelos económicos podrían mejorar la productividad del mantenimiento y además ayudar a ganar un mejor entendimiento del proceso. Boehm representa el proceso de mantenimiento como un ciclo cerrado, como se indica en la Figura 2.1.

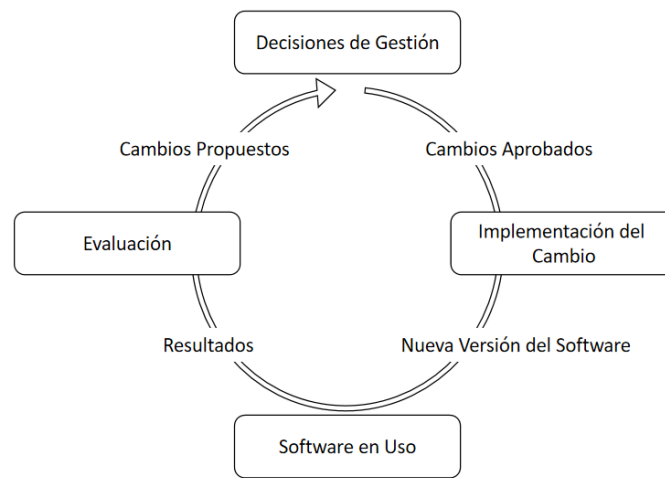


Figura 2.1: Modelo de Boehm [5]

Boehm teoriza que todo el proceso se rige por las decisiones tomadas en la fase de gestión. En esta, se determinan un conjunto de cambios aprobados aplicando estrategias particulares y evaluaciones costo-beneficio sobre un conjunto de cambios propuestos. Los cambios aprobados vienen acompañados de su propio presupuesto los cuales determinarán a gran escala los tipos y cantidades de recursos invertidos.

2.1.3.2. Proceso de Mejora Iterativo

El Proceso de Mejora Iterativo es un modelo de mantenimiento de software que propone que la implementación de cambios a un sistema de software durante su tiempo de vida útil es un proceso iterativo que involucra la mejora de dicho sistema de forma continua. Originalmente fue propuesto como un modelo de desarrollo de software, pero se adapta a las labores de mantenimiento dado que fue desarrollado para ser utilizado en ambientes donde los requerimientos no son comprendidos en su totalidad y por ende no se puede construir un sistema completo al primer intento.

Adaptando el Proceso de Mejora Iterativo para labores de mantenimiento, el modelo asume que la documentación del sistema está completa debido a que se apoya en esta como punto de partida de cada iteración para realizar modificaciones al sistema. El modelo es un ciclo de tres fases, ilustradas en la Figura 2.2.

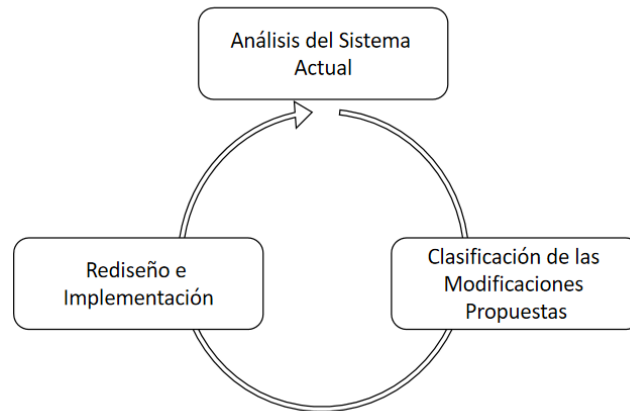


Figura 2.2: Fases del Proceso de Mejora Iterativo [5]

En la fase de análisis, se analizan los requerimientos y las capacidades del sistema de software actual para comenzar el proceso de mantenimiento del software. Después del análisis, se clasifican las modificaciones propuestas o requeridas de acuerdo a su complejidad, posibles problemas técnicos y se realiza la identificación de los módulos que se verán afectados por dichas modificaciones. En la última fase, se rediseña el software (de ser necesario) y se implementan y prueban las modificaciones.

En cada fase, la documentación disponible es actualizada para reflejar las modificaciones realizadas, comenzando por la documentación de más alto nivel afectada por las modificaciones propuestas. Luego estas modificaciones son propagadas por el resto del conjunto de documentos.

2.1.3.3. Modelo Orientado al Reuso

Este modelo está basado en el principio de que el mantenimiento debe ser visto como una actividad que incluye el reuso de componentes existentes de un software. El Modelo Orientado al Reuso descrito por Basili posee cuatro fases principales:

- Identificación de las partes del viejo sistema que pueden ser candidatas para reuso.
- Comprensión de las partes candidatas del sistema.
- Modificación de las partes del viejo sistema de acuerdo a los nuevos requerimientos.
- Integración de las partes modificadas al nuevo sistema.

Se necesita de un marco de trabajo detallado para la clasificación de componentes y sus posibles modificaciones. Con el Modelo Orientado a Reuso, el punto de partida del modelo puede ser cualquier fase del ciclo de vida del software: los requerimientos, el diseño, el código e incluso las pruebas.

2.1.4. Refactorización del Código

Una de las técnicas más utilizadas durante el mantenimiento de software es la refactorización del código. Podemos definir una refactorización como un cambio realizado a la estructura interna de un software para facilitar su comprensión y abaratar los costos de modificación sin alterar su comportamiento observable [6]. Esta es una herramienta cuyo uso le brinda a los integrantes de un equipo de desarrollo o mantenimiento las siguientes ventajas:

- **Mejorar el diseño del software:** A medida que las personas realizan cambios al código para cumplir con metas al corto tiempo o sin tener una comprensión completa del diseño de un software, este pierde su estructura. Se dificulta comprender su diseño al leer el código y por ende es más difícil mantenerlo. La refactorización permite depurar el software y mantener su diseño original al eliminar secciones redundantes de código que comprometen su facilidad de modificación.
- **Facilitar la comprensión del software:** Durante la creación de una pieza de software los desarrolladores se concentran en que el programa funcione y cumpla con los requerimientos definidos, no en su legibilidad o que pueda ser entendido por otros desarrolladores a futuro. La refactorización permite alterar una pieza de código funcional para estructurarla de una manera que permita mejor comunicar cuál es su propósito.
- **Encontrar fallas o errores:** Al clarificar la estructura de un programa y obtener un conocimiento más profundo de su propósito y funcionamiento gracias a la refactorización, cualquier falla o error se vuelve más fácil de encontrar o incluso resalta.
- **Aumentar la velocidad de desarrollo de los programadores:** Un buen diseño es esencial para lograr un avance veloz y constante en el desarrollo de software. La refactorización ayuda a detener la descomposición del diseño de un sistema y permite reducir la cantidad de código necesario para agregar nuevas funcionalidades.

2.1.5. Facilidad de Mantenimiento

La facilidad de mantenimiento del software ha sido definida de varias formas en la literatura, para este Trabajo Especial de Grado se utilizará la definición de Pressman, quien la define como “la facilidad con que un producto de software puede ser entendido, corregido, adaptado y/o mejorado” [7]. La facilidad de mantenimiento es un atributo externo del software, esto indica que calcularla requiere del conocimiento del producto de software en sí, así como como factores externos a este como el proceso de mantenimiento a utilizar y las habilidades y experiencia del personal de mantenimiento.

La facilidad de mantenimiento también puede ser percibida como un atributo interno del software si se deriva solamente del sistema de software. Varios atributos internos del código fuente del programa pueden afectar su facilidad de mantenimiento, por ejemplo su modularidad y la cantidad de líneas de código. Por ende, para calcular la facilidad de mantenimiento de un programa, se deben obtener medidas para estos otros atributos internos. Sin embargo, aún no

existe un modelo exacto para determinar la facilidad de mantenimiento de un software basado en estos atributos internos.

Hoy en día medidas como la complejidad de un programa de software son utilizadas como indicadores y/o pronosticadores de su facilidad de mantenimiento.

2.1.6. Complejidad del Software

Para el mantenimiento de software se utiliza la definición de complejidad descrita por Zuse: “La complejidad es la dificultad de comprender, cambiar y mantener programas”. Uno de los mayores retos que el personal de mantenimiento enfrenta es el de lidiar con la siempre creciente complejidad del código fuente del programa que deben modificar.

La complejidad de un programa abarca varias nociones como contenido semántico, estructura, flujo de control, flujo de datos y complejidad algorítmica. Aunque el cálculo de una única cifra para denotar la complejidad puede ocultar información importante, a veces es necesario para ser utilizada como indicador de otros atributos como comprensibilidad, facilidad de mantenimiento y el esfuerzo requerido para su desarrollo y prueba.

Mientras más complejo es un programa, existe una mayor probabilidad que el personal de mantenimiento cometa un error al implementar un cambio. Mientras mayor sea el valor de complejidad, más difícil es la comprensión del programa y por ende disminuye su facilidad de mantenimiento. Dado el argumento de que programas inherentemente complejos requieren más tiempo para ser comprendidos y modificados que programas simples, se concluye que la complejidad puede ser utilizada para estimar el esfuerzo requerido para efectuar un cambio en un programa.

Hablando sobre complejidad del software es importante mencionar la segunda ley de evolución o cambio del software de Lehman, formulada en 1974, la cual indica que “A medida que un sistema evoluciona y cambia, su complejidad aumenta a menos que se inviertan recursos para mantenerla o reducirla. Si se realizan cambios sin pensar en la estructura del sistema, su complejidad aumentará y dificultará cambios a futuro. Por otra parte, si se invierten recursos en combatir la complejidad, entonces menos recursos quedan disponibles para implementar cambios en el sistema. No importa cómo se reconcilie este balance, la tasa de crecimiento de un sistema inevitablemente decrece” [5].

Actualmente existen más de cien medidas de complejidad definidas en la literatura, sin embargo, a continuación se mencionan las dos más populares.

2.1.6.1. Complejidad Ciclomática de McCabe

M McCabe representa un programa como un grafo dirigido en el cual las líneas de instrucciones del programa son representadas por nodos y el flujo de control entre estas es representado por aristas. La complejidad ciclomática de McCabe es el número de caminos linealmente independientes a través del programa (o grafo de flujo) y este valor es calculado utilizando la Fórmula 2.1:

$$v(F) = e - n + 2 \quad (2.1)$$

Donde n es el número total de nodos, e representa el número total de aristas o arcos y $v(F)$ es la complejidad ciclomática, también conocido como el número ciclomático.

Esta medida es utilizada como una indicación de la dificultad psicológica para el programador de comprender y alterar todas las condiciones lógicas de un programa, incluyendo las ramificaciones derivadas de dichas condiciones. Durante el mantenimiento, un programa con un número ciclomático elevado (usualmente por encima de 10) es generalmente considerado como muy complejo.

2.1.6.2. Métricas de Halstead

Halstead propuso un número de ecuaciones para calcular los atributos de un programa como la longitud, volumen potencial, nivel de claridad del lenguaje, tiempo de implementación y tasas de errores. A continuación se describe el cálculo del esfuerzo de un programa, que es uno de los atributos que impactan la complejidad de un programa. La medida de este atributo puede ser calculada a partir de cuatro cuentas básicas:

- n_1 = número de operadores únicos utilizados
- n_2 = número de operandos únicos utilizados
- N_1 = número total de operadores utilizados
- N_2 = número total de operandos utilizados

Un operando es una variable o constante. Un operador es una entidad que puede cambiar un operando o que puede afectar el orden en que es cambiado. Los operadores incluyen: operadores aritméticos, palabras claves reservadas del lenguaje, operadores lógicos y delimitadores. A continuación se presenta la Fórmula 2.2 que se utiliza para el cálculo del esfuerzo estimado necesario para implementar cambios a un programa:

$$E = \frac{n_i * N_2 * (N_1 + N_2) * \log(n_1 + n_2)}{2 * n_2} \quad (2.2)$$

Basados en este número de esfuerzo, se puede estimar el tiempo en segundos que toma entender y escribir el programa utilizando la Fórmula 2.3 y la cantidad de posibles fallas o errores (*bugs* en inglés) en la implementación utilizando la Fórmula 2.4.

$$T = \frac{E}{18} \quad (2.3)$$

$$B = \frac{E^{\frac{2}{3}}}{3000} \quad (2.4)$$

2.2. Reingeniería de Software

La reingeniería es el análisis y alteración de un sistema de software existente anticuado o heredado pero aún en uso (referido como *legacy* de ahora en adelante) para rediseñarlo con una nueva estructura y la posterior implementación con dicha estructura. La reingeniería suele ser una conglomeración de otros procesos como la ingeniería inversa, redocumentación, reestructurado, traducción e ingeniería directa. La meta es entender el sistema de software *legacy* (especificación, diseño e implementación) y luego reimplementarlo para mejorar su desempeño y facilidad de mantenimiento, conservando la funcionalidad actual del sistema y preparándolo para futuras adiciones de nuevas funcionalidades [8].

2.2.1. Objetivos de la Reingeniería de Software

El reto principal en la reingeniería de software consiste en la integración de buenas prácticas y métodos del desarrollo de software en un sistema de software *legacy*, generando un nuevo sistema que mantiene la misma funcionalidad pero hace uso de nuevas tecnologías. Los objetivos específicos de un proceso de reingeniería suelen estar definidos en base a las metas de la entidad u organización dueña o a cargo del sistema, sin embargo se pueden definir cuatro objetivos generales:

- **Preparación para mejoras funcionales:** Si bien la reingeniería no debería realizarse para implementar mejoras funcionales al sistema *legacy*, se suele utilizar como paso previo a dichas mejoras. Los sistemas *legacy* son costosos y difíciles de mantener, reparar y mejorar debido a falta de documentación o estructura lógica coherente y clara. El nuevo sistema, producto de la reingeniería puede ser concebido para facilitar las mejoras. Por ejemplo, si las mejoras requeridas del sistema están basadas en un diseño orientado a objetos entonces el nuevo sistema puede ser desarrollado utilizando un diseño y tecnologías orientadas a objetos anticipando las mejoras funcionales a realizar sobre el sistema *legacy*.
- **Incrementar la facilidad de mantenimiento:** A medida que los sistemas crecen y evolucionan, los costos de mantenimiento aumentan, pues cada cambio dificulta y aumenta el tiempo de desarrollo de cambios subsiguientes. Uno de los objetivos de la reingeniería es rediseñar el sistema con módulos funcionales e interfaces explícitas. Esto, junto a la actualización de la documentación, aumenta la facilidad de mantenimiento.
- **Migración a nuevas tecnologías:** La industria de la computación continúa creciendo a un ritmo exponencial, nuevo hardware y sistemas de software incluyendo nuevas funcionalidades, superan rápidamente las capacidades de los sistemas actuales. A medida que

estos sistemas cambian, las habilidades del personal migran hacia las nuevas tecnologías dejando menos personas a cargo del mantenimiento de los sistemas más antiguos. En un periodo relativamente corto de tiempo los fabricantes dejan de brindar soporte de software y las partes de hardware se vuelven más costosas. Por estas y otras razones las compañías con un software operativo que satisface sus necesidades pueden necesitar una migración a nuevas plataformas de hardware, sistemas operativos, lenguajes o tecnologías de software.

- **Mejorar la confiabilidad del sistema:** La confiabilidad de un sistema se ve reducida cada vez que se le realizan cambios, ya que estos pueden tener un efecto dominó. Un cambio puede causar varios problemas adicionales y a medida que el mantenimiento y los cambios continúan, la confiabilidad del software disminuye constantemente hasta un nivel inaceptable.

2.2.2. Niveles de Abstracción del Desarrollo de Software

Los niveles de abstracción, representados en la Figura 2.3, que guían el proceso de desarrollo de software también guían el proceso de reingeniería de software. Cada nivel está asociado con una fase en el ciclo de vida del desarrollo y define o describe el sistema de software en un nivel de detalle particular (o abstracción).

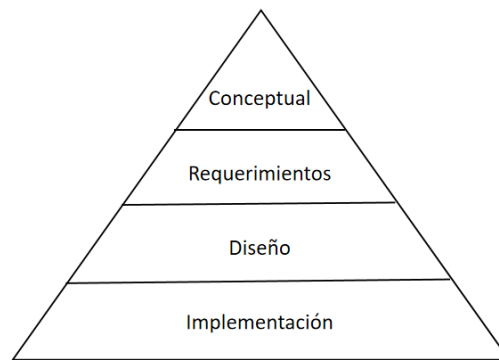


Figura 2.3: Niveles de Abstracción [9]

El nivel conceptual es el nivel más alto de abstracción, donde se describe el concepto y/o la razón de existencia del sistema. En este nivel las características funcionales son descritas sólo en términos generales. En el nivel de requerimientos las características funcionales del sistema son descritas en términos detallados y explícitos. Los detalles internos del sistema no son mencionados en estos dos primeros niveles de abstracción. En el nivel de diseño se describen las propiedades del sistema como estructura, componentes, interfaces entre los componentes, procedimientos o algoritmos y estructuras de datos. La implementación es el nivel más bajo de abstracción, en donde la descripción de un sistema se concentra en las características de la implementación y es representado en un lenguaje que la computadora entiende [9].

2.2.3. Modelo General para la Reingeniería de Software

La reingeniería de software siempre comienza a partir del código fuente del sistema *legacy* y termina con el código fuente del nuevo sistema deseado. Este proceso puede ser tan sencillo como el uso de una herramienta de traducción automática de código para traducir código de un lenguaje a otro o de un sistema operativo a otro (*Windows* a *MacOS*, por ejemplo). Por otra parte el proceso puede llegar a ser tan complejo como se necesite, requiriendo la ejecución de varias tareas como analizar el código fuente para recrear el diseño original, identificar los requerimientos del sistema *legacy* y compararlos con los requerimientos actuales descartando los requerimientos en desuso, reestructurar y rediseñar el sistema y finalmente la escritura del código del nuevo sistema deseado.

La Figura 2.4 describe un modelo general para la reingeniería de software que señala todos los procesos a través de todos los niveles de la reingeniería tomando como base los niveles de abstracción descritos previamente.

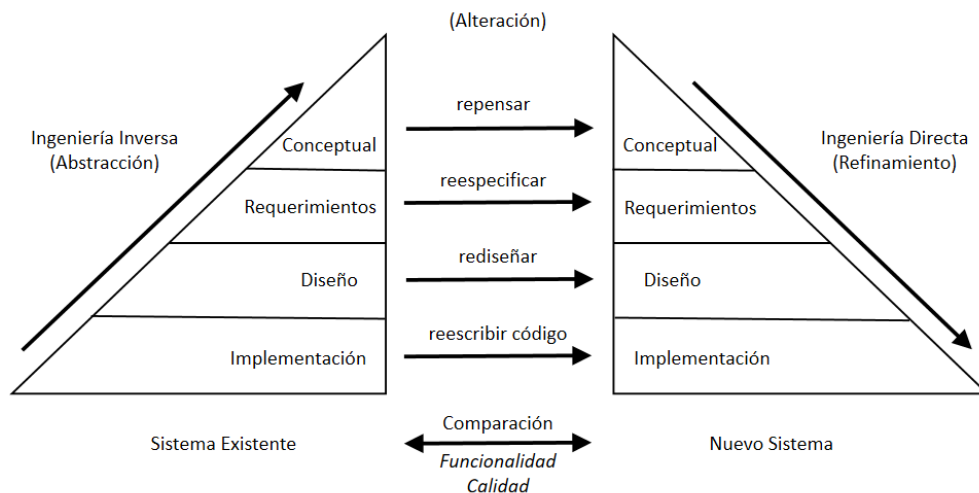


Figura 2.4: Modelo General para la Reingeniería del Software [9]

El modelo descrito en la Figura 2.4 señala tres procesos de la reingeniería: abstracción, alteración y refinamiento. La abstracción es el proceso de incremento gradual en los niveles de abstracción que produce una representación del sistema que resalta ciertas características del sistema mientras ignora otras. Este movimiento en subida es denominado Ingeniería Inversa y tiene una serie de herramientas, técnicas y subprocesos asociados. La alteración es la generación de uno o más cambios en la representación de un sistema sin cambiar el nivel de abstracción incluyendo la adición, eliminación y modificación de información, pero no de funcionalidades. El refinamiento es la disminución gradual del nivel de abstracción de la representación del sistema y es causada mediante la sustitución sucesiva de información del sistema *legacy* por información más detallada. Este proceso es conocido como Ingeniería Directa y se asemeja al desarrollo de software “desde cero” pero con algunas mejoras del proceso.

Para alterar una propiedad del sistema *legacy*, se trabaja en el nivel de abstracción en el que

la información acerca de dicha propiedad es expresada explícita y detalladamente. Para traducir el código existente a un nuevo lenguaje no se requiere ingeniería inversa, la alteración (reescribir código) es realizada en el nivel de implementación. A medida que el nivel de abstracción aumenta, las tareas de alteración y la cantidad de tareas de ingeniería inversa cambian. Por ejemplo, para reespecificar los requerimientos se deben aplicar técnicas de ingeniería inversa a la implementación y al diseño para obtener las propiedades funcionales.

2.2.4. Ingeniería Inversa

La ingeniería inversa es el proceso de análisis de un sistema particular para identificar sus componentes internos y las interrelaciones entre estos, con el fin de crear representaciones del sistema en otra forma o en niveles superiores de abstracción. En la ingeniería inversa, los requerimientos, el diseño, la estructura y el contenido de los sistemas *legacy* deben ser recuperados. Otras propiedades que también deben ser recuperadas son las reglas del negocio y los procesos que han sido útiles para la ejecución exitosa del sistema en su entorno de producción.

Los objetivos clave de la ingeniería inversa son: generar vistas alternativas del sistema, recuperar información perdida, detectar efectos secundarios, generar abstracciones de alto nivel y facilitar el reuso. La efectividad de este proceso impactará el éxito del proyecto de reingeniería. La ingeniería inversa no incluye cambios al sistema o la creación de un nuevo sistema, sino solo el proceso de análisis sin cambio alguno en la funcionalidad general del sistema.

El proceso de ingeniería inversa ilustrado en la Figura 2.5, comienza con la extracción de información sobre los requerimientos y el diseño detallado a partir del código fuente y los documentos existentes. Posteriormente se define un documento de requerimientos y un diseño de alto nivel de abstracción expresado en diagramas de flujo de datos y diagramas de flujo de control.

2.2.5. Ingeniería Directa

En el proceso de ingeniería directa, el nuevo sistema es creado a través de un descenso gradual entre niveles de abstracción como consecuencia de reemplazos sucesivos de información general del sistema existente por información más detallada. Este movimiento descendente es equivalente al proceso utilizado en el desarrollo de software tradicional. La ingeniería directa se desplaza desde los niveles de alta abstracción con diseños descritos sin detalles lógicos de implementación hasta los niveles más bajos que describen la implementación física del sistema. Este proceso sigue una secuencia desde requerimientos, pasando por el diseño, hasta llegar a la implementación.

2.2.6. Enfoques de Reingeniería

Existen varios enfoques para la implementación de la reingeniería de software, con diferencias entre cada uno de ellos sobre la cantidad y ritmo de reemplazo del sistema *legacy* por el

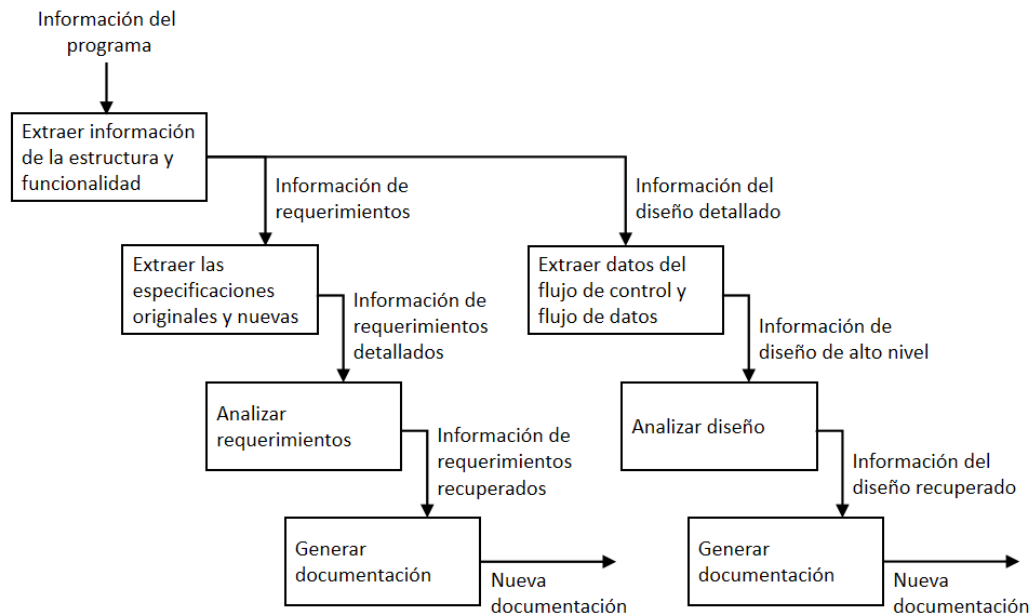
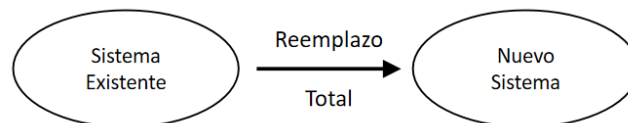


Figura 2.5: Proceso de Ingeniería Inversa [8]

nuevo sistema. Cada enfoque posee sus propios riesgos y beneficios.

2.2.6.1. Enfoque *Big Bang*

El enfoque *Big Bang* reemplaza todo el sistema de una sola vez como se ilustra en la Figura 2.6. Este enfoque es utilizado comúnmente en proyectos que requieren solventar un problema inmediato como la migración a un sistema con arquitectura diferente.

Figura 2.6: Enfoque *Big Bang* [8]

La ventaja de este enfoque es que todo el sistema es llevado a un nuevo ambiente de una sola vez. No se deben desarrollar interfaces entre los componentes viejos y los nuevos, y no se debe operar ni mantener ambientes mezclados. La desventaja de este enfoque es que el producto resultante suele ser un sistema monolítico (autónomo e independiente de otros sistemas) que no siempre es adecuado o deseado. Para sistemas grandes, este enfoque puede consumir muchos recursos o requerir gran cantidad de tiempo antes de obtener el nuevo sistema. El riesgo de este enfoque es alto, porque el sistema debe estar funcionalmente intacto y funcionar en paralelo con el viejo sistema para validar su funcionalidad. Este funcionamiento paralelo puede resultar difícil y costoso.

2.2.6.2. Enfoque Incremental

En el enfoque incremental de reingeniería, ilustrado en la Figura 2.7, se realiza un reemplazo incremental y gradual de diferentes partes del sistema *legacy*, a las cuales se les aplica el proceso de reingeniería para obtener nuevas versiones del sistema que cumplan con los requerimientos definidos. El proyecto se divide en diferentes secciones de reingeniería, basadas en las secciones del sistema *legacy*.

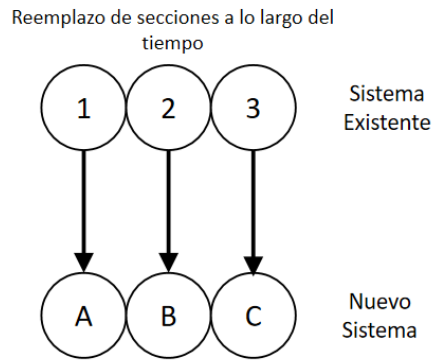


Figura 2.7: Enfoque Incremental [8]

La ventaja de este enfoque es que las nuevas versiones de las secciones del sistema son desarrolladas más rápido comparadas con el desarrollo de todo el sistema y además es más fácil identificar y rastrear fallas debido a que los nuevos componentes están claramente definidos y delimitados. Como las versiones intermedias pueden ser desplegadas, el cliente puede ver el progreso y detectar rápidamente cualquier funcionalidad faltante o anómala. Una desventaja de este enfoque es que toma más tiempo completar el sistema nuevo debido a la necesidad de configurar y evaluar varias versiones intermedias. Otra desventaja es que no se puede alterar la estructura general del sistema, sino solo la estructura interna de cada una de las secciones sujetas a la reingeniería. Esto requiere una cuidadosa identificación de los componentes en el sistema *legacy* y la planificación extensiva de la estructura del nuevo sistema. Este enfoque posee menos riesgo que el enfoque *Big Bang* porque a medida que se inicia el proceso de reingeniería de cada una de las secciones del sistema, se pueden identificar y monitorear los riesgos de cada sección por separado.

2.2.6.3. Enfoque Evolutivo

En el enfoque evolutivo, al igual que en el enfoque incremental, se reemplazan secciones del sistema *legacy* con nuevas secciones, producto de un proceso de reingeniería. En este proceso, sin embargo, las secciones son escogidas en base a su funcionalidad, no en base a la estructura del sistema *legacy*. El sistema nuevo es construido a partir de secciones funcionalmente cohesivas a medida que se necesiten. El enfoque evolutivo permite a los desarrolladores enfocarse en identificar los objetos funcionales independientemente de donde estos se encuentren dentro del sistema actual. Como se ilustra en la Figura 2.8, los componentes del sistema existente se

dividen por su funcionalidad y mediante un proceso de reingeniería son agrupados en nuevos componentes.

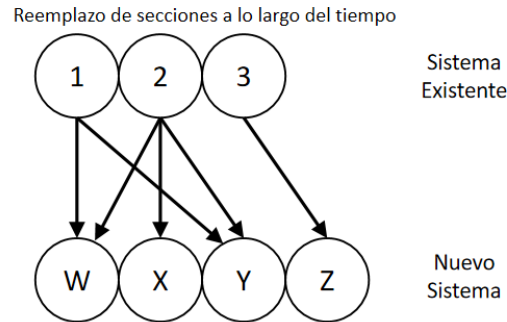


Figura 2.8: Enfoque Evolutivo [8]

Las ventajas del sistema evolutivo de reingeniería son el diseño modular resultante y el alcance reducido para cada componente. Este enfoque tiene buen desempeño cuando se utiliza para migrar hacia tecnologías orientadas a objetos. Una desventaja es que funcionalidades similares deben ser identificadas a lo largo del sistema existente y luego refinadas en una única unidad funcional. Además también pueden existir errores de interfaces y reducción del tiempo de respuesta puesto que se aplica el proceso de reingeniería a las secciones funcionales del sistema *legacy* en vez de sus secciones arquitectónicas.

2.2.7. Tareas y Fases de la Reingeniería del Software

Existe un proceso básico que cada organización debería seguir para llevar a cabo exitosamente un proyecto de reingeniería de software. La reingeniería posee sus propios retos técnicos que pueden desperdiciar esfuerzo, dinero y tiempo sin el seguimiento de un proceso de desarrollo completo. El proceso de reingeniería de software puede ser dividido en cinco fases y sus tareas asociadas, estas fases son:

1. Formación del Equipo para la Reingeniería
2. Estudio de Viabilidad del Proyecto
3. Análisis y Planificación
4. Implementación de la Reingeniería
5. Transición y Pruebas

2.2.7.1. Formación del Equipo para la Reingeniería

Este equipo se encargará de llevar a cabo el proceso de reingeniería desde su inepción hasta su conclusión y necesitará entrenamiento adecuado sobre el manejo del cambio tecnológico, los

principios básicos del proceso de reingeniería y el uso de procesos de mantenimiento y desarrollo de software. Sus tareas son diversas, incluyendo la definición de metas, estrategias y un plan de acción dentro del ambiente actual y basado en las necesidades del negocio identificadas, incluyendo justificaciones de costos. Aunque los miembros del equipo deben poseer las habilidades tradicionales o estándar del desarrollo de software, también necesitarán adquirir nuevas habilidades específicas. Los integrantes de este equipo son responsables de identificar, probar y comprar nuevas herramientas así como asegurarse de que todo el personal esté debidamente entrenado en las mismas para su uso efectivo. Además, los miembros del equipo deberán dominar un conjunto de habilidades suaves o interpersonales para lidiar con la resistencia al cambio a nuevos conceptos y tecnologías. Como el campo de la reingeniería siempre está evolucionando, los miembros del equipo deberán mantenerse al día para realizar el trabajo de la forma más efectiva posible.

2.2.7.2. Estudio de Viabilidad del Proyecto

La primera tarea que el equipo de reingeniería debe realizar es la evaluación de cuáles metas y necesidades de la organización cumple el sistema *legacy*. Es importante que la estrategia de reingeniería escogida se ajuste a las normas culturales de la organización. Los productos de software actuales deben ser analizados en términos de especificaciones de problemas incluyendo objetivos, justificación, restricciones y reglas del negocio.

El valor de las aplicaciones debe ser investigado para determinar cuál será el retorno de la inversión del esfuerzo de reingeniería: el grado de aumento esperado de mejoría del software, el aumento de la eficiencia del proceso de mantenimiento del software y el aumento esperado al valor del negocio. Una vez las expectativas han sido establecidas deben ser expresadas en una manera cuantificable: reducción de costo de operaciones, reducción de costos de mantenimiento, mejoras del desempeño, entre otras. Luego los costos de la reingeniería deben ser comparados con el ahorro proyectado de costos y el incremento del valor del sistema.

2.2.7.3. Análisis y Planificación

Esta fase se subdivide en tres subfases: análisis del sistema *legacy*, especificación de las características y requerimientos del nuevo sistema y la creación de un conjunto de pruebas para validar la transferencia correcta de funcionalidades.

La subfase de análisis comienza con la búsqueda de todo el código y documentación disponible (incluyendo manuales de usuario, documentos del diseño y especificaciones de requerimientos). Una vez reunida toda la información disponible sobre el sistema *legacy*, esta se analiza para identificar sus aspectos únicos. Se debe especificar la condición del sistema *legacy*, su facilidad de mantenimiento y de operación para expresar el retorno de inversión a obtener del nuevo sistema. Luego, un conjunto de métricas del sistema deberá ser seleccionado para apoyar la identificación de posibles deficiencias en la calidad del sistema *legacy* y la priorización de las aplicaciones que son candidatas para la reingeniería de acuerdo a su calidad técnica y su valor comercial. Si una de las metas del proceso de reingeniería es el incremento de la facilidad de mantenimiento entonces las métricas deberán incluir aproximaciones de los costos de los

cambios al sistema.

Una vez se especifican las características de calidad del sistema *legacy*, se procede a establecer las características y especificaciones de requerimientos del nuevo sistema. Se especifican cuales son las características del sistema original que deben cambiar, por ejemplo: sistema operativo, plataforma de hardware, diseño de estructura y/o lenguaje.

Finalmente se crea un conjunto de pruebas o controles de calidad. Este conjunto será utilizado para demostrar que el nuevo sistema es funcionalmente equivalente al sistema *legacy* y que no existen variaciones o desviaciones de las mismas funcionalidades después del proceso de reingeniería. Este conjunto de pruebas puede ser implementado de forma incremental de ser necesario, pero lo importante es que cada prueba esté relacionada con las funciones del sistema *legacy*.

2.2.7.4. Implementación de la Reingeniería

Una vez los objetivos del proceso de reingeniería han sido definidos, el enfoque ha sido seleccionado y el sistema *legacy* ha sido analizado, se inician los procesos de ingeniería inversa y directa. Recorriendo los niveles de abstracción de la Figura 2.3, se aplica un proceso de ingeniería inversa para comprender las funciones del sistema *legacy*. Para este proceso existen varias herramientas disponibles, sin embargo, cada herramienta debe ser examinada para corroborar que su uso es viable en el contexto de los objetivos del proceso de reingeniería definidos. El uso de estas herramientas debe ser fácilmente integrable al proceso.

Cuando se alcanza el nivel de abstracción deseado, el proceso de ingeniería directa comienza. Las tareas de la ingeniería directa corresponden con las tareas del proceso de desarrollo de software tradicional, comenzando por el nivel de abstracción alcanzado durante el proceso de ingeniería inversa. Por ejemplo, si el software fuese a ser rediseñado para adaptarse a una arquitectura de sistema completamente nueva, el proceso de ingeniería inversa debería extraer los requerimientos del software y el proceso de ingeniería directa podría comenzar con el desarrollo de un nuevo diseño. Se debe evitar cualquier adición o cambio en la funcionalidad del sistema durante el proceso de ingeniería directa puesto que esto dificultará el proceso de validación.

2.2.7.5. Transición y Pruebas

A medida que la funcionalidad del nuevo sistema aumenta, se deben realizar pruebas para detectar cualquier error introducido durante el proceso de reingeniería. Las técnicas utilizadas para el control de calidad son las mismas que se utilizan durante el desarrollo de un sistema “desde cero”. Asumiendo que los requerimientos del nuevo sistema son los mismos que los del sistema *legacy*, se puede utilizar el mismo conjunto de pruebas definidas durante la fase de planificación del proceso de reingeniería. Dicho conjunto de prueba se le puede aplicar a ambos sistemas (nuevo y *legacy*) en paralelo para comparar los resultados y validar las funcionalidades del sistema nuevo. Posteriormente, la documentación del software del sistema *legacy* debe ser actualizada, reescrita o reemplazada durante esta fase para que sean aplicables al nuevo sistema y que contengan la información necesaria para su operación y mantenimiento.

2.3. Estado del Arte de Tecnologías Web *FrontEnd*

Se entienden por tecnologías *FrontEnd* a todas aquellas tecnologías web que son ejecutadas en el navegador web del lado del cliente, en el esquema de cliente/servidor. Esta ejecución puede ocurrir en cualquier tipo de dispositivo, ya sean computadores de escritorio, laptops, tabletas y/o teléfonos inteligentes. A continuación se presenta un breve estado del arte de las tecnologías para desarrollo web de aplicaciones *FrontEnd* más populares al momento de realizar este Trabajo Especial de Grado y otras tecnologías complementarias.

2.3.1. *AngularJS*

AngularJS es un *framework* estructural de desarrollo de aplicaciones web dinámicas que usa tecnologías del lado del cliente (*HTML*, *CSS* y *Javascript*). Fue un *framework* desarrollado en el año 2012 por Google y mantenido hasta Septiembre del 2016 para el desarrollo de aplicaciones web de una sola página (*SPA* por sus siglas en inglés) donde no se realiza una carga o recarga completa de la página sino de secciones específicas usando técnicas como *AJAX*. Este *framework* utiliza el patrón de diseño Modelo-Vista-Controlador o *MVC* [2].

2.3.2. *Angular 2*

Angular 2, actualmente conocido solo como *Angular* es un *framework* de código abierto para el desarrollo de aplicaciones web complejas concebido por *Google* en Septiembre del 2016 y mantenido hasta el día de hoy. Esta versión de *Angular* fue diseñada como una versión completamente reescrita de *AngularJS*. Está basado en un lenguaje de programación *TypeScript*, que es un super conjunto fuertemente tipado de *JavaScript*. A diferencia de *AngularJS*, *Angular* fue diseñado con la capacidad de desarrollo para múltiples plataformas: web, móvil y computadores de escritorio.

2.3.3. *React*

React es una biblioteca creada para construir interfaces de usuarios rápidas e interactivas para aplicaciones móviles y web. Es una biblioteca *FrontEnd* de código abierto, basada en componentes responsable solo por la capa de vista de una aplicación. En una aplicación que usa el patrón *MVC*, la capa de vista es la responsable de la apariencia e interacción de la aplicación. *React* fue creado por Jordan Walke, un ingeniero de software de *Facebook* en Mayo del 2013. Actualmente *React* cuenta con la comunidad y ecosistema de mayor tamaño entre las librerías *FrontEnd* más populares.

2.3.4. *VueJS*

VueJS o *Vue* es un *framework* progresivo diseñado para construir interfaces de usuarios. A diferencia de otros *frameworks* monolíticos, *Vue* fue diseñado desde cero para ser incremen-

talmente adoptable. La biblioteca principal está enfocada solamente en la capa de Vista (del patrón *MVC*) y es fácil de comprender e integrar con otras bibliotecas o en proyectos existentes. Por otra parte, *Vue* es perfectamente capaz de potenciar aplicaciones sofisticadas y complejas *SPA* cuando se utiliza en combinación con otras herramientas modernas y bibliotecas de apoyo. *Vue* fue creado por Evan You, un ex empleado de *Google*, en Febrero de 2014 [10].

En proyectos *SPA*, las partes funcionales de la aplicación se conocen como componentes de *Vue*. Cada componente de *Vue* es una instancia de la biblioteca principal y se divide en 3 partes principales: La plantilla *HTML* (delimitada por las etiquetas `<template>`/`</template>`), los atributos de la instancia y sus métodos descritos en *JavaScript* (delimitados por las etiquetas `<script>`/`</script>`) y finalmente las reglas de estilos *CSS* (delimitadas por las etiquetas `<style>`/`</style>`). A continuación se presenta como ejemplo el Código 2.1 que describe un componente de *Vue* sencillo que retorna el saludo “Hola Mundo!” en color rojo. Cada componente de *Vue* es escrito en su propio archivo y poseen el sufijo “.vue”.

```
1 <template>
2   <div>{{ mensaje }}</div>
3 </template>
4
5 <script>
6 export default {
7   data () {
8     return {
9       mensaje: "Hola Mundo!"
10    }
11  }
12 }
13 </script>
14
15 <style>
16   div {
17     color: red;
18   }
19 </style>
```

Código 2.1: Ejemplo de Componente de *Vue*

2.3.5. *Vuex*

Vuex es una biblioteca de apoyo que ofrece un patrón de manejo de estado para aplicaciones *Vue*. *Vuex* hace las veces de un almacén centralizado de datos para todos los componentes de una aplicación, con reglas que aseguran que el estado solo pueda ser mutado de manera predecible. *Vuex* permite lidiar con los problemas más comunes que surgen cuando múltiples componentes de *Vue* poseen un estado en común, separando todas las propiedades compartidas de la capa de vista (componentes de *Vue*) en un *Singleton* global, el cual es accedido o alterado por cualquier componente [11].

Las partes principales de una instancia de *Vuex* son el estado, los métodos *getters*, las mutaciones y las acciones. El estado es un objeto único que contiene todas las propiedades y datos compartidos entre todos los componentes de *Vue* de la aplicación. Los métodos *getters*

son propiedades calculadas en base al estado, por ejemplo, si el estado posee una lista de objetos, un método *getter* podría retornar la lista filtrada de objetos en base a un parámetro proporcionado sin afectar el estado de forma alguna. Las mutaciones son similares a los eventos de *JavaScript*, donde cada mutación posee funciones manejadoras utilizadas para alterar una propiedad particular del estado que deben ser atómicas y síncronas, y para ejecutar la lógica de las funciones manejadoras se debe utilizar el método **commit**(⟨nombre de mutación⟩) de la instancia de *Vuex*. Las acciones son las piezas de código despachadas por los componentes de *Vue* que nos permiten realizar *commits* de mutaciones y operaciones asíncronas arbitrarias, por ejemplo, realizar una petición a una *API* externa a la aplicación. En la Figura 2.9 se presenta un esquema de la arquitectura de una aplicación *Vue* que utiliza *Vuex*.

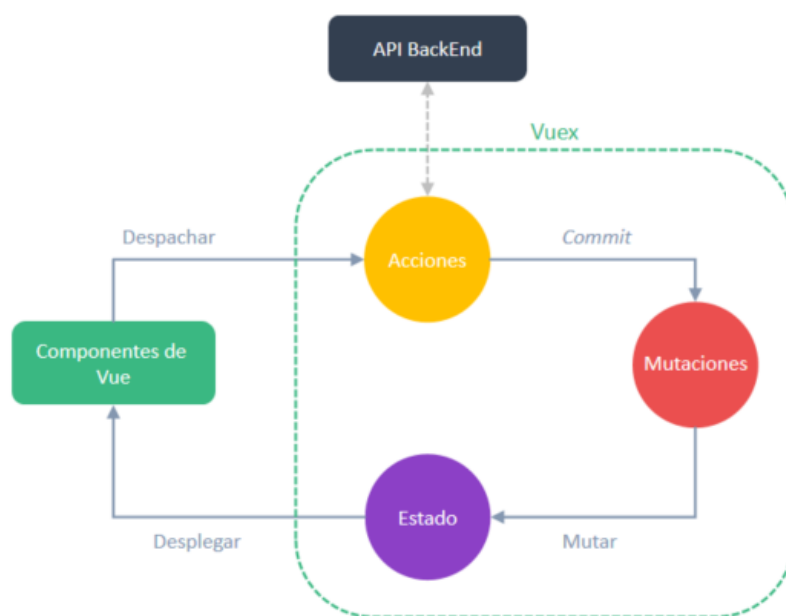


Figura 2.9: Estructura de Aplicación de *Vue* con *Vuex* [11]

2.3.6. *Vue Router*

Vue Router es la solución de enrutamiento oficial de *Vue*, capaz de permitir el desarrollo rápido de *SPA* gracias a su profunda integración con la biblioteca principal de *Vue*. Entre sus funcionalidades se cuentan rutas anidadas, configuración del modular del enrutador, basada en componentes de *Vue*, parámetros de rutas, controles de navegación granular, carga perezosa (del inglés *lazy loading*) de componentes, entre otras [12].

2.3.7. *Bootstrap*

Bootstrap es uno de los *framework* de *CSS* de código abierto más populares y utilizados en el mundo, diseñado para crear sitios web responsivos y aplicaciones web de forma rápida y eficiente. Además, posee una colección de piezas de código reutilizables escritas en *HTML*,

CSS y *JavaScript* que permiten agilizar el desarrollo de aplicaciones *FrontEnd* mediante un prototipado rápido de las interfaces, listas para ser desplegadas en dispositivos de cualquier tamaño de pantalla debido a su naturaleza responsiva [13].

2.3.8. *BootstrapVue*

BootstrapVue es una biblioteca de apoyo y código abierto para el *framework* de desarrollo de aplicaciones *FrontEnd*, *Vue*, que provee un conjunto de componentes de interfaces de usuario (*UI* en inglés) de *Bootstrap* reescritos como componentes de *Vue*, listos para ser importados, parametrizados y utilizados en un proyecto sin necesidad de instalar otras dependencias de *Bootstrap*, como *jQuery*. Los componentes de *BootstrapVue* son enteramente responsivos, es decir que están optimizados para ser desplegados en cualquier tipo de dispositivo. Son modulares, lo cual implica que solo se importan los componentes utilizados, reduciendo el tamaño de los archivos compilados generados. Generan automáticamente código *HTML* accesible para usuarios con discapacidades sensoriales. Además son fácilmente personalizables mediante el uso de variables globales [14].

2.4. Tecnologías Web *BackEnd* Relevantes

Se entienden por tecnologías *BackEnd* a todas aquellas tecnologías web que son ejecutadas en el servidor, el cual escucha peticiones de clientes y sirve los archivos apropiados. A continuación se describen brevemente las tecnologías para desarrollo web de aplicaciones *BackEnd* relevantes a este Trabajo Especial de Grado.

2.4.1. *Node.js*

Node.js es un ambiente de ejecución de *JavaScript*, diseñado para construir aplicaciones de redes escalables. Las aplicaciones de *Node.js* pueden ser ejecutadas en distintas plataformas como *MacOS*, *Microsoft Windows* y *Linux*. *Node.js* funciona como un ambiente totalmente asíncrono basado en eventos no bloqueantes, lo cual permite realizar gran cantidad de operaciones de entrada/salida en un intervalo relativamente corto de tiempo, en parte también debido a sus optimizaciones para realizar y recibir peticiones *HTTP* [15].

2.4.2. *NPM (Node Package Manager)*

NPM son las siglas en inglés de Manejador de Paquetes de *Node*, este cumple con dos objetivos principales: primero, *NPM* es un repositorio en línea para la publicación de proyectos de código abiertos basados en *Node.js* y segundo, es una utilidad para consolas de comandos para interactuar con dicho repositorio, esta utilidad facilita la instalaciones de paquetes, manejo de versiones y finalmente de manejo de dependencias entre paquetes [16].

2.4.3. *Express.js*

Express.js es un *microframework* escrito en *JavaScript* para el desarrollo de aplicaciones web usando *Node.js*, que provee un robusto conjunto de funcionalidades para aplicaciones web y móviles. Permite la creación fácil y rápida de *APIs* gracias a la gran cantidad de métodos utilitarios *HTTP* y de *middlewares* puestos a disposición del programador. Además *Express.js* no toma muchas decisiones por el desarrollador, tales como qué base de datos utilizar. La estructura y funcionalidades adicionales dependen de las necesidades y/o casos de uso del desarrollador [17].

2.4.4. *Knex.js*

Knex.js es un constructor de sentencias y peticiones *SQL* de código abierto con “baterías incluidas” para las bases de datos *PostgreSQL*, *MSSQL*, *MySQL*, *MariaDB*, *SQLite3*, *Oracle*, and *Amazon Redshift*. Está diseñado para ser flexible, portable y fácil de aprender y usar. Entre sus funcionalidades posee las capacidades de: Utilizar el sistema de llamadas a funciones de vuelta tradicional de *Node.js* o también una interfaz que permite un control más prolijo del flujo asíncrono, constructores de esquemas y sentencias completos, control de conexiones a las bases de datos y respuestas estandarizadas entre diferentes dialectos y clientes de sentencias y peticiones [18]. *Knex.js* ofrece una capa de abstracción para que el programador sólo escriba código en lenguaje *JavaScript* para realizar peticiones a bases de datos, abstrayéndose de los detalles particulares de cada una de las bases de datos mencionadas.

2.4.5. *Bookshelf.js*

Bookshelf.js es un *ORM* (siglas del inglés *Object-Relational Mapper*) escrito en el lenguaje *JavaScript* para *Node.js*, utilizando el constructor de sentencias *SQL*, *Knex.js*. Su objetivo principal es proveer una librería que facilite las tareas comunes al realizar peticiones a bases de datos en *JavaScript* mediante la definición de modelos y las relaciones entre estos. Entre sus funcionalidades se encuentran la carga temprana y anidada de objetos relacionados, soporte para transacciones, asociaciones polimórficas y soporte para relaciones uno-a-uno, uno-a-muchos y muchos-a-muchos. Además permite bajar el nivel de abstracción al utilizar *Knex.js* directamente para realizar peticiones *SQL* personalizadas que no se encuentran implementadas en *Bookshelf.js*. Está diseñado para trabajar con las bases de dato *PostgreSQL*, *MySQL* y *SQLite3* [19].

Capítulo 3

Marco Aplicativo

En este capítulo se definen las metodologías de mantenimiento y reingeniería del software utilizadas y se describen las actividades que tomaron lugar durante sus implementaciones, como el análisis de las diferentes partes del sistema *PortalAsig2*, actualización de su documentación, rediseño realizado, entre otras.

3.1. Metodología de Mantenimiento

Para realizar las labores de mantenimiento de software contenidas en este Trabajo Especial de Grado, se requirió definir o adaptar una metodología de mantenimiento al problema para su posterior implementación. La metodología empleada fue una adaptación del Proceso de Mejora Iterativo descrito en la Sección 2.1.3.2. A continuación se describen las adaptaciones realizadas:

- El rol de Cliente fue ejercido por el tutor principal de este Trabajo Especial de Grado, el cuál tuvo las responsabilidades de explicar las fallas detectadas en el sistema, proponer una serie de nuevas funcionalidades y finalmente juzgar las soluciones implementadas como satisfactorias o no.
- El rol del Equipo de Mantenimiento fue compuesto únicamente por el estudiante autor de este Trabajo Especial de Grado. Sus responsabilidades incluyeron, pero no se limitaron a, el análisis del sistema, la corrección de los diagramas de la documentación, el diseño de las modificaciones a implementar y la realización de pruebas técnicas.
- La duración de las iteraciones estuvo sujeta a la implementación de una o varias modificaciones del sistema *PortalAsig2*: corrección de fallas, adaptaciones al ambiente e implementación de nuevas funcionalidades. En vez de un límite de tiempo arbitrario, sin embargo, las actividades descritas en este Trabajo Especial de Grado fueron planeadas para ser implementadas en un período no mayor a un semestre de clases o aproximadamente 16 semanas.
- La única iteración que posee la fase de clasificación de modificaciones propuestas será la primera, debido a que esta implementación del Proceso de Mejora Iterativo posee un

tiempo limitado y las posibles modificaciones a realizar en el sistema fueron definidas y delimitadas a priori entre el Cliente y el Equipo de Mantenimiento. El resto de las iteraciones solo estarán compuestas por las fases de análisis del sistema actual y de rediseño e implementación.

- Los entregables de cada iteración consisten de los diagramas actualizados de la documentación y el código fuente modificado, este último fue desplegado al servidor de operación del Centro de Computación de la Facultad de Ciencias al finalizar cada iteración.

El Proceso de Mejora Iterativo fue escogido por los siguientes motivos:

- Un requisito fundamental para su implementación es una documentación completa, criterio que se cumple en el caso de *PortalAsig2*. Esta incluye diagramas de casos de uso y sus especificaciones, diagramas de clase, diagramas de despliegue, diagramas de estructura del sistema, guía de estilo e interfaces y los comentarios anotados en el código fuente. A pesar de esto, durante las actividades de mantenimiento se procedió a agregar nueva documentación o corregir la existente con detalles que se consideraron pertinentes.
- Se requería una metodología que permitiera responder rápidamente a las condiciones del ambiente de producción, puesto que *PortalAsig2* es una plataforma en uso activo.
- El uso de iteraciones permitió priorizar adecuadamente las modificaciones a realizar y controlar la cantidad de nuevo código que era desplegado en el servidor de producción.

3.2. Iteración 1

Esta primera iteración describe el análisis inicial del sistema, la corrección de varios diagramas de la documentación, las modificaciones propuestas y cómo se clasificaron según su prioridad, así como sus implementaciones. El foco de atención de esta iteración fue la aplicación *FrontEnd* del sistema. Debido a un problema crítico descubierto durante el análisis, se requirió definir e implementar una metodología de reingeniería del software, cuyas actividades también se incluyen y describen como parte de esta iteración. El producto de software de la reingeniería realizada, sirvió como base fundamental para la próxima iteración.

3.2.1. Análisis del Sistema Actual

El análisis comenzó con la recolección y revisión de la toda la documentación disponible [2] y la inspección del código fuente del sistema para facilitar la comprensión de su estructura, reglas de negocio y funcionamiento general por parte del Equipo de Mantenimiento. *PortalAsig2* es un sistema que permite la creación de sitios de asignaturas dictadas en la Facultad de Ciencias de la Universidad Central de Venezuela. Cada sitio está asociado con un semestre de clases y posee un conjunto de secciones que se encargan de desplegar información y funcionalidades diferentes basadas en el rol que el usuario posea en dicho sitio. Existen tres tipos de cuentas de usuarios a nivel de base de datos: Administradores, Docentes y Estudiantes. Cada uno

puede tener uno de los siguientes roles en un sitio de asignatura (con algunas excepciones): Usuario Anónimo, Usuario Autenticado, Estudiante, Grupo Docente, Docente, Coordinador y Administrador. A continuación se explican cada uno de los roles mencionados, sus privilegios y las responsabilidades que poseen dentro de un Sitio en el que estén registrados:

Usuario Anónimo

Este rol representa a todos los usuarios que no posean una cuenta registrada en el sistema o aquellos que no se han autenticado en el mismo iniciando una sesión, así posean una cuenta. Estos usuarios poseen los siguientes permisos:

- Consultar las asignaturas disponibles.
- Iniciar sesión si poseen una cuenta registrada en el sistema.
- Visitar sitios de asignaturas disponibles.
- Consultar o leer los siguientes detalles de un sitio: objetivos, bibliografía, contenidos temáticos, grupo docente, horarios de clases, evaluaciones, planificación, noticias, descargas y sitios de semestres anteriores.
- Descargar los archivos disponibles en la sección de descargas.

Usuario Autenticado

Cualquier tipo de cuenta de usuario desempeña este rol cuando se autentica en el sistema iniciando una sesión. Estos usuarios tienen los siguientes permisos:

- Todos los permisos del Usuario Anónimo.
- Cerrar o terminar su sesión en el sistema.
- Consultar cuáles son los sitios de asignaturas en los que están registrados.
- Editar detalles de su cuenta de usuario, en particular: contraseña, correo electrónico y sus preferencias para correos de notificación.

Estudiante

Las cuentas de usuario Estudiante desempeñan este rol en un sitio cuando un Docente los registra. Una vez inician sesión, tienen los siguientes permisos en un sitio de asignaturas:

- Todos los permisos del Usuario Autenticado.
- Consultar, agregar y responder en los foros del sitio.
- Consultar calificaciones de las evaluaciones.

- Consultar entregas planificadas, subir archivos para completar la entrega, editar y revisar la entrega hecha.
- Recibir correos de notificación (opcional) cada vez que se agregan o editan noticias, evaluaciones, calificaciones, descargas, foros y entregas.

Grupo Docente

Las cuentas de usuario Docente y Estudiante desempeñan este rol en un sitio cuando un Docente los agrega, en el caso de los estudiantes, estos obtienen el título de Preparador tipo I o II. Una vez inician sesión, tienen los siguientes permisos en un sitio de asignaturas:

- Todos los permisos del Usuario Autenticado.
- Consultar, agregar, responder y eliminar los foros del sitio.
- Consultar, agregar, editar entregas planificadas así como revisar y descargar los archivos subidos por los estudiantes.
- Enviar un correo electrónico personalizado a uno o más usuarios registrados en el sitio.
- Agregar y eliminar descargas.
- Agregar, editar y eliminar noticias.
- Consultar y editar calificaciones de las evaluaciones de todos los estudiantes registrados en el sitio.
- Agregar nuevos eventos en la planificación o editar los existentes.

Docente

Las cuentas de usuario Docente que son parte del grupo docente de un sitio poseen los siguientes permisos adicionales una vez inician sesión:

- Todos los permisos del Grupo Docente.
- Agregar un nuevo sitio de asignatura.
- Consultar, agregar, editar y eliminar evaluaciones, horarios de clase, integrantes del grupo docente, los estudiantes registrados, los contenidos temáticos, la bibliografía y los objetivos.
- Configurar el comportamiento de las noticias de un sitio (si utilizan contenido de *Twitter* o no).

Coordinador

Las cuentas de usuario Docente que agregan un nuevo sitio de asignatura quedan automáticamente asignadas como Coordinador en dicho sitio, solo existe un usuario con el rol de Coordinador en un sitio a la vez. Una vez inician sesión, tienen los siguientes permisos en un sitio de asignaturas:

- Todos los permisos de Docente.
- Consultar el registro de actividades del sitio, el cual mantiene una entrada por cada acción que se considere importante y qué usuario la realizó, por ejemplo, edición de una calificación.
- Eliminar el sitio de asignatura.
- Ceder el rol de Coordinador a otra cuenta de usuario docente registrada en el sitio.

Administrador

Las cuentas de usuario Administradores son las únicas cuentas que pueden ejercer este rol. Una vez inician sesión, tienen los siguientes permisos en un sitio de asignaturas:

- Todos los permisos de Usuario Autenticado.
- Consultar, agregar, editar y eliminar asignaturas, cuentas de usuario Estudiante y cuentas de usuario Docente.

Durante el análisis, se detectaron deficiencias o incongruencias en varios de los diagramas de la documentación que fueron corregidas utilizando como base de comparación el código fuente y las reglas de negocio previamente descritas. En el primer Apéndice se presentan los diagramas de la documentación actualizados que representan con mayor fidelidad al sistema *PortalAsig2* y que facilitaron las actividades de rediseño e implementación posteriores. A continuación se describen en términos generales los cambios realizados por cada tipo de diagrama:

1. Diagramas de Casos de Uso:

- Cambio de Verbos: Esto permitió mejorar la precisión de la representación de las acciones que un usuario puede realizar en el sistema y reducir la ambigüedad, por ejemplo, el caso de uso “4.1 Ver semestres anteriores” fue cambiado a “UC4.1.9. Consultar Sitios de Semestres Anteriores” con su respectiva extensión del caso de uso “UC4.1.9.1 Visitar Sitio de Semestre Anterior” (Figura A.1) para indicar que se puede no solo consultar o “ver” un sitio sino visitarlo también.
- Reordenamiento de los casos de uso: Se alteró la numeración de los casos de uso para utilizar un orden intuitivo que corresponde con la navegación implementada en el sistema. Adicionalmente algunos casos de uso de extensión fueron colocados como casos de uso principales o viceversa, por ejemplo, “1.1. Recuperar Contraseña” que era una extensión de “1. Iniciar Sesión” fue actualizado a “UC2. Recuperar Contraseña” porque la recuperación de la contraseña no es opcional o depende del inicio de sesión, sino una acción que el usuario puede realizar en cualquier momento.

- Uso del delimitador del sistema: Se agregó a todos los diagramas un delimitador de sistema etiquetado *PortalAsig2* que encapsula todos los casos de uso y los separa limpiamente de los usuarios.
- Generalización de Actores: Se agregaron relaciones de herencia entre diferentes tipos de actores (roles de usuarios), por ejemplo, un usuario autenticado hereda de un usuario anónimo, con el fin de señalar que un usuario autenticado también interactúa con todos los casos de uso de un usuario anónimo. Esto no era evidente en los diagramas originales [20].
- Corrección de casos de uso por roles de usuario: Varios casos de uso que estaban relacionados sólo con un rol, también estaban relacionados con otros roles según el código fuente y las reglas de negocio extraídas de este, por ejemplo, los usuarios Docente tienen la permisología para Administrar horarios, contenidos temáticos, estudiantes y otros docentes, sin embargo, estos casos de uso no estaban reflejados en los diagramas.

2. Diagrama de Entidad-Relación:

- Agregar las entidades y relaciones que faltan: El diagrama entidad-relación disponible en [2] no representaba todas las entidades existentes en el sistema ni las relaciones entre estas.

3. Diagramas de Componentes de las aplicaciones *FrontEnd* y *BackEnd*:

- Estos diagramas no formaban parte de la documentación original, sin embargo, fueron generados como complemento del diagrama de entidad-relación para apreciar a simple vista la estructura y envergadura de ambas aplicaciones.

Acto seguido, se procedió a establecer un entorno local de desarrollo en la terminal de trabajo del equipo de mantenimiento para analizar las aplicaciones en funcionamiento, cuando se hicieron las siguientes observaciones:

1. La aplicación *BackEnd* (desarrollada en *Express.js*) y la aplicación *FrontEnd* (desarrollada en AngularJS), que conforman el sistema de *PortalAsig2*, poseen dependencias de software desactualizadas, anticuadas o sin mantenimiento. Esto era de esperarse puesto que no se habían realizado labores de mantenimiento en el sistema *PortalAsig2* desde su puesta en operación en el año 2016.
2. La aplicación *BackEnd* logró ser ejecutada de forma correcta en el entorno local.
3. A pesar de seguir la documentación del repositorio del código y utilizar diferentes métodos, no se pudo ejecutar la aplicación *FrontEnd* en un entorno local. Por lo que el resto del análisis se basó en el funcionamiento del sistema en producción para hallar otras posibles fallas.

3.2.2. Clasificación de las Modificaciones Propuestas

A partir de los problemas descubiertos en la fase de análisis, el funcionamiento del sistema en producción y los requerimientos solicitados por el cliente, se definió una lista con las modificaciones a realizar sobre el sistema *PortalAsig2*. Luego, estas fueron clasificadas en base a 3 parámetros: según su complejidad técnica en una escala del 1 (muy baja) al 5 (muy alta), cuáles partes del sistema afectarían al ser implementadas (*FrontEnd*, *Backend* o ambas) y su tipo. Las modificaciones y sus clasificaciones son ilustradas en la Tabla 3.1.

Modificaciones	Complejidad Técnica	Partes Afectadas	Tipo
Recrear la aplicación <i>FrontEnd</i> para ejecutarla correctamente en un entorno local	5	<i>FrontEnd</i>	Corrección de Fallas
Cambiar la implementación del <i>widget</i> de <i>Twitter</i> para presentar las noticias de un sitio de asignaturas	3	<i>FrontEnd</i>	Corrección de Fallas
Rediseñar la interfaz de usuario (<i>UI</i> en inglés) para la administración de calificaciones de estudiantes	4	<i>FrontEnd</i>	Corrección de Fallas
Evitar cambios en el orden de despliegue al editar objetivos, contenidos temáticos y bibliografías de un sitio	2	<i>FrontEnd</i>	Corrección de Fallas
Limitar el despliegue de fechas al formato: DD/MM/AAAA	3	<i>FrontEnd</i>	Corrección de Fallas
Agregar nueva funcionalidad de encuestas y quices con preguntas de selección simple o múltiple	5	Ambas	Nueva funcionalidad
Otorgar al rol Administrador todos los permisos del rol Coordinador, para todos los sitios	3	Ambas	Adaptación a Nuevas Reglas de Negocio
Agregar opción para ocultar algunas secciones de estudiantes en partes de un sitio	4	Ambas	Corrección de Fallas
Agregar nueva sección de cursos académicos no acreditados en la página principal	1	<i>FrontEnd</i>	Nueva funcionalidad
Buscar estudiantes y docentes por nombre o apellidos	3	Ambas	Nueva funcionalidad
Eliminar permiso del rol Docente para agregar nuevas asignaturas	1	Ambas	Adaptación a Nuevas Reglas de Negocio

Desplegar cuerpo de noticias completo en la lista de noticias	2	<i>FrontEnd</i>	Nueva funcionalidad
Incorporar texto enriquecido al cuerpo de noticias	2	<i>FrontEnd</i>	Nueva funcionalidad
Agregar cuerpo de noticias a los correos de notificación	2	<i>BackEnd</i>	Nueva funcionalidad

Tabla 3.1: Clasificación de Modificaciones Propuestas

Debido a su naturaleza bloqueante, se definió la siguiente modificación como objetivo principal de esta primera iteración:

Recrear la aplicación *FrontEnd* para ejecutarla correctamente en un entorno local.

Además, por sólo afectar la aplicación *FrontEnd* y corregir fallas presentes en el sistema, también se implementarán las siguientes modificaciones en esta iteración:

- Cambiar la implementación del *widget* de *Twitter* para presentar las noticias de un sitio de asignaturas.
- Rediseñar la *UI* para la administración de calificaciones de estudiantes.
- Limitar el despliegue de fechas al formato: DD/MM/AAAA
- Evitar cambios en el orden de despliegue al editar objetivos, contenidos temáticos y bibliografías de un sitio.

Para cumplir con el objetivo principal definido para esta iteración, se acordó entre el equipo de mantenimiento y el cliente, la definición e implementación de una metodología de reingeniería del software para recrear la aplicación *FrontEnd* del sistema *PortalAsig2* con tecnologías modernas para, en iteraciones posteriores, implementar el resto de las modificaciones. La metodología implementada para llevar a cabo el proceso de reingeniería es descrita en la próxima sección.

3.2.2.1. Metodología de Reingeniería

La metodología implementada para llevar a cabo las actividades de reingeniería, está basada en las tareas y fases de la reingeniería del software explicadas en la Sección 2.2.7 con las siguientes modificaciones:

- El equipo de reingeniería estuvo conformado por el mismo equipo de mantenimiento, es decir, compuesto únicamente por el estudiante autor de este Trabajo Especial de Grado.

- En la fase del estudio de viabilidad del proyecto, no se tomaron en cuenta cifras de costos monetarios dado que las actividades de reingeniería se desarrollaron en un entorno académico sin objetivos de lucro.
- En la fase de implementación de la reingeniería, solo fue necesario recorrer los dos niveles más bajos de abstracción, diseño e implementación, debido a que el concepto de la aplicación y sus requerimientos no fueron alterados.

Cabe destacar que en el contexto de la reingeniería, la aplicación *FrontEnd* del sistema *PortalAsig2* será referida como sistema *legacy* y la nueva aplicación *FrontEnd* recreada será referida como nuevo sistema.

3.2.3. Rediseño e Implementación

A continuación se describen en orden secuencial las implementaciones de cada una de las fases de la metodología de reingeniería definida y aplicada, así como las actividades más importantes de cada fase:

3.2.3.1. Estudio de Viabilidad del Proyecto

La viabilidad del proyecto de reingeniería del sistema *legacy* dependió enteramente de si este podía ser completado en un tiempo definido, no se consideró el lado monetario/económico debido al ámbito académico de este Trabajo Especial de Grado. A continuación se describen varios aspectos que impactan la viabilidad del proyecto desde el punto de vista operacional:

- El desarrollo y finalización de este proyecto es de fundamental importancia para la realización del resto de las actividades de mantenimiento del sistema *PortalAsig2*.
- El tiempo máximo definido para completar el proyecto de reingeniería es ocho (8) semanas.
- El equipo de reingeniería cuenta con una estación de trabajo adecuada y la experiencia necesaria para realizar las tareas de análisis, implementación y desarrollo.
- El equipo de reingeniería requiere entrenamiento en el uso y manejo de la tecnología a utilizar para implementar el nuevo sistema. Tiempo estimado de tres(3) a seis(6) días.
- Se realizó un trabajo previo de análisis y actualización de la documentación relevante.
- No se requiere invertir esfuerzos en rediseñar la mayoría de la *UI* y la experiencia de usuario (*UX* por sus siglas en inglés), lo cual reduce el tiempo de desarrollo considerablemente.

En base a los aspectos descritos anteriormente y la experiencia del equipo de reingeniería se determinó que el proyecto no solo es viable sino necesario, dando lugar al comienzo de las actividades de análisis y planificación.

3.2.3.2. Análisis y Planificación

Esta fase se compone de tres subfases: Análisis del Sistema *Legacy*, Especificación de las Características y Requerimientos del Nuevo Sistema y Creación de un Conjunto de Pruebas, las cuales se describen a continuación.

- **Análisis del Sistema *Legacy*:** La mayoría de las actividades de esta subfase, como la recolección y análisis de toda la documentación disponible y de especificación de condición del sistema coincidieron con las actividades realizadas en la Sección 3.2.1. Debido a esto, se procede a especificar las características técnicas del sistema *legacy* en la Tabla 3.2.

Tecnología	Versión
<i>Node.js</i>	4.2.6
<i>AngularJS</i>	1.3
<i>Bootstrap</i>	3
Sintaxis <i>ES</i>	5

Tabla 3.2: Versiones de Tecnologías del Sistema *Legacy*

A continuación se describen los beneficios a obtener como retorno de la inversión hecha al completar el proyecto de reingeniería del sistema *legacy*:

- Un nuevo sistema actualizado, modularizado y con capacidad de expansión gracias a futuras mejoras y adiciones de funcionalidades.
- El proyecto de reingeniería reducirá la complejidad y el esfuerzo para comprender y modificar el software y mejorará la documentación.

Además de estas ventajas, se estimó que el uso del sistema *PortalAsig2* se vería intensificado como consecuencia de las circunstancias globales y nacionales al momento de realizar el desarrollo de este Trabajo Especial de Grado, causadas por la pandemia del virus SARS-CoV-2, la cual promueve el aprendizaje remoto. Este aumento de su uso multiplica el valor agregado al sistema de cualquier corrección o adición hecha gracias a la culminación exitosa de este proyecto de reingeniería.

- **Especificación de las Características y Requerimientos del Nuevo Sistema:** Para llevar a cabo el proyecto de reingeniería se definió el enfoque de reingeniería a utilizar, las características técnicas del nuevo sistema y cuáles eran los objetivos a cumplir.

El enfoque de reingeniería seleccionado fue *Big Bang*, debido a la necesidad inmediata de resolver un problema: “Recrear la aplicación *FrontEnd* para ejecutarla correctamente en un entorno local” y la necesidad secundaria de mantener el sistema *legacy* y el nuevo en operación de forma paralela para compararlos y validar la correcta transferencia de funcionalidades entre sistemas.

A continuación se listan en la Tabla 3.3 las características técnicas que el nuevo sistema debe tener:

Tecnología	Versión
<i>Node.js</i>	12.16.3
<i>Vue.js</i>	2.6.10
<i>Vuex</i>	3.1.3
<i>Vue-Router</i>	3.16
<i>BootstrapVue</i>	2.16+
Sintaxis <i>ES</i>	7

Tabla 3.3: Versiones de Tecnologías del Nuevo Sistema

La característica más resaltante del nuevo sistema es el uso de un nuevo *framework*, *Vue*, para la implementación del nuevo sistema. Esta decisión fue motivada por el estado actual del *framework AngularJS*, el cual se encuentra sin mantenimiento oficial de *Google*, en desuso por la mayoría de la industria del desarrollo de aplicaciones web y cuyo uso prolongado podría seguir dificultando labores de mantenimiento a futuro. Para reemplazar *AngularJS* se analizaron tres opciones principales: *React*, *Vue* y *Angular*, donde las características más importantes a considerar fueron la facilidad de aprendizaje y la facilidad de prototipado disponibles. *Vue* fue seleccionado como el *framework* a utilizar dado que supera a las otras dos opciones en las características mencionadas [21] y además cuenta con todas las herramientas y bibliotecas de apoyo necesarias para implementar exitosamente el nuevo sistema.

Los objetivos del proyecto de reingeniería incluyen:

- Preservar las funcionalidades intactas del sistema *legacy*.
 - Incorporar las modificaciones correctivas al sistema *legacy*.
 - Preservar la mayor parte de la *UI* y *UX* posible para minimizar la resistencia al cambio de los usuarios.
 - Reducción del 20 % de la complejidad ciclomática de McCabe y del esfuerzo del programa de Halstead.
 - Descomponer los controladores *JavaScript* y las vistas *HTML* del sistema *legacy* en componentes de *Vue* modulares, livianos y reutilizables.
- **Creación de un Conjunto de Pruebas:** Como el enfoque *Big Bang* seleccionado requiere que ambos sistemas (*legacy* y nuevo) estén operativos de forma paralela para validar que la funcionalidad del nuevo sistema se mantuvo constante, y que la mayoría de la lógica del negocio es ejecutada en la aplicación *BackEnd* y no en la *FrontEnd*, se determinó que el conjunto de pruebas apropiado para garantizar y validar que la funcionalidad no varió entre sistemas consiste de una serie de pruebas de alto nivel de caja negra donde se ingresan las mismas entradas en ambos sistemas y luego se comparan las salidas.

3.2.3.3. Implementación de la Reingeniería

La implementación de la reingeniería fue dividida en varias secciones, correspondientes con las divisiones funcionales de los componentes del sistema *legacy* para facilitar las actividades de reingeniería, estas partes fueron:

- Página Principal
- Autenticación y Opciones de Usuarios
- Permisología de Usuarios
- Panel de Administración
- Sitios de Asignaturas
 - Creación y Eliminación de Sitios
 - Navegación entre Secciones de Sitio
 - Información General del Sitio y Contenido Temático
 - Evaluaciones, Horarios
 - Noticias
 - Entregas, Foros y Envío de Correos
 - Grupo Docente y Estudiantes
 - Calificaciones
 - Descargas
 - Planificación

Por cada una de las secciones nombradas anteriormente se aplicaron los procesos de ingeniería inversa, para comprender el código fuente y su diseño como parte de una aplicación *AngularJS*, e ingeniería directa, con el fin de obtener el código fuente del nuevo sistema correspondiente a la sección trabajada con un nuevo diseño, que utilice los componentes de la tecnología *Vue* y que mantenga su funcionalidad constante. Durante dichos procesos solo fue necesario recorrer los siguientes niveles de abstracción: Implementación y Diseño. Esto se debe a que no se requirió ningún repensado del sistema (alteración al nivel conceptual) y los requerimientos se mantuvieron constantes por lo que no ocurrió ninguna reespecificación (alteración al nivel de requerimientos). Esta reducción en los niveles de abstracción a recorrer ayudó a reducir el tiempo de la implementación considerablemente.

Como parte del proceso de ingeniería inversa, además de analizar el código fuente y el diseño, también se realizó el aislamiento y comprensión de cada una de las dependencias externas del sistema *legacy* apoyadas en *AngularJS* que permitieron cumplir con algún requisito. Luego de estudiarlas, se realizó una búsqueda de dependencias análogas, apoyadas en *Vue*, que fuesen lo más semejante visual y funcionalmente posibles a las anteriores para minimizar el impacto visual por cambios a los usuarios y permitir que el nuevo sistema implementado cumpliera con los mismos requerimientos.

El diseño estructural interno de la aplicación tuvo que ser rediseñado para aprovechar las virtudes y ventajas que *Vue* y sus bibliotecas de apoyo ofrecen. Los dos cambios estructurales más importantes fueron, primero, la descomposición de cada una de las vistas *HTML* y su correspondiente controlador escrito en *JavaScript* en diferentes componentes de *Vue* y, segundo, la utilización de un estado central de aplicación (*Vuex*) en vez de mantener un estado por cada controlador activo en la aplicación.

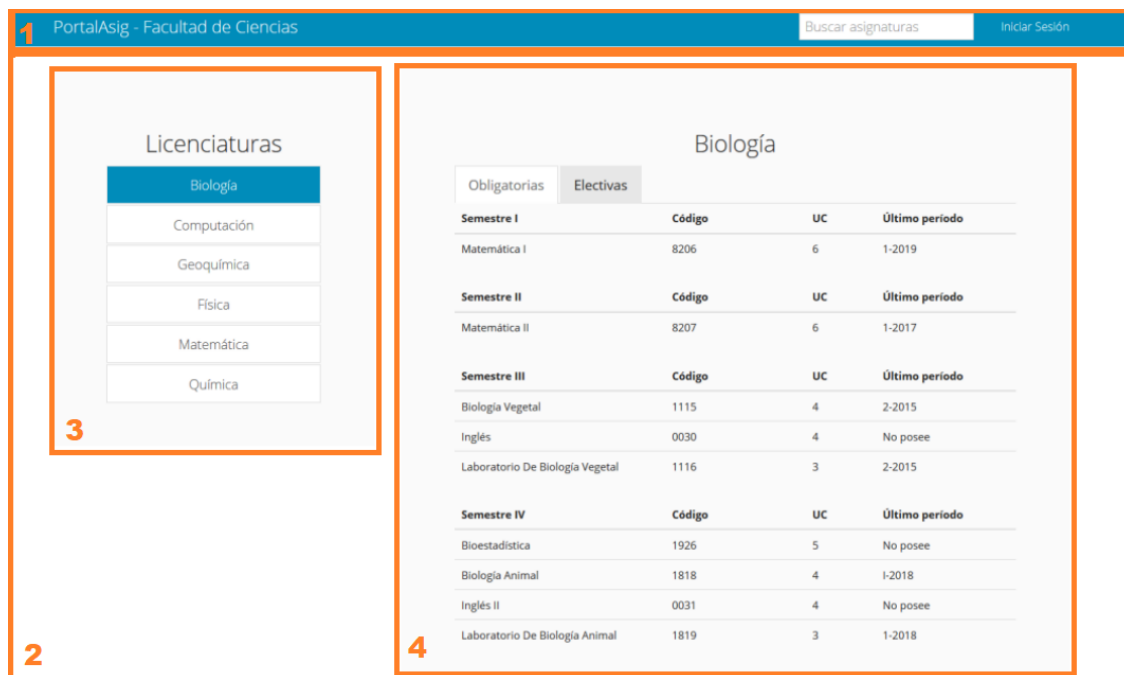


Figura 3.1: Descomposición en Componentes de *Vue* de la Página Principal

En la Figura 3.1 se ilustra la descomposición a componentes de *Vue* realizada a lo largo del proyecto de reingeniería, utilizando la página principal de *PortalAsig2* como ejemplo. Cada uno de los recuadros naranja representa un componente de *Vue*, estos componentes son:

1. Barra de navegación
2. Contenedor de la página principal
3. Selector de licenciaturas
4. Contenedor de listas de asignaturas por tipo y semestres.

Debido al reemplazo de *AngularJS* por *Vue* y el rediseño estructural efectuado, se actualizó el diagrama de componentes del nuevo sistema, este se presenta en la Figura 3.2.

A continuación se describen la implementaciones de cada una de las modificaciones correctivas incorporadas como parte del proyecto de reingeniería, en orden de implementación:

Evitar cambios en el orden de despliegue al editar objetivos, contenidos temáticos y bibliografías de un sitio

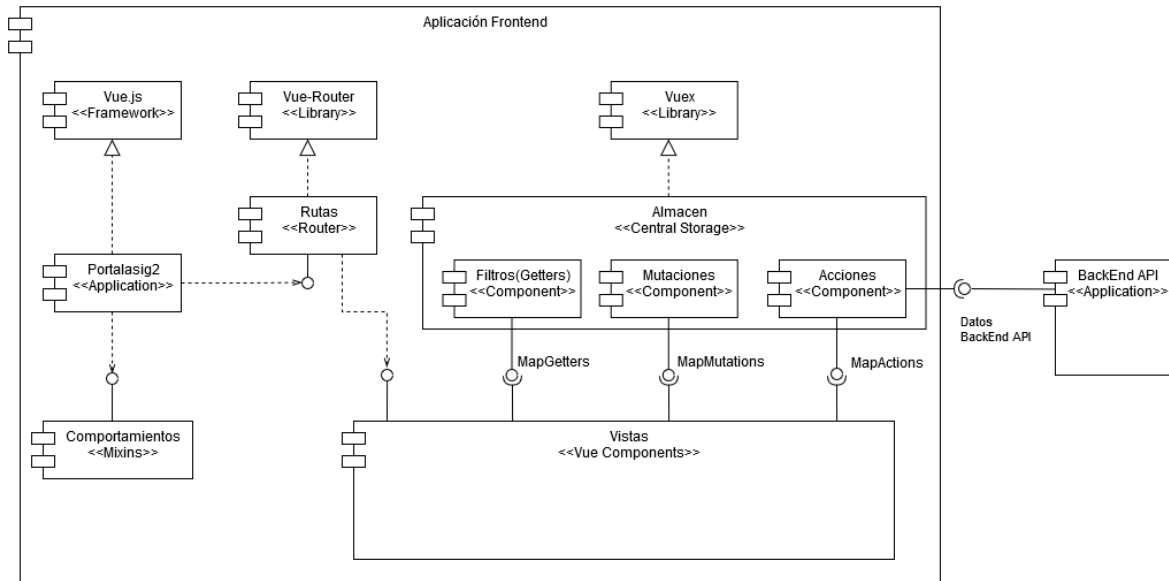


Figura 3.2: Diagrama de Componentes del Nuevo Sistema

En el sistema *legacy* cuando un usuario con rol Docente en un sitio de asignaturas editaba un objetivo, contenido temático o bibliografía, el orden visual de despliegue se alteraba de forma impredecible. Los usuarios del sistema *PortalAsig2* reportaron este comportamiento anómalo para que fuese solucionado. El orden numérico de los objetivos y contenidos temáticos de un sitio de asignaturas estaba determinado por su última fecha de edición lo cual produce los cambios de orden. Para obtener el comportamiento esperado por los usuarios, basta con ordenar los elementos por su fecha de creación previo a su despliegue.

La implementación requirió alterar los métodos *getter* que obtienen y dan formato a los objetivos, contenidos temáticos y bibliografías del estado central de la aplicación, agregando un ordenamiento obligatorio por fecha de creación. En el segmento de Código 3.1 se ilustra un ejemplo de esta implementación realizada en para los objetivos de un sitio, de la línea 5 a la 11 se le aplica un formato de código llamado *camelCase* a las propiedades relevantes de los objetivos y en la línea 13 se les aplica el ordenamiento por fecha de creación antes de retornar los objetos. Esta modificación evita que el orden visual de los objetivos y contenidos temáticos se vea alterado cuando alguno es editado.

```

1  getActiveSiteObjectives (state) {
2    // Return an array of formatted objective objects sorted by created_at
   date
3    if (state.activeSite && state.activeSite.objectives.length) {
4      var objectivesList = []
5      state.activeSite.objectives.map(objective => {
6        objectivesList.push({
7          id: objective.id,
8          description: objective.description,
9          createdAt: objective.created_at
10         })
11     })

```

```

12      // Sort by created at date ascending
13      return objectivesList.sort(function (a, b) {
14          var dateA = dateObjectFromTimestamp(a.createdAt)
15          var dateB = dateObjectFromTimestamp(b.createdAt)
16          return dateA - dateB
17      })
18  }
19  return null
20 },

```

Código 3.1: Método *Getter* de Objetivos Ordenados por Fecha de Creación

Limitar el despliegue de fechas al formato: DD/MM/AAAA

El formato de las fechas en el sistema *legacy* dependía enteramente de la configuración del navegador del usuario, algunos de estos vienen configurados por defecto con el formato de fecha utilizado en norteamérica: MM/DD/AAAA, esto resultó en confusión de algunos usuarios los cuales reportaron esta falla. La solución consiste en obligar el despliegue de las fechas en el formato deseado DD/MM/AAAA para evitar futuras confusiones y quejas por parte de los usuarios.

Para implementar esta modificación se creó una función auxiliar reutilizable (Código 3.2), que fue empleada a lo largo de toda la aplicación a la hora desplegar cualquier fecha. Esta función toma una fecha con tipo cadena de caracteres (línea 1) y le aplica el formato **en-GB** que corresponde a DD/MM/AAAA (línea 5), solo si el navegador del usuario lo permite puesto que no todos poseen esta funcionalidad.

```

1 export function getSpanishLocaleDateFormat (dateString) {
2     // The expected return format is dd/mm/yyyy
3     let dateObj = dateObjectFromTimestamp(dateString)
4     try {
5         return dateObj.toLocaleDateString('en-GB')
6     } catch (error) {
7         return dateObj.toLocaleDateString()
8     }
9 }

```

Código 3.2: Función Auxiliar para dar Formato a Fechas

En el Código 3.3 se ejemplifica uno de sus usos (línea 9) para dar formato en un método *getter* de las fechas a desplegar en los registros de actividad de un sitio de asignaturas.

```

1 getActiveSiteLogs (state) {
2     // Return an object of site logs already formatted.
3     if (state.activeSite && state.activeSiteLogs && state.activeSiteLogs.
4         length) {
5         var formattedLogsRows = state.activeSiteLogs.map((log) => {
6             return {
7                 id: log.id,
8                 ip: log.ip,
9                 description: log.description,
10                created_at: `${getSpanishLocaleDateFormat(log.created_at)}`
11            }
12        })
13    }
14 }

```

```

12     return formattedLogsRows
13   }
14   return null
15 },

```

Código 3.3: Ejemplo de Uso de Función Auxiliar Para Formato de Fechas

Cambiar la implementación del *widget* de *Twitter* para presentar las noticias de un sitio de asignaturas

Una de las funcionalidades implementadas en el sistema *legacy*, fue la configuración opcional de noticias desplegadas como una línea temporal de *posts* de *Twitter*, utilizando un *widget* preconfigurado de una librería externa llamada *ngtweet*. Sin embargo, poco tiempo después que el sistema *PortalAsig2* entrará en operación, la compañía *Twitter* realizó un cambio interno que inutilizó la implementación de esta funcionalidad en *ngtweet* por lo cual quedó en desuso por los usuarios. Para implementar esta modificación se buscó una dependencia similar a *ngtweet* que fuera compatible con *Vue*. La dependencia seleccionada fue *vue-tweet-embed* la cual posee una implementación del *widget* de *Twitter* deseado y sólo requiere el *id* del perfil de usuario de *Twitter* a desplegar en el *widget*.

A continuación se presenta el Código 3.4 del componente de *Vue SiteTwitterFeed* que aloja la implementación de la nueva dependencia externa *vue-tweet-embed* para desplegar el *widget* de *Twitter* correctamente. En la línea 23, se importa el componente *Timeline* de la librería *vue-tweet-embed*, en la línea 29 se declara como un subcomponente de *SiteTwitterFeed*, en las líneas 31 a 34 se definen un par de propiedades que serán utilizadas para parametrizar el componente *Timeline*, en la línea 6 se utiliza el componente *Timeline* como parte del *HTML* seguido de sus configuraciones, además se le agrega un componente interno en las líneas 13 a 17 que permite desplegar un texto y una animación mientras se realiza la carga de la información proveniente de *Twitter*.

```

1 <template>
2   <div
3     v-if="activeSite && $route.name !== 'site-news-config'"
4     id="twitter-feed-container"
5     :style="'height: ${widgetHeight}px'"
6     <Timeline
7       sourceType="profile"
8       error-message="<p>Lo sentimos pero ha ocurrido un error al cargar
9       el feed de Twitter.<br/>Por favor recargue la página.</p>"
10      :id="twitterId"
11      :key="'t-${twitterId}-h-${widgetHeight}'"
12      :options="{ height: widgetHeight - 5 }">
13        <div class="text-center">
14          <b-spinner
15            label="Cargando..."
16            variant="primary"/>
17          <h5>Cargando Feed de Twitter...</h5>
18        </div>
19      </Timeline>
20    </div>
21  </template>
22 <script>

```

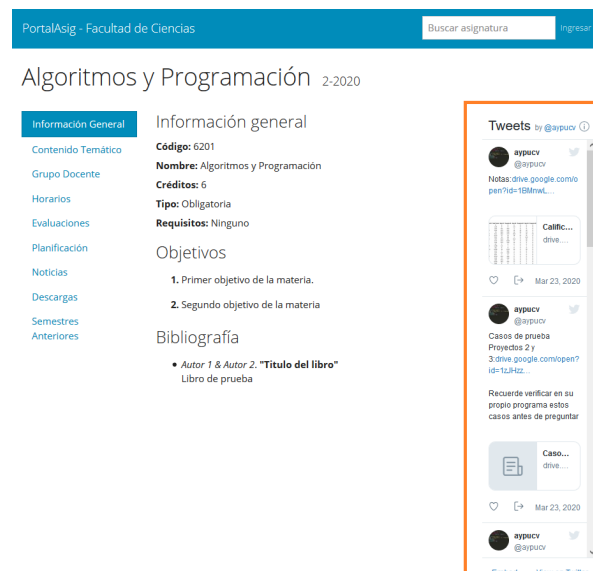
```

23 import { Timeline } from 'vue-tweet-embed'
24 import { mapState } from 'vuex'
25
26 export default {
27   name: 'SiteTwitterFeed',
28   components: {
29     Timeline
30   },
31   props: {
32     widgetHeight: Number,
33     twitterId: String
34   },
35   computed: {
36     ...mapState([
37       'activeSite'
38     ])
39   }
40 }
41 </script>

```

Código 3.4: Implementación de Widget de Twitter en el Nuevo Sistema

En la Figura 3.3 se ilustra la *UI* del *widget* de *Twitter* (resaltado en un recuadro anaranjado) en el nuevo sistema utilizando la cuenta oficial de *Twitter* de la asignatura Algoritmos y Programación @aypucv como *id*.

Figura 3.3: *UI* del *Widget* de *Twitter* en el Nuevo Sistema

Rediseñar la *UI* para la administración de calificaciones de estudiantes

La sección de administración de calificaciones de estudiantes de un sitio de asignaturas, a la cual sólo pueden acceder los usuarios que pertenezcan al grupo docente, permite a preparadores, docentes y al coordinador editar cualquier calificación de cualquier tipo de evaluación definida para cualquier estudiante asociado al sitio.

La *UI* de dicha sección en el sistema *legacy*, ilustrada en la Figura 3.4, presentaba varias fallas de diseño de *UI* y usabilidad, ente ellas, desplegar las calificaciones de cada sección de estudiantes presentaba un considerable retraso, la separación de las calificaciones en una tabla por cada evaluación resultó en el despliegue de múltiples tablas de calificaciones computacionalmente intensivas en una sola página, lo que promovía una falta de actualización correcta de las calificaciones al navegar entre las diferentes tablas resultando en información incorrecta o corrupta.



Calificaciones		
C1		
Parcial 1 Teoría		
Cédula	Estudiante	Nota
20870886	Juan Sanchez	15

Parcial 2 Teoría

Proyecto 1 Práctica

Proyecto Final Práctica

Figura 3.4: *UI* de Administración de Calificaciones del Sistema *Legacy* [2]

Para solucionar todos estos problemas se rediseñó e implementó una interfaz simplificada donde se presenta al usuario una tabla de calificaciones por cada sección de estudiantes. Dichas tablas poseen en sus filas los nombres y cédulas de los estudiantes y en sus columnas las evaluaciones con sus notas. Para editar la calificación de un estudiante, el usuario solo debe modificar la calificación presente en la intersección entre la fila del estudiante deseado y la columna de la evaluación relevante. Para facilitar la navegación de la tabla, la columna con el nombre de los estudiantes siempre se mantiene visible, al igual que la fila cabecera con los nombres de las evaluaciones. En la Figura 3.5 se presenta una captura de pantalla de la nueva *UI* de administración de calificaciones.

La única limitación presente es que solo se puede editar una calificación a la vez. Esto se debe a que el servicio proveído por la *API* para la edición de calificaciones solo acepta una calificación de una evaluación de un estudiante por cada llamada.

3.2.3.4. Transición y Pruebas

Una vez culminadas las actividades de implementación de reingeniería el producto obtenido es el código fuente del nuevo sistema, es decir una nueva aplicación *FrontEnd* desarrollada con *Vue*. Para comenzar a realizar las pruebas primero fue necesario ejecutar este código fuente en un ambiente de pruebas.

Cédula	Estudiante	Quiz 1	Examen 1	Quiz 2	Examen 2
3003	Anido Salamanca, Estudiante3	19.25	10	11.6	11
3002	Cabedo Bodas, Estudiante2	5.25	10	15.5	0

Figura 3.5: Nueva UI de Administración de Calificaciones

Como entorno de pruebas se utilizó una combinación de las plataformas *Heroku* para ejecutar y servir un clon de la aplicación *BackEnd* desarrollada en *Express.js* y la plataforma *Firebase* para ejecutar y servir la nueva aplicación *FrontEnd* desarrollada con *Vue*.

Una vez establecido este entorno de pruebas se aplicaron un conjunto de pruebas de caja negra de alto nivel donde en ambos sistemas, el *legacy* y el nuevo, se realizaban las mismas entradas y se comparaban las salidas. Gracias a este proceso se lograron descubrir algunas fallas presentes en la implementación, en su mayoría anomalías visuales o de diseño, que fueron solucionadas prontamente y vueltas a probar hasta que todas las pruebas pasaron exitosamente.

Para realizar la transición entre sistemas se decidió utilizar un método de reemplazo en vez de mantener ambos sistemas en operación paralela debido a la similitud en la UI y UX entre ambos sistemas. Se llevaron a cabo los siguientes pasos para realizar una transición impecable entre la aplicación *FrontEnd legacy* y la nueva:

1. Se detuvo la ejecución de ambas aplicaciones que conforman el sistema *PortalAsig2* en el servidor de producción.
2. Se realizó un respaldo de la base de datos y fue almacenado en varios servicios en la nube por redundancia y seguridad.
3. Se subieron los archivos pertenecientes a la nueva aplicación *FrontEnd*.
4. Se actualizó la versión de *Node.js* del servidor de producción de la versión 4.2.6 a la versión 12.16.3. Esta actualización de versiones fue probada en el ambiente de desarrollo local y el ambiente de pruebas, para verificar que no ocurrirían efectos secundarios inesperados.
5. Se realizó la configuración preliminar de la nueva aplicación *FrontEnd*.
6. Se ejecuta y sirve la aplicación *BackEnd* y en la URL: <http://www.ciens.ucv.ve/portapi>
7. Se ejecuta y sirve la aplicación *FrontEnd* en la URL: <http://www.ciens.ucv.ve/portalasig2>

Al concluir estos pasos, la nueva aplicación *FrontEnd* quedó disponible en el ambiente de producción como el nuevo punto de acceso de los usuarios al sistema *PortalAsig2* de forma transparente a los usuarios. Para representar el estado final del sistema, también se actualizó el diagrama de despliegue, ilustrado en la Figura 3.6.

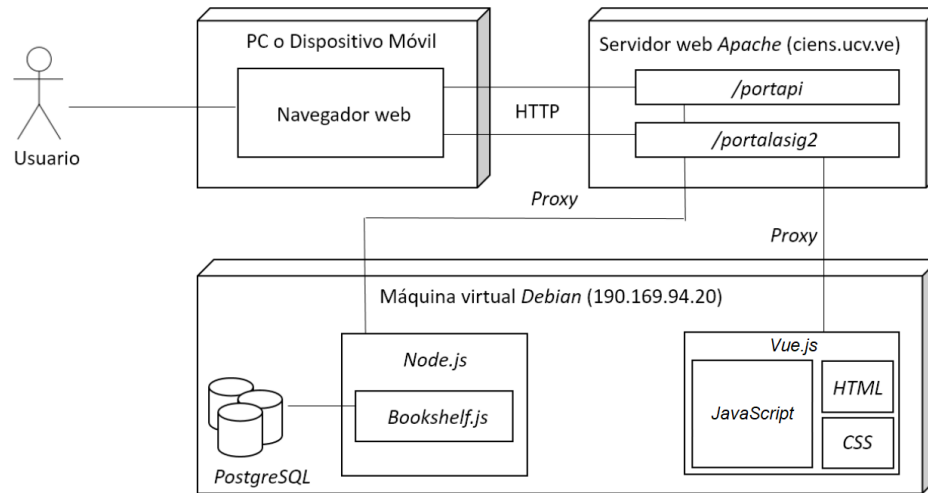


Figura 3.6: Diagrama de Despliegue Actualizado de *PortalAsig2*

3.3. Iteración 2

Si bien el sistema PortalAsig2 se puede clasificar como un Sistema de Administración de Aprendizaje (Learning Management System o LMS) [22] por sus características funcionales, el mismo carecía de un módulo para aplicar directamente algún tipo de encuesta o evaluación a los estudiantes y registrar los respectivos resultados. Debido a la complejidad técnica de esta importante funcionalidad, en esta iteración solo se implementó esta única modificación: Agregar nueva funcionalidad de encuestas y quices con preguntas de selección simple o múltiple.

3.3.1. Análisis del Sistema Actual

A diferencia de la primera iteración, las modificaciones realizadas durante esta incluyeron la implementación de cambios significativos a varios componentes de la aplicación *BackEnd* del sistema *PortalAsig2*. Estos cambios demandaron un análisis más detallado de la aplicación *BackEnd* que el realizado durante la Sección 3.2.1.

En particular, se analizaron los mecanismos disponibles para agregar nuevos modelos de datos (o editar los existentes) y sus correspondientes tablas de base de datos. Los modelos y sus relaciones con otros modelos son agregados o editados en los archivos designados como modelos de *Bookshelf.js*. Estos modelos, sus propiedades y modelos deben coincidir con los nombres utilizados en las tablas de base de datos que los representan correctamente. Esta coincidencia es realizada manualmente, lo cual dificulta las actividades de mantenimiento.

El mecanismo utilizado para agregar o modificar tablas de la base de datos es conocido como migraciones de *Knex*. Dichas migraciones son escritas de forma manual, lo cual solo brinda una fina capa de abstracción superior a escribir las modificaciones en lenguaje *SQL*. A pesar de esto, el sistema de migraciones utilizado permite aplicar cambios estandarizados y graduales a

la base de datos, que además permiten la creación de respaldos intermedios en caso de alguna corrupción de datos.

Una vez comprendidos ambos mecanismos, basta con utilizar la *API* brindada por los modelos de *Bookshelf.js* en las funciones manejadoras de las rutas para realizar la creación, lectura, modificación y eliminación de cualquier instancia de modelo.

Debido a la situación global y nacional como consecuencia de la pandemia provocada por el virus SARS-CoV-2 al momento de realizar este Trabajo Especial de Grado, la necesidad de realizar encuestas o evaluaciones a distancia se vio intensificada. Esta nueva funcionalidad de encuestas/quices fue pensada e implementada como una forma de cubrir la debilidad del sistema *PortalAsig2* cuando se trata de evaluaciones a distancia. Esta funcionalidad complementaría las entregas planificadas.

Se realizó una reunión con el Cliente para elicitación de los requerimientos, estos fueron obtenidos en forma de historias de usuario. A partir de estas historias de usuario se obtuvieron los casos de uso para cada rol del sistema. A continuación se presentan en las Figuras 3.7 y 3.8 los diagramas de casos de uso con sólo los nuevos casos de uso agregados.

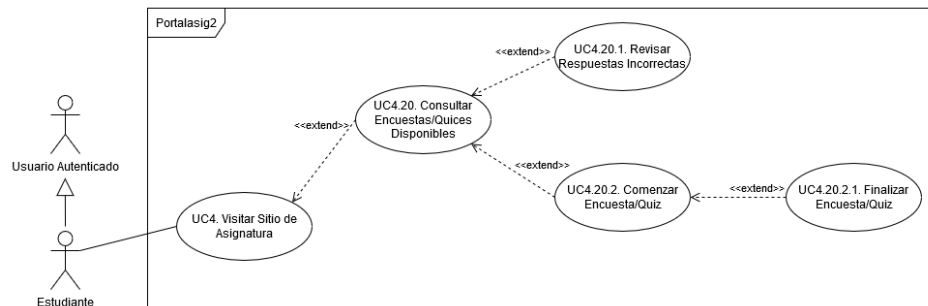


Figura 3.7: Nuevos Casos de Uso de Encuestas para el Rol Estudiante

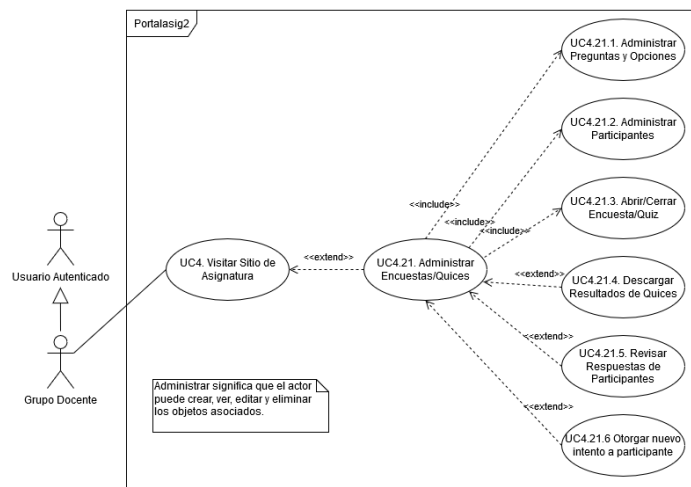


Figura 3.8: Nuevos Casos de Uso de Encuestas del Grupo Docente

A partir estos nuevos casos de uso se derivaron los requerimientos funcionales indicados en la Tabla 3.4.

Requerimiento	Descripción
FR-1: Un sitio de asignaturas puede tener una o más encuestas.	Un usuario del grupo docente puede agregar una o más encuestas para cada sitio de asignaturas al que pertenecen. Solo los usuarios que participan en dicha encuesta o los integrantes del grupo docente pueden ver o interactuar con la encuesta.
FR-2: Evaluar encuesta como Quiz.	Un usuario del grupo docente puede volver una encuesta en un quiz y viceversa. La diferencia radica en que los quices poseen preguntas ponderadas y presentan un resultado final, las encuestas no.
FR-3: Preguntas de selección simple o múltiple.	Un usuario del grupo docente puede definir si una pregunta puede aceptar una o varias opciones como respuesta.
FR-4: Ponderación porcentual en base a 100 para quices.	Un usuario del grupo docente debe ingresar valores de ponderación para cada pregunta de un quiz. Todas las ponderaciones de las preguntas sumadas deben ser igual a 100. Una pregunta es considerada como correcta y su ponderación será sumada al total de un quiz solo si el estudiante marca exactamente las opciones correctas, ni una más ni una menos.
FR-5: Varias opciones por preguntas	Un usuario del grupo docente debe colocar un mínimo de 2 opciones por pregunta y mínimo una de ellas debe ser correcta. Una pregunta puede tener cualquier cantidad de opciones que el usuario desee.
FR-6: Selección de participantes de la Encuesta/Quiz	Un usuario del grupo docente puede escoger uno o más estudiantes a participar en una encuesta/quiz, estos estudiantes pueden pertenecer a una o varias secciones de estudiantes.
FR-7: Una encuesta/quiz puede estar abierta o cerrada	Un usuario del grupo docente puede escoger abrir la encuesta/quiz a un grupo de participantes para que estos envíen sus respuestas. La encuesta también puede ser cerrada para evitar que los participantes envíen más respuestas. Al cerrar un quiz todas las ponderaciones resultantes deben ser calculadas por cada participante.
FR-8: Una encuesta/quiz puede tener un tiempo límite.	Un usuario del grupo docente puede definir la duración en minutos de una encuesta/quiz. Cuando un participante comienza a responder dicha encuesta/quiz el tiempo comienza a correr, al finalizar, las respuestas se envían automáticamente y si es un quiz, su ponderación final es calculada. Se almacena cuanto tiempo el participante tardó en responder la encuesta/quiz.

FR-9: Participante puede tener varios intentos por encuesta/quiz	Un usuario del grupo docente puede otorgarle un nuevo intento a un participante para que vuelva a enviar sus respuestas. Las respuestas de los intentos anteriores no son almacenadas en el sistema.
FR-10: Los resultados finales de los quices son descargables	Los usuarios del grupo docente pueden descargar un archivo de Excel con los resultados finales de todos los quices cerrados. Este archivo separa a los participantes por secciones de estudiantes.
FR-11: Envío de correo a los usuarios pertinentes.	Al abrir una encuesta/quiz, se le envía un correo a todos los integrantes del grupo docente y los participantes del quiz.
FR-12: Creación de registros de actividad del sitio.	Por motivos de administración de un sitio, cuando un integrante del grupo docente crea, edita o elimina una encuesta, se crea un registro de actividad en el sitio de asignaturas.
FR-13: Participantes pueden enviar respuestas de encuestas/quices	Los usuarios que participan en una encuesta/quiz deciden cuando la inician y cuando la terminan enviando sus respuestas. Los usuarios pueden seguir editando sus respuestas siempre que la encuesta/quiz no tenga tiempo límite y no hayan enviado sus respuestas.
FR-14: Revelar respuestas incorrectas de un quiz a participantes.	Un usuario del grupo docente puede indicar si al finalizar un quiz, un participante puede ver sus respuestas incorrectas o no. Esta no indica cuales fueron las opciones seleccionadas, o cuales son las correctas, solo si una pregunta fue respondida correctamente o no.
FR-15: Revisar Respuestas de Participantes	Los usuarios del grupo docente pueden revisar las respuestas enviadas de cada participante.
FR-16: Clonar una Encuesta	Los usuarios del grupo docente pueden clonar una encuesta con otro nombre diferente. La nueva encuesta mantiene sus preguntas y opciones.

Tabla 3.4: Requerimientos Funcionales de Encuestas/Quices de Sitios de Asignatura

En base a estos requerimientos funcionales y los casos de uso se diseñó un diagrama entidad-relación, ilustrado en la Figura 3.9, con las nuevas entidades(modelos) y sus relaciones a las entidades previamente existentes en el sistema (resaltadas con líneas azules).

3.3.2. Rediseño e Implementación

Una vez los requerimientos fueron definidos y el diseño de los modelos fue estructurado, se comenzó la implementación de esta modificación en la aplicación *BackEnd*: generando los modelos con la ayuda de la biblioteca *Bookshelf.js*, creando y aplicando migraciones de *Knex.js* para alterar la estructura de la base de datos, creando las diferentes funciones manejadoras o *handlers* que se encargan de utilizar los modelos generados para facilitar la lectura y modi-

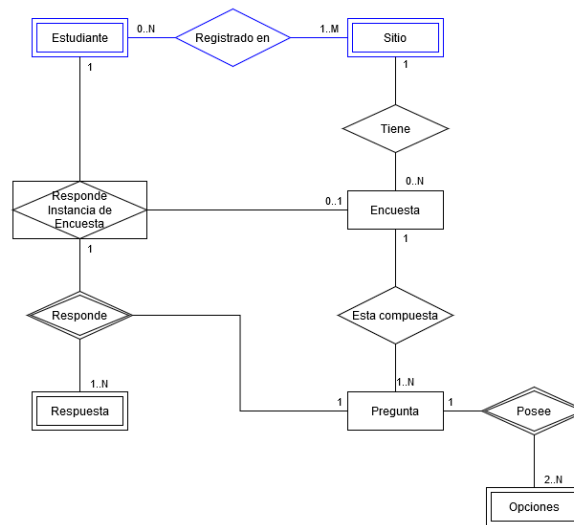


Figura 3.9: Nuevas Entidades y Relaciones de Encuestas de Sitios de Asignaturas

ficación de la base de datos y finalmente se crearon y asociaron las rutas necesarias con sus *middlewares* respectivos para asegurarlas contra accesos de usuarios sin los permisos correctos.

Del lado de la aplicación *FrontEnd*, se crearon nuevos componentes de *Vue* para manejar las nuevas funcionalidades de encuestas y quices. Un usuario del grupo docente de un sitio puede consultar la lista de encuestas o quices creados y administrarlos, como se ilustra en la Figura 3.10, utilizando el prefijo `/encuestas-quices/` en la *URL* de un sitio de asignaturas.

Encuestas/Quices

+ Agregar

Descargar Resultados

Encuestas

Quices

Nombre	Participantes	Estado	Duración	Acciones
Nueva Encuesta	0	Borrador	15 mins.	
Test1	1	Cerrado	Indefinida	

Figura 3.10: UI Para Grupo Docente de Lista de Encuestas Creadas

Al seleccionar el nombre de una encuesta/quiz el usuario navega a la vista de detalles, donde se pueden administrar los detalles de la encuesta/quiz, sus preguntas, opciones, participantes, respuestas obtenidas, entre otras funcionalidades. En la Figura 3.11 se ilustra la página de detalles de un quiz, con una pregunta de selección simple y una de selección múltiple con una suma de ponderación diferente a 100 % para demostrar las funcionalidades de validación de ponderación de los quices. La *UI* de una encuesta es idéntica a la de un quiz con excepción de los detalles de ponderación y los mensajes de validación de preguntas.

Editar Encuesta/Quiz:

Nuevo Quiz

Detalles



Clonar



Abrir

**Tipo:** Quiz**Duración Máxima:** 15 minutos.**Estado:** Borrador**Desplegar Respuestas:** No**Descripción:**

No hay descripción disponible.

Preguntas	Participantes	Respuestas
Ponderación Total: 90.00 % + Agregar <i>La ponderación total del quiz debe ser igual a 100.00 %, falta 10.00 %.</i> 1. Primera Pregunta Simple Ponderación: 50.00 % ☐ Primera opción simple ☐ Segunda opción simple correcta		

2. Pregunta de Selección Multiple
Ponderación: **40.00 %**

☐ Opción 1

☐ Opción 2

☐ Opción 3

Figura 3.11: UI Para Grupo Docente de Detalles y Preguntas de un Quiz

Luego de construir la encuesta/quiz y resolver cualquier error de ponderación de preguntas u opciones se procede a seleccionar o eliminar a los estudiantes como participantes. Esta *UI* se ilustra en la Figura 3.12 con un participante ya seleccionado y resaltado en azul.

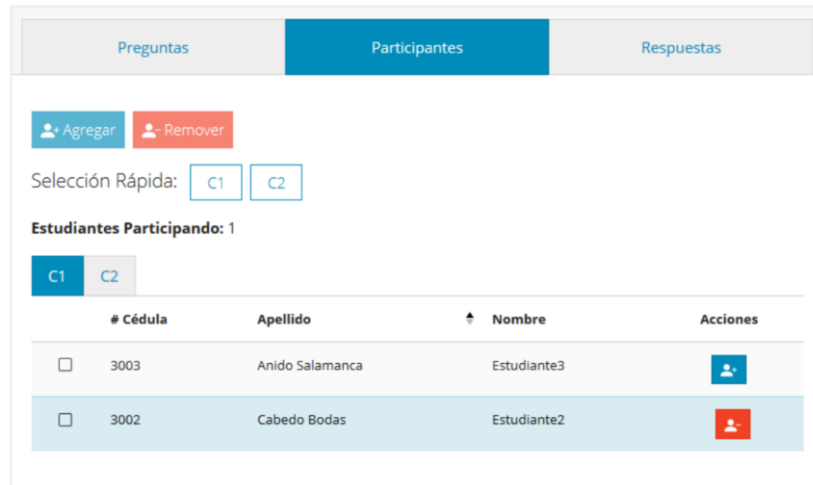


Figura 3.12: *UI* Para Grupo Docente de Participantes de una encuesta/quiz

Después de agregar todos los estudiantes a participar en la encuesta/quiz, se procede a abrirla, esto permite que los participantes visualicen dicha encuesta/quiz como disponible y envíen sus respuestas. Para abrir la encuesta se utiliza el botón **Abrir** ilustrado en la Figura 3.11, esto despliega el modal de la Figura 3.13 que indica las condiciones posteriores de una encuesta que ha sido abierta.

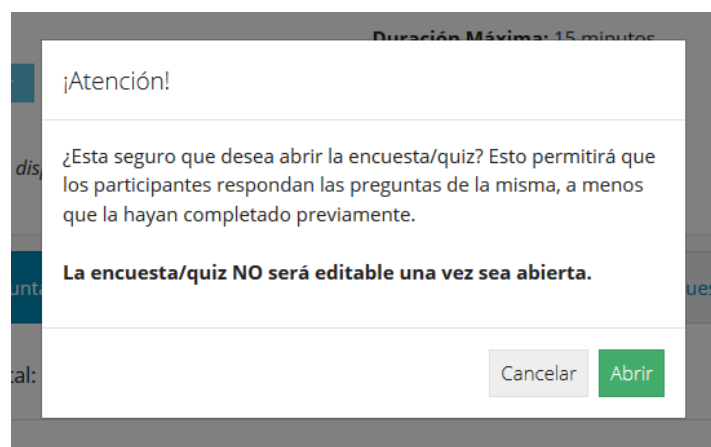


Figura 3.13: Modal para Abrir Encuesta/Quiz

Al confirmar la acción de abrir una encuesta/quiz en el modal anterior, los usuarios del grupo docente pueden acceder a la sección de Respuestas de la misma, como se muestra en la Figura 3.14 con un único estudiante participando de la sección C2.

Preguntas

Participantes

Respuestas

Para ver las respuestas haga click en el nombre del estudiante deseado. Solo se pueden ver las respuestas de estudiantes que hayan completado la encuesta/quiz. Para completar automáticamente todas las encuestas/quices puede cerrarla.

Si desea reiniciar las respuestas de un estudiante en particular, puede otorgarle un nuevo intento haciendo click en el botón amarillo en la fila del estudiante deseado.

C2

#						
Cédula	Nombre	Completo	Intento	Resultado	Acciones	
20116813	Vasquez Balza, Rafael Arturo	No	1	-		

Figura 3.14: UI Para Grupo Docente de Respuestas de una Encuesta/Quiz

Continuando con las UI para los Estudiantes, en la Figura 3.15 se muestra la UI del lado del estudiante, el cual solo podrá ver una encuesta/quiz como disponible cuando esta sea abierta por un usuario del grupo docente y el estudiante haya sido escogido como participante.

Encuestas/Quices

Encuestas		Quices		
Nombre		Duración	Completo	Puntaje
Non timed Quiz		Indefinida	SI	50 %
Nuevo Quiz		15 mins.	No	-

Figura 3.15: UI Para Estudiantes de Lista de Encuestas o Quices Disponibles

Cuando un Estudiante selecciona una encuesta/quiz sin completar o iniciar, se despliega la UI de detalle como la que se muestra en la Figura 3.16.

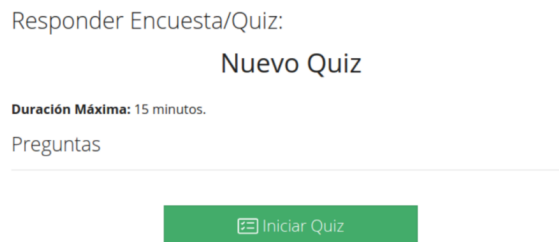


Figura 3.16: UI Para Estudiantes de Detalle de un Quiz sin Empezar

Si el Estudiante selecciona la opción de iniciar la encuesta/quiz, y esta posee un tiempo límite configurada, entonces se despliega un modal de confirmación como el de la Figura 3.17, de caso contrario, se inicia la encuesta/quiz directamente.

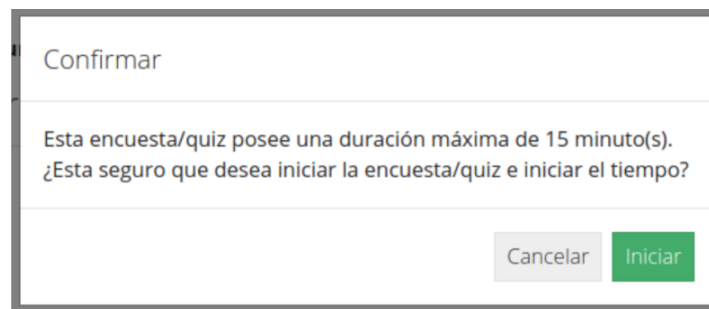


Figura 3.17: Modal para Estudiantes de Inicio de Encuesta/Quiz

Al iniciar una encuesta/quiz, el estudiante puede comenzar a marcar las opciones que considere correctas y estas son automáticamente almacenadas en el sistema en tiempo real. Si la encuesta/quiz posee un tiempo límite asignado en minutos, entonces también se despliega junto a las preguntas un reloj con cuenta regresiva que le permite al estudiante estar al tanto del tiempo disponible. Esta UI está ilustrada en la Figura 3.18.

Además si un quiz tiene la opción “Desplegar Respuestas Incorrectas” activada, los estudiantes pueden revisar sus respuestas de un quiz mediante la UI ilustrada en la Figura 3.19. Es importante destacar que no se despliegan las opciones de las preguntas correctas o incorrectas.

Los usuarios del grupo docente también pueden revisar, en la sección de Respuestas de una encuesta/quiz, las respuestas enviadas por los participantes (Figura 3.20) o reiniciar las respuestas de algún participante otorgándoles otro intento.

Responder Encuesta/Quiz:

Nuevo Quiz

Duración Máxima: 15 minutos.

Preguntas

✓ Enviar Quiz

0 14 53
Horas Minutos Segundos

1. Pregunta de selección simple

Ponderación: **50.00 %**

☐ Primera Opción

☐ Segunda Opción Correcta

2. Pregunta de selección múltiple

Ponderación: **50.00 %**

☐ Opción 1 Correcta

☐ Opción 2

☐ Opción 3 Correcta

✓ Enviar Quiz

Figura 3.18: UI para que Estudiantes Respondan Quiz

Respuestas de Encuesta/Quiz:

Nuevo Quiz

Duración Máxima: 15 minutos.

Respuestas

Ponderación Obtenida: **50.00 %**

1. Pregunta de selección simple

Ponderación: **50.00 %**

2. Pregunta de selección múltiple

Ponderación: **50.00 %**

Figura 3.19: UI para que Estudiantes Revisen Respuestas de Quiz

Respuestas a Encuesta/Quiz:
Nuevo Quiz

[← Atrás](#) [20116813] Vasquez Balza, Rafael Arturo

Resultado Obtenido: **50.00 %**

1. Pregunta de selección simple
Ponderación: **50.00 %**

☒ Primera Opción

☐ Segunda Opción Correcta

2. Pregunta de selección múltiple
Ponderación: **50.00 %**

☒ Opción 1 Correcta

☐ Opción 2

☒ Opción 3 Correcta

Figura 3.20: UI para que Grupo Docente Revise Respuestas de Quiz

3.4. Iteración 3

Las modificaciones implementadas durante esta iteración fueron: “Otorgar al rol Administrador todos los permisos del rol Coordinador, para todos los sitios”, “Agregar opción para ocultar algunas secciones de estudiantes en partes de un sitio” y “Agregar nueva sección de cursos académicos no acreditados en la página principal”.

3.4.1. Análisis del Sistema Actual

A continuación se describen las justificaciones y cómo se podrían implementar cada una de las modificaciones en el sistema.

Otorgar al rol Administrador todos los permisos del rol Coordinador, para todos los sitios

Inicialmente el rol de Administrador del sistema *PortalAsig2* fue concebido como un rol con pocas capacidades, entre estas se encuentran las funcionalidades para agregar, consultar, editar y eliminar asignaturas y usuarios estudiantes y docentes. Sin embargo, una vez el sistema *PortalAsig2* fue puesto en operación, resultó obvio que el Administrador del sistema requería poseer las mismas funcionalidades y permisos que un usuario Coordinador tiene sobre un sitio, para todos los sitios de asignaturas con el fin de revisar y resolver cualquier problema del sistema sin necesidad de obtener las credenciales de los usuarios Coordinador y Docentes de un sitio.

Para implementar esta modificación se requirió cambiar varias de las reglas de acceso (*middlewares* en inglés) de las rutas de la *API* de la aplicación *BackEnd*. Estos *middlewares* son los encargados de verificar en cada ruta, que el usuario posee el rol necesario en el sitio para acce-

der el recurso alojado en la ruta, de lo contrario se obtiene una respuesta *HTTP* con el código de estado 403 que indica que el recurso que se intento acceder está prohibido o el acceso fue denegado.

Agregar opción para ocultar algunas secciones de estudiantes en partes de un sitio

Un caso de uso común e ignorado durante el diseño del sistema *PortalAsig2* fue crear un horario de clases para múltiples secciones de estudiantes al mismo tiempo, actualmente en el sistema solo se puede asociar una única sección de estudiantes por horario. Un ejemplo de este caso de uso es: un sitio con cuatro secciones de estudiantes C1, C2, C3, y C4 puede necesitar dividir las entre 2 laboratorios como se ilustra en la Figura 3.21.

Horarios

Teoría Práctica Laboratorio					
Docente	Sección	Día	Hora	Aula	
Fernandez Makoukji, Yuliana Trinidad	C1/C2	Miércoles	11:00 am - 1:00 pm	2	
Belisario Frasca, Gabriel Enrique	C3/C4	Jueves	11:00 am - 1:00 pm	1	

Figura 3.21: Ejemplo de Horarios con Múltiples Secciones de Estudiantes

Para cumplir con este caso de uso, los usuarios del sistema recurrieron a crear nuevas secciones como “C1/C2” y asociarlas a los horarios, el problema con este enfoque es que dichas secciones también aparecen en otras áreas del sitio, como calificaciones y entregas desde el punto de vista del grupo docente, creando confusión entre los usuarios. Una forma de resolver este problema conlleva un rediseño de la base de datos y la actualización de toda la lógica de los horarios y las secciones de estudiantes. Sin embargo, estos cambios son muy invasivos y extensos, afectando varios módulos y componentes de las aplicaciones *BackEnd* y *FrontEnd*.

Otra forma de resolver el problema es agregar un campo booleano a la tabla de datos de los horarios que indique si la sección que tienen asociada debe aparecer en el resto de las áreas del sitio o no. Este cambio a pesar de afectar la base de datos también, solo realiza cambios aditivos que no requieren de muchos cambios en el resto de los componentes de las aplicaciones *BackEnd* y *FrontEnd*.

Agregar nueva sección de cursos académicos no acreditados en la página principal

Debido a la situación global y nacional como consecuencia de la pandemia provocada por el virus SARS-CoV-2 al momento de realizar este Trabajo Especial de Grado, la Facultad de Ciencias de la Universidad Central de Venezuela decidió ofrecer cursos educativos a distancia, con contenido similar a las asignaturas dictadas regularmente para que los estudiantes de la facultad pudiesen continuar con su educación de alguna forma. Sin embargo estos cursos no son evaluados de la misma manera que las asignaturas regulares, por lo que se necesita una separación de estas en el sistema *PortalAsig2*.

3.4.2. Rediseño e Implementación

Otorgar al rol Administrador todos los permisos del rol Coordinador, para todos los sitios

Los *middlewares* modificados fueron: *isTeacherGroup* y *isProfessorSite*, los cuales se encargan de verificar que el usuario, pertenece al grupo docente del sitio o es un usuario con rol de Docente o Coordinador registrado en el sitio respectivamente. En el Código 3.5 se ilustra la modificación realizada en el *middleware isProfessorSite*. La modificación hecha fue la adición de un condicional (línea 7) en el cual se pregunta si el usuario que realizó la petición *HTTP* posee el rol de Administrador, si es así entonces se le permite consultar el recurso protegido por el *middleware*, de lo contrario se procede a ejecutar la lógica original donde se verifica si el usuario posee el rol de Docente o Coordinador.

```

1 module.exports = function (req, res, next) {
2   // First check if user is Administrator
3   req.app.locals.models.Administrator
4     .forge({ id: res.locals.user.id })
5     .fetch()
6     .then(function (admin) {
7       if (admin)
8         return next();
9       else {
10        // Lógica original
11      }
12    })
13    .catch(function (err) {
14      res.status(500).send(err);
15    });
16 };

```

Código 3.5: *Middleware isProfessorSite* Modificado

Agregar opción para ocultar algunas secciones de estudiantes en partes de un sitio

Para implementar esta modificación se requirieron cambios en ambas aplicaciones del sistema *PortalAsig2*. En la aplicación *BackEnd* se agregó un nuevo campo booleano llamado *schedule_only* al modelo y la tabla *schedule* de la base de datos mediante una migración de *Knex*, en el Código 3.6 se presenta el contenido de dicha migración. El campo *schedule_only* permite seleccionar si la sección utilizada en un horario solo debe ser desplegada en la sección de horarios y no en otras secciones del sitio de asignaturas, como calificaciones.

```

1 exports.up = function(knex, Promise) {
2   return knex.schema.table('schedule', table => {
3     table.boolean('schedule_only').defaultTo(false).notNull();
4   })
5 };

```

Código 3.6: Migración para Alterar Tabla *schedule* de la Base de Datos

En la aplicación *FrontEnd*, se agregó el campo *schedule_only* al formulario de creación y

edición de horario. La representación del nuevo campo en el formulario esta resaltado en la Figura 3.22.

Editar Horario

*Sección:
C2

Busque y haga click en una de las opciones que se despliegan para seleccionar la sección deseada. Este campo es requerido.

☒ Desplegar solo en sección de Horarios
Los horarios marcados con esta opción no aparecerán en el resto de las secciones del sitio como Calificaciones, Estudiantes, etc.

*Instructor:
[XXXXXXX] Hernandez, Walter

*Día:
Lunes

*Tipo:
Teoría

*Hora Inicial:
7:00 am

*Hora Final:
9:00 am

*Nombre o Número de Aula:
11

Cancelar Aceptar

Figura 3.22: Formulario de Horarios con Campo *schedule_only*

Agregar nueva sección de cursos académicos no acreditados en la página principal

Esta modificación aditiva solo requirió cambios en la aplicación *FrontEnd*, específicamente se modificó el componente encargado de desplegar la lista de asignaturas por tipo y semestre, agregando la nueva sección “Cursos” y modificando el método *getter* que segrega la lista de asignaturas obtenida desde la aplicación *BackEnd* por tipo y semestre para separar aquellas asignaturas clasificadas como “Cursos” del resto. En la Figura 3.23 se muestra la *UI* de la lista de cursos en la página principal con una materia de prueba clasificada como “Curso”. Los cursos no poseen código de asignatura o número de créditos universitarios asociados debido a que no son evaluados como asignaturas regulares.

Computación

Obligatorias Electivas **Cursos**

Nombre	Último período
Materia de Prueba Curso	U-2020

Figura 3.23: Nueva *UI* de Lista de Cursos en Página Principal

3.5. Iteración 4

En esta última iteración del mantenimiento se implementaron el resto de las modificaciones restantes.

3.5.1. Análisis del Sistema Actual

A continuación se describen las justificaciones y cómo se podrían implementar cada una de las modificaciones en el sistema.

Buscar estudiantes y docentes por nombre o apellidos

En el sistema original, para que los integrantes del grupo docente agregaran a otros docentes o estudiantes como preparadores a un sitio de asignaturas de forma individual, se utilizaba una búsqueda por número de cédula, sin embargo, esta es una funcionalidad poco intuitiva que dificulta el proceso, pues requiere que un profesor primero indague el número de cédula del docente o estudiante antes de agregarle al sitio. Es preferible agregar como parámetros de búsqueda el nombre o apellido de un usuario para facilitar este flujo.

Eliminar permiso del rol Docente para agregar nuevas asignaturas

Esta modificación surgió por la necesidad de mantener los permisos y privilegios entre los tipos de cuentas de usuarios (Administrador, Docentes y Estudiantes) claramente separadas. Solo un usuario Administrador debería administrar las asignaturas, por lo que se decidió eliminar esta funcionalidad para las cuentas de usuarios Docentes.



Figura 3.24: Botón para Agregar Asignaturas

Esta funcionalidad está disponible en la aplicación *FrontEnd* como un botón ubicado en la lista de asignaturas de la pagina principal (Figura 3.24), que despliega un formulario para agregar asignaturas. Además en la aplicación *BackEnd*, se debe cambiar los *middlewares* de la ruta encargada de aceptar la petición de creación de asignatura para protegerla.

Desplegar cuerpo de noticias completo en la lista de noticias

En el sistema original, el contenido de una noticia era recortado visualmente a solo una

única línea de 144 caracteres en la lista de noticias de un sitio. Para visualizar la noticia completa, el usuario debe seleccionar el título de la noticia lo cual despliega un modal con la noticia completa. Esto es un diseño que entorpece el proceso de lectura de todas las noticias, al aumentar la cantidad de *clicks* que un usuario tiene que realizar innecesariamente.

Incorporar texto enriquecido al cuerpo de las noticias

Los cuerpos de las noticias no poseían ningún tipo de formato adicional, solo texto plano. Un usuario del sistema *PortalAsig2* sugirió esta modificación para mejorar las capacidades de edición del contenido de las noticias, el texto enriquecido (con formato adicional como negritas, itálicas, subrayado, entre otros) es muy común hoy en día en cualquier plataforma de generación y edición de texto. Cabe destacar que los foros y sus comentarios ya cuentan con esta funcionalidad por lo que se puede reutilizar la lógica ya implementada.

Agregar cuerpo de noticias a los correos de notificación

Actualmente cuando una noticia es creada en un sitio de asignaturas, todos los usuarios, que tengan la opción activada, reciben un correo electrónico de notificación con un enlace para ver la noticia en el sistema. Sin embargo, para agilizar este procedimiento, fue sugerido que se colocara el título y cuerpo de la noticia en dicho correo. Esto evitaría que los usuarios tuviesen que ingresar para leer cualquier noticia.

3.5.2. Rediseño e Implementación

Buscar estudiantes y docentes por nombre o apellidos

Para implementar esta modificación se agregaron cambios en la aplicación *BackEnd*, específicamente en los parámetros aceptados y en la lógica de búsqueda de la ruta encargada de retornar los estudiantes y docentes dado un número de cédula. Se generalizó la lógica de búsqueda utilizando un único parámetro *where* el cuál puede ser una cédula de identidad o una cadena de caracteres representando el nombre o apellido del usuario a buscar. Esta búsqueda es realizada a nivel de base de datos como se ejemplifica en las líneas 13 a 18 del Código 3.7.

```

1 req.app.locals.models.Professor
2   .forge()
3   .query(function (qb) {
4     qb.join('user', 'user.id', '=', 'professor.id')
5     .select(
6       'user.id',
7       'user.identity_card',
8       'user.name',
9       'user.lastname',
10      'user.email'
11    )
12    .orderBy(orderCol, orderDir);
13    if (where) {
14      qb.where('user.name', 'ILIKE', where)
15        .orWhere('user.lastname', 'ILIKE', where)
16        .orWhere(req.app.locals.bookshelf.knex(
17          'CAST("user"."identity_card" AS TEXT) LIKE ?', where)
18        );

```

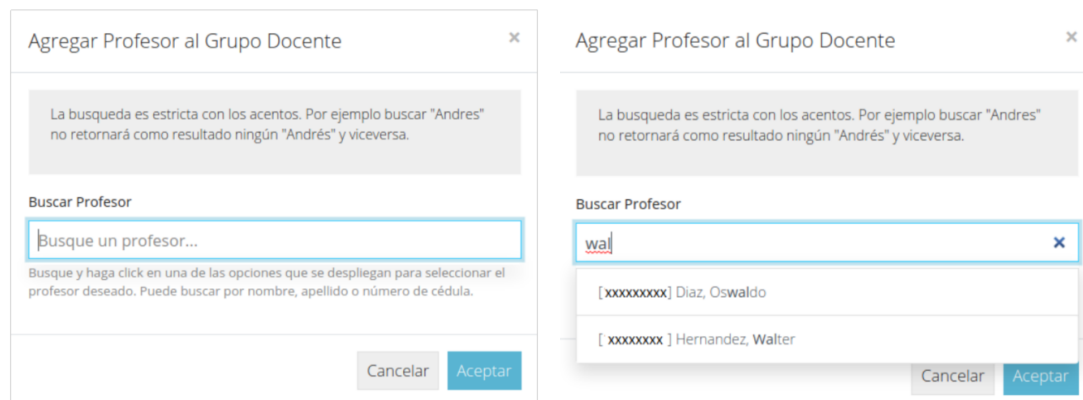
```

19     }
20   })
21   .fetchAll()

```

Código 3.7: Búsqueda de Profesores por Cédula, Nombre o Apellido

Además en la aplicación *FrontEnd* se alteraron los componentes de *Vue* y las acciones de *Vuex* que envían y solicitan información de las rutas modificadas en la aplicación *BackEnd*, para incluir el nuevo parámetro *where*. En la Figura 3.25 se ilustran las *UI* para agregar un profesor al grupo docente y la búsqueda realizada por nombre.



(a) Modal Para Agregar Profesores a Sitio

(b) Ejemplo de Búsqueda por Nombre

Figura 3.25: *UI* para Agregar Profesores a Sitio de Asignaturas

Eliminar permiso del rol Docente para agregar nuevas asignaturas

Para implementar esta modificación, se alteró la permisología de visualización de dicho botón en el componente de *Vue* para que sólo las cuentas de usuario Administrador puedan verlo y utilizarlo. Además, para proteger la ruta encargada de agregar nuevas asignaturas en la aplicación *BackEnd*, solo se requirió reemplazar el *middleware isProfessor* por *isAdmin* para asegurar la ruta contra intentos de agregar nuevas asignaturas de usuarios no Administradores.

Desplegar cuerpo de noticias completo en la lista de noticias

Se alteraron los componentes de *Vue* encargados de desplegar los contenidos de las noticias para desplegar todo el contenido y eliminar los eventos que despliegan el modal con la noticia completa, dado que ya no son necesarios. El resultado final se aprecia en la Figura 3.26.

Incorporar texto enriquecido al cuerpo de las noticias

Esta modificación requirió la integración de un editor de texto enriquecido (conocido como *WYSIWYG* por sus siglas en inglés) en el componente de *Vue* encargado del formulario de creación y edición de las noticias, dicho editor de texto *WYSIWYG* fue el mismo utilizado en los componentes de foros y sus comentarios, dicho editor también cuenta con funcionalidades de depuración del texto para evitar que los usuarios inserten caracteres maliciosos y puedan afectar de forma negativa el sistema. En las Figuras 3.27 se ilustra el nuevo formulario de cuerpo de una noticia.



(a) UI Original

(b) UI Nueva

Figura 3.26: UI de Lista de Noticias

Figura 3.27: Formulario para Noticias con Editor WYSIWYG

La noticia creada en la Figura 3.27, se despliega en la lista de noticias como se muestra en la Figura 3.28.

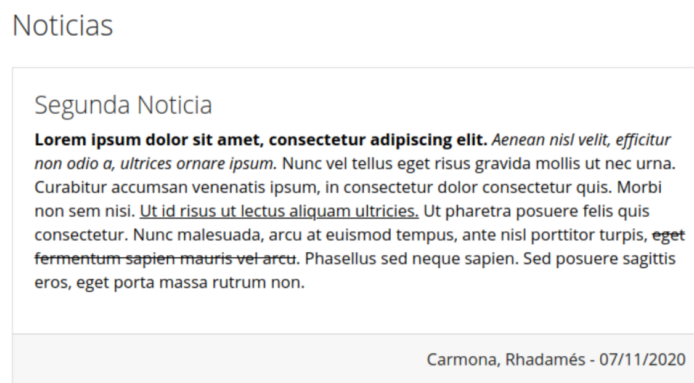


Figura 3.28: Texto Enriquecido en Lista de Noticias

Agregar cuerpo de noticias a los correos de notificación

Este cambio requirió editar la plantilla *HTML* utilizada como base para el envío de correos y proporcionar los datos de la noticia a los métodos auxiliares que generan los correos electrónicos y los envían a todos los recipientes indicados. La Figura 3.29 demuestra un ejemplo del nuevo contenido de los correos electrónicos que notifican al usuario de nuevas noticias en un sitio.

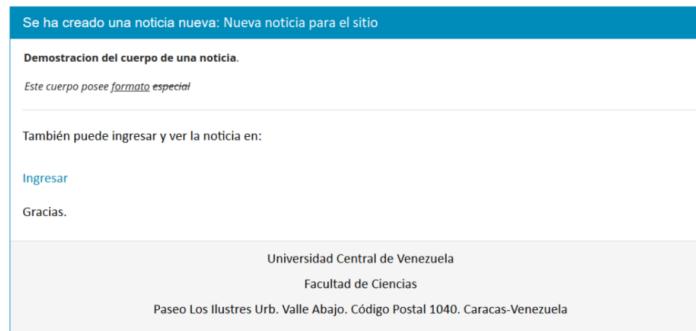


Figura 3.29: Contenido de Correos Electrónicos de Noticias

En el segundo Apéndice se presentan todos los diagramas actualizados correspondientes a los cambios realizados durante todas las actividades de mantenimiento.

Capítulo 4

Pruebas y Resultados

Para guiar y demostrar la efectividad de las actividades de mantenimiento y reingeniería de software desarrolladas como parte de este Trabajo Especial de Grado, se presentan a continuación las pruebas y resultados de las mismas, aplicadas sobre las diferentes versiones de ambas aplicaciones que conforman el sistema *PortalAsig2* (*FrontEnd* y *BackEnd*). Las versiones corresponden con el código fuente producido al concluir cada iteración de la metodología de mantenimiento de software previamente descrita.

Las pruebas realizadas consistieron en la toma de mediciones de diferentes métricas de software para comparar el estado anterior y posterior a la realización de cada iteración de la metodología de mantenimiento de software utilizada. Las métricas medidas fueron la complejidad ciclomática de McCabe, obtenida mediante la Fórmula 2.1, el esfuerzo del programa de Halstead según la fórmula 2.2, el tiempo aproximado requerido para entender o modificar el programa y el número de errores previstos en el programa, utilizando las Fórmulas 2.3 y 2.4 respectivamente. Estas mediciones fueron realizadas a la aplicación *BackEnd* y *FrontEnd* utilizando la biblioteca *complexity-report* [23] la cual, hace un estudio del código fuente y retorna un reporte con el promedio por función de varias métricas, entre ellas las mencionadas anteriormente.

Estas mediciones permitieron guiar las actividades de desarrollo. Si las medidas de una iteración eran mayores a las obtenidas en su predecesora entonces durante la próxima iteración se deben realizar esfuerzos de refactorización para reducir o limitar el crecimiento de la complejidad ciclomática y/o el esfuerzo del programa de Halstead.

En la Tabla 4.1 se muestran las mediciones obtenidas de las diferentes versiones de la aplicación *FrontEnd* de *PortalAsig2*. La columna **Original** indica los resultados de la aplicación original desarrollada con *AngularJS*. La columna **Iteración 1** indica los resultados de la aplicación recreada utilizando *Vue* derivada del proceso de reingeniería de software. Las columnas de **Iteración 2**, **3** y **4** representan los resultados obtenidos al completar las implementaciones de las modificaciones en sus respectivas iteraciones.

Métrica	Resultados (Promedio por función)				
	Original	Iteración 1	Iteración 2	Iteración 3	Iteración 4
Complejidad Ciclomática	3,1957	1,9876	2,0338	2,0551	2,0679
Esfuerzo del Programa	4.662,9550	3.599,0631	3.806,0741	3.856,1754	3.888,4922
Tiempo Estimado del Programa	259 seg.	200 seg.	211 seg.	214 seg.	216 seg.
Número de errores previstos en el programa	0,0930	0,0782	0,0812	0,0819	0,0824

Tabla 4.1: Resultados de Versiones de Aplicación *FrontEnd*

En la Tabla 4.2 se muestran las mediciones obtenidas de la aplicación *BackEnd* en su estado original y las versiones de cada iteración a partir de la segunda, la primera iteración no se toma en cuenta porque no ocurrió modificación de la aplicación *BackEnd* en ella.

Métrica	Resultados (Promedio por función)			
	Original	Iteración 2	Iteración 3	Iteración 4
Complejidad Ciclomática	1,2454	1,2706	1,2561	1,2503
Esfuerzo del Programa	2.883,7657	2.841,3498	2.805,4542	2.762,4737
Tiempo Estimado del Programa	160 seg.	158 seg.	156 seg.	153 seg.
Número de errores previstos en el programa	0,0675	0,0668	0,0663	0,0656

Tabla 4.2: Resultados de Versiones de Aplicación *BackEnd*

De estos resultados se derivan las Tablas 4.3 y 4.4 las cuales reflejan los porcentajes de reducción o aumento de las métricas de cada iteración comparada con la versión original de cada aplicación que componía el sistema *PortalAsig2*. La Figura 4.1 permite visualizar los porcentajes expresados en la Tabla 4.3.

Métrica	Resultados (Promedio por función)			
	Iteración 1	Iteración 2	Iteración 3	Iteración 4
Complejidad Ciclomática	-37,803 %	-36,358 %	-35,691 %	-35,291 %
Esfuerzo del Programa	-22,815 %	-18,376 %	-17,301 %	-16,608 %

Tabla 4.3: Resultados de Aplicación *FrontEnd* en Porcentajes

La reducción substancial de complejidad y esfuerzo promedios del programa apreciable en columna de la Iteración 1 de la Tabla 4.3, es consecuencia directa de la correcta implementación de la metodología de reingeniería de software que permitió rediseñar la estructura del sistema *PortalAsig2* y aumentar la modularidad al descomponer las secciones funcionales escritas en *AngularJS* en componentes reducidos y reutilizables de *Vue*. El ligero aumento de complejidad y del esfuerzo a lo largo de las iteraciones 2, 3 y 4 comparado con la Iteración 1, se atribuye a las modificaciones posteriores, incluyendo la adición de nuevas funcionalidades, las cuales, a pesar de ser planificadas correctamente, no agotaron todos los recursos posibles para combatir

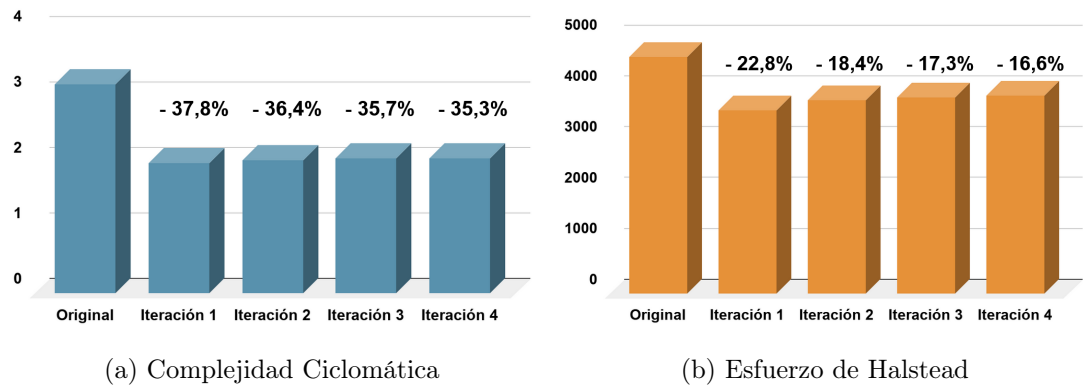


Figura 4.1: Variaciones de Métricas de Aplicación *FrontEnd*

la complejidad presente en el sistema sino en una implementación flexible y comprensible. Este comportamiento se adhiere a la segunda ley de Lehman enunciada en la Sección 2.1.6.

Métrica	Resultados (Promedio por función)		
	Iteración 2	Iteración 3	Iteración 4
Complejidad Ciclomática	2,023 %	0,859 %	0,393 %
Esfuerzo del Programa	-1,470 %	-2,715 %	-4,206 %

Tabla 4.4: Resultados de Aplicación *BackEnd* en Porcentajes

La Figura 4.2 permite visualizar los porcentajes expresados en la Tabla 4.4.

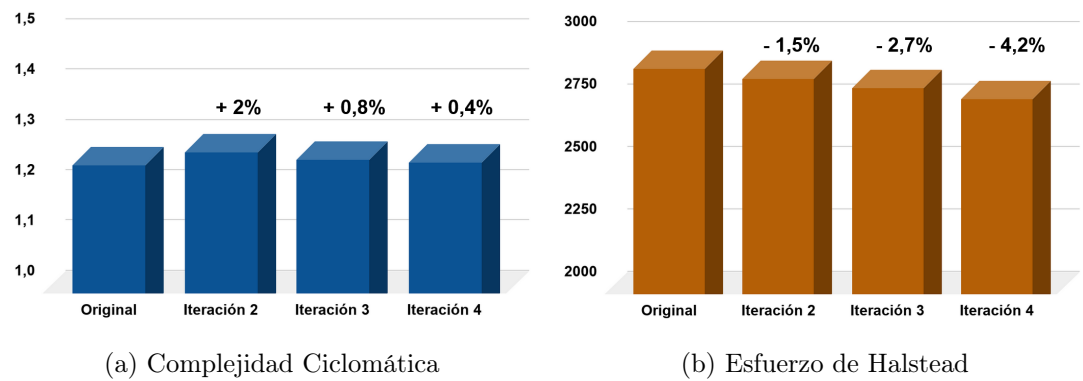


Figura 4.2: Variaciones de Métricas de Aplicación *BackEnd*

En la Tabla 4.4 se presenta una interesante dicotomía, la complejidad aumentó ligeramente, mientras el esfuerzo del programa fue reducido ligeramente. El aumento de la complejidad promedia después de la Iteración 2 y su reducción se debe a refactorizaciones realizadas en las iteraciones posteriores. La reducción del esfuerzo promedio del programa ocurrió como consecuencia de las mejores prácticas de programación y diseño de solución utilizadas durante la implementación de todas las modificaciones, que si bien no fueron planeadas para combatir la complejidad del todo, si lo fueron para ser más legibles y modulares.

Para demostrar que el desempeño del sistema no fue degradado como consecuencia de las actividades de mantenimiento desarrolladas en este Trabajo Especial de Grado, se realizaron las mismas pruebas de desempeño descritas en la sección de pruebas y resultados de [2]. Estas pruebas consistieron en medir el tiempo transcurrido desde que se solicita una página web en el campo de direcciones del navegador hasta que es cargada completamente. Cada prueba fue repetida 10 veces, utilizando las herramientas de “Red” para desarrolladores ofrecidas por el navegador *Mozilla Firefox* en su versión 82.0.2. En la Tabla 4.5 se presentan los resultados promedios obtenidos, incluyendo los resultados de versiones del sistema *PortalAsig* y *PortalAsig2* anteriores para comparar adecuadamente. La Figura 4.3 permite visualizar la diferencia de magnitudes entre los resultados de las pruebas registradas en la Tabla 4.5.

Descripción de la prueba	Resultados (segundos)		
	<i>PortalAsig</i>	<i>PortalAsig2</i> (2016)	<i>PortalAsig2</i> (2020)
Visitar el URL principal del portal	3,42	8,25	2,56
Dirigirse a una licenciatura diferente	1,28	0,00	0,00
Dirigirse a un sitio web	0,84	0,42	0,64
Dirigirse a secciones distintas de un sitio web	0,57	0,00	0,32
Crear un objetivo nuevo en un sitio web	0,22	0,08	0,09
Ver estudiantes de un sitio web	0,24	0,05	0,06
Entrar en el módulo de administración	1,06	0,35	0,89
Ver docentes en el módulo de administración	3,68	0,00	0,20
Ver estudiantes en el módulo de administración	8,54	0,00	0,59

Tabla 4.5: Resultados de Prueba de Desempeño de Velocidad de Carga

La Tabla 4.5 y la Figura 4.3 indican que la nueva versión de *PortalAsig2* (2020) concebida en el desarrollo de este Trabajo Especial de Grado presenta una carga mucho más rápida inicial de la página principal que *PortalAsig2* (2016), sin embargo, presenta mayores intervalos de tiempo para la carga de otras secciones del sitio y el módulo de Administración. Estos intervalos mayores solo ocurren la primera vez que un usuario accede a una sección. Cualquier acceso posterior a dichas secciones presenta velocidades de carga semejantes a las de la *PortalAsig2* (2016). Esta variación de las velocidades es consecuencia de la utilización de *lazy loading* o descarga por demanda de componentes de *Vue*, la cual prioriza la descarga y despliegue de la página principal de una aplicación web y solo descarga el resto de los componentes a medida que el usuario los requiera.

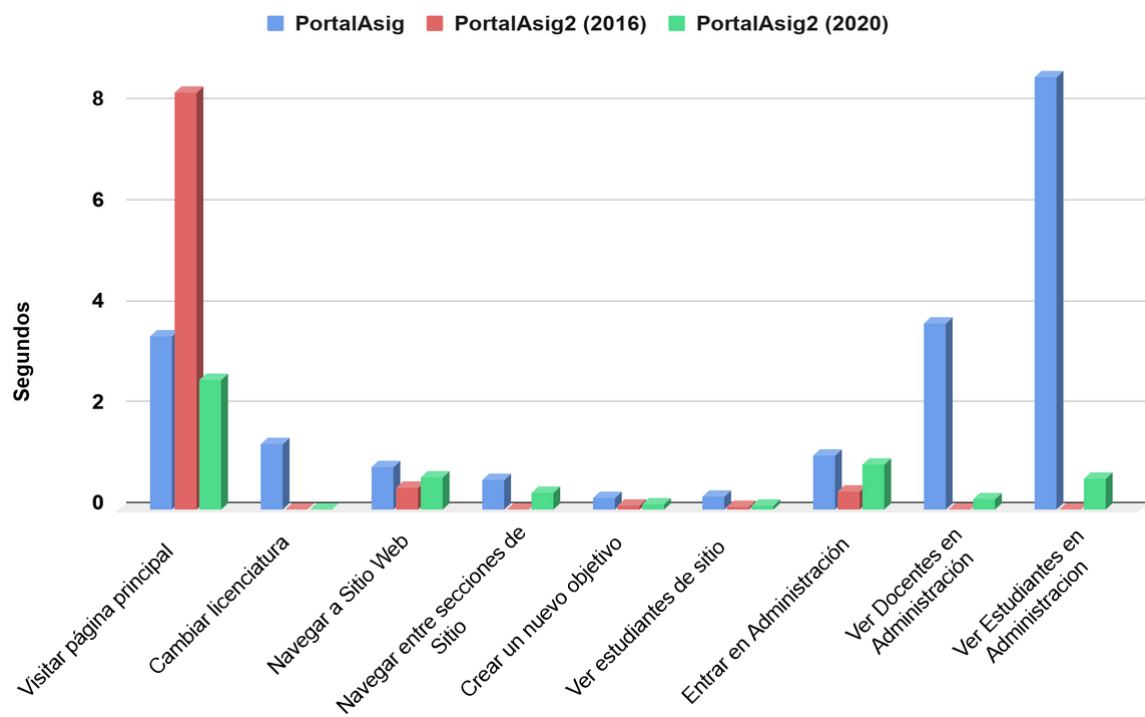


Figura 4.3: Diferencia de desempeño entre distintas versiones de *PortalAsig* y *PortalAsig2*

Capítulo 5

Conclusiones y Recomendaciones

Durante el proceso de desarrollo de un sistema de software, es de gran importancia proponer un plan o proceso de mantenimiento para poner en acción, posterior al despliegue de dicho sistema, con el fin de mantener el propósito general de una aplicación de software: ser un producto que genere un valor agregado y continuo en el tiempo para sus usuarios.

A lo largo del presente Trabajo Especial de Grado, se llevaron a cabo actividades de mantenimiento sobre la segunda versión del Portal de Asignaturas de la Facultad de Ciencias de la Universidad Central de Venezuela, sistema denominado *PortalAsig2*. La planeación, ejecución, validación y finalmente éxito de estas actividades dependieron ampliamente de la definición previa de una metodología de mantenimiento de software adaptada al sistema en cuestión basándose en diversos aspectos como la solución de fallas preexistentes en el sistema, incorporación de nuevas tecnologías y la integración de nuevas funcionalidades.

Además, como parte de estas actividades también se desarrollaron actividades de reingeniería del software, cuyo éxito también dependió en gran parte de la implementación de una metodología previamente definida y que sentó las bases para la mejora de funcionalidades previas, como la *UI* y *UX* de la sección de Calificaciones y la implementación de nuevas funcionalidades como un sistema de Encuestas y Quices, a la vez que se realizaba la migración de la aplicación *FrontEnd* a un *framework* moderno y con mayor soporte como *Vue*.

Se pudo corroborar con las pruebas realizadas que gracias a la adaptación de una metodología de mantenimiento y reingeniería al sistema *PortalAsig2* y su posterior implementación, la complejidad promedia del sistema y el esfuerzo requerido para comprenderlo y modificarlo fueron reducidas considerablemente y su adaptación a correcciones y mejoras futuras aumentó con la misma magnitud. Todo esto gracias a la modularidad inducida en la aplicación *FrontEnd* del sistema, que permitió incluir nuevas funcionalidades que serán mantenidas a futuro gracias a su aprovechamiento de tecnologías modernas, con lo que se confirma que los objetivos planteados para este Trabajo Especial de Grado fueron logrados con éxito.

Las pruebas y resultados obtenidos son testimonio de la efectividad de las metodologías de mantenimiento y reingeniería del software definidas y de sus correctas implementaciones, todo en aras de lograr la evolución sostenida del sistema *PortalAsig2*, de manera que pueda

continuar su funcionamiento como un sistema eficiente y útil para la comunidad estudiantil y la comunidad docente de la Facultad de Ciencias de la Universidad Central de Venezuela.

Recomendaciones y Trabajos Futuros

Para garantizar que el sistema *PortalAsig2* se mantenga como una solución de software efectiva ante las necesidades de la comunidad de la Facultad de Ciencias de la Universidad Central de Venezuela se formulan una serie de recomendaciones listadas a continuación:

- Aumentar el repertorio de métricas disponibles para la medición de la efectividad de sucesivas actividades de mantenimiento aplicadas al sistema *PortalAsig2*.
- Realizar un trabajo de reingeniería sobre la aplicación *BackEnd* del sistema *PortalAsig2* utilizando *frameworks* y técnicas de desarrollo modernas que incluyan la generación de un conjunto de pruebas unitarias de sus componentes para aumentar su facilidad de mantenimiento y capacidad de adaptación a futuras necesidades de los usuarios.
- Incorporar nuevos desarrolladores al equipo de mantenimiento del sistema aprovechando sus nuevas facilidades de integración.
- Desarrollar una aplicación móvil que utilice la *API* disponible del sistema *PortalAsig2*.
- Realizar un rediseño del sistema que permita generalizar todas sus funcionalidades de forma que pueda ser fácilmente exportado y configurado por otras Facultades de la Universidad Central de Venezuela e incluso otras universidades nacionales.

Apéndice

Diagramas Actualizados de *PortalAsig2*

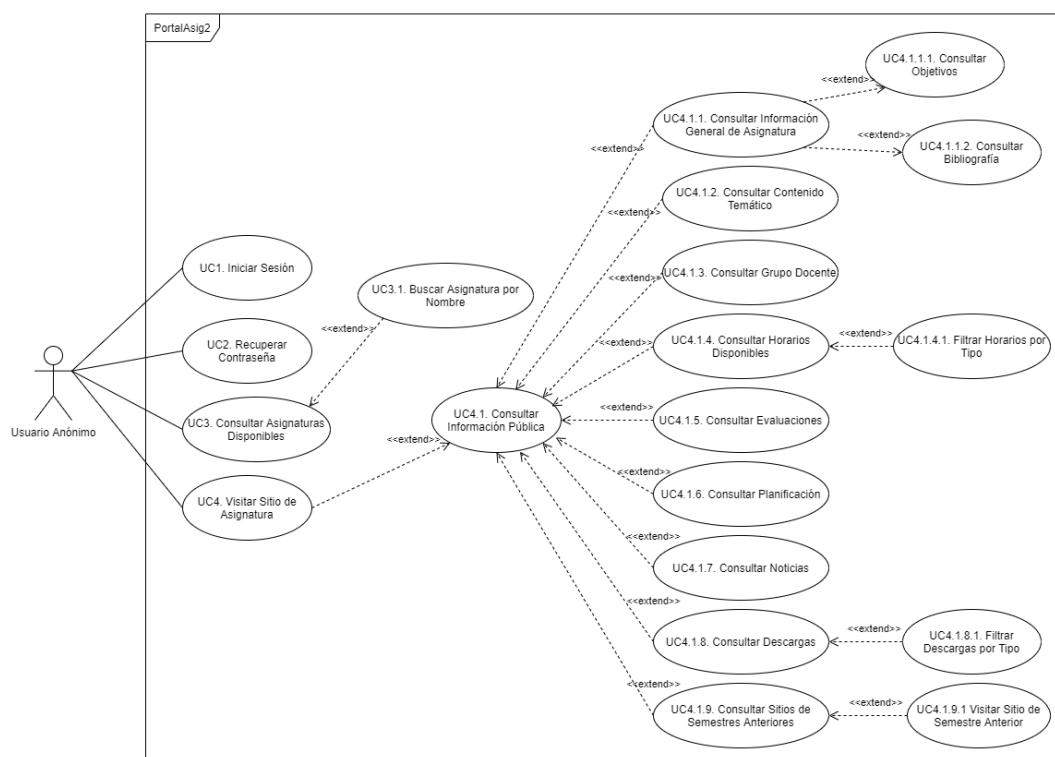


Figura A.1: Diagrama de Casos de Uso Actualizado - Usuario Anónimo

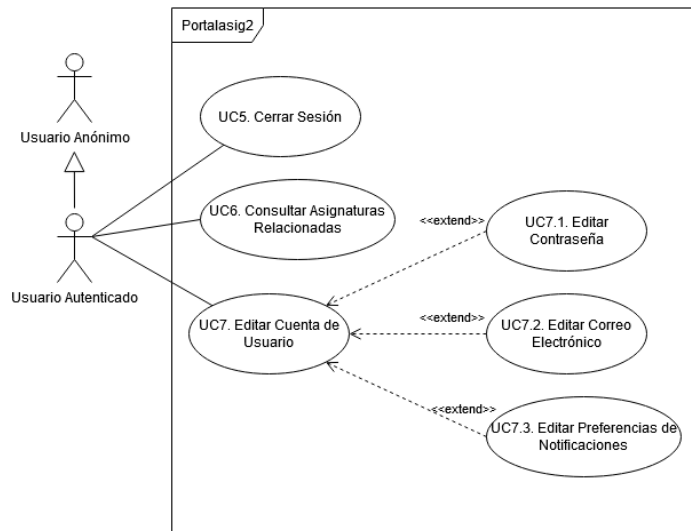


Figura A.2: Diagrama de Casos de Uso Actualizado - Usuario Autenticado

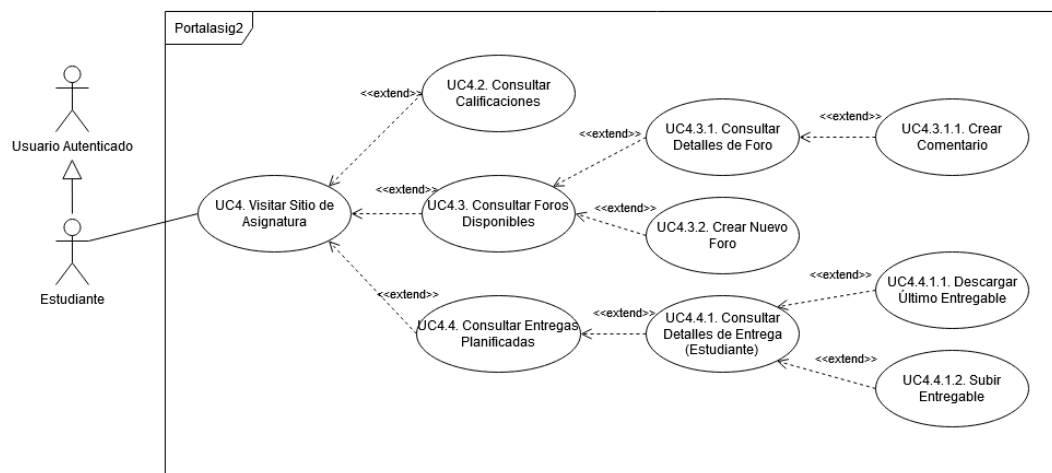


Figura A.3: Diagrama de Casos de Uso Actualizado - Estudiante

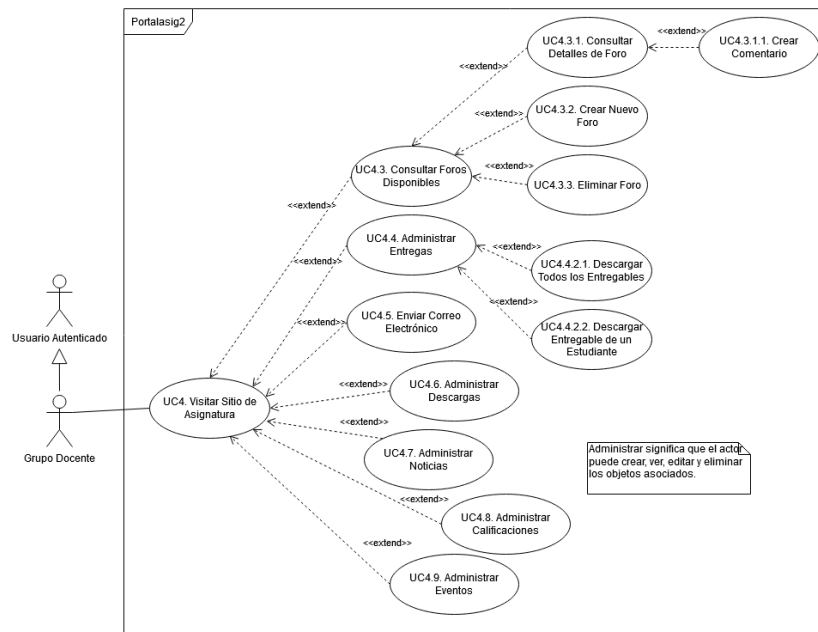


Figura A.4: Diagrama de Casos de Uso Actualizado - Grupo Docente

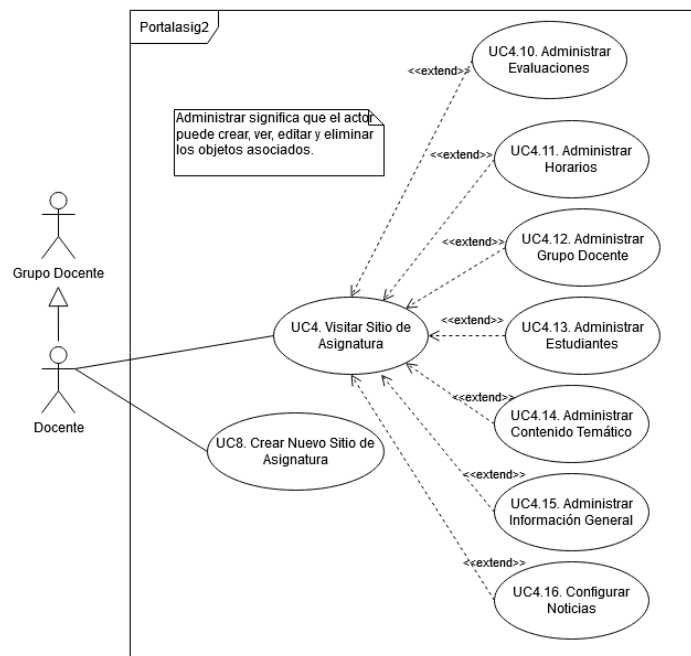


Figura A.5: Diagrama de Casos de Uso Actualizado - Docente

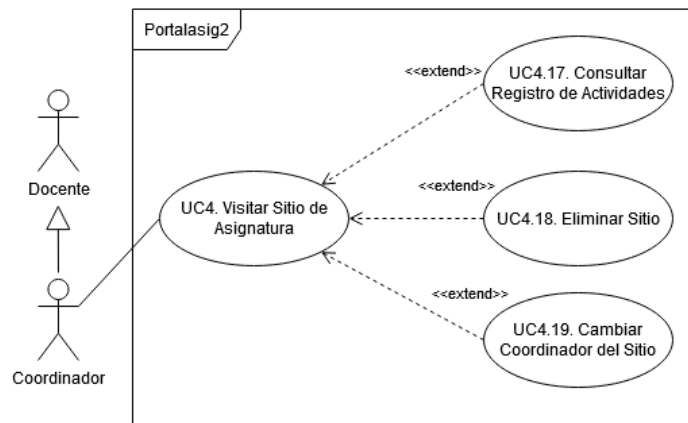


Figura A.6: Diagrama de Casos de Uso Actualizado - Coordinador

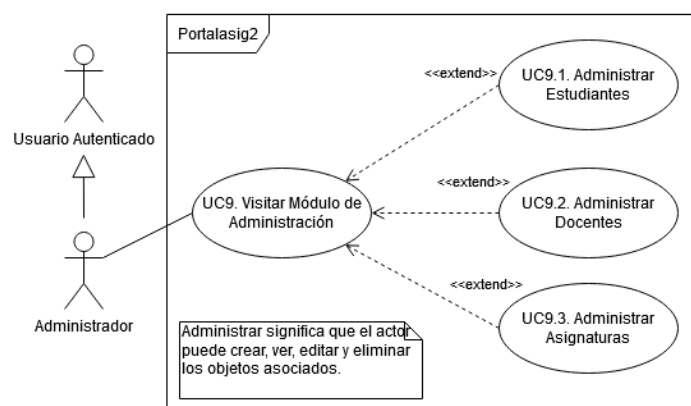


Figura A.7: Diagrama de Casos de Uso Actualizado - Administrador

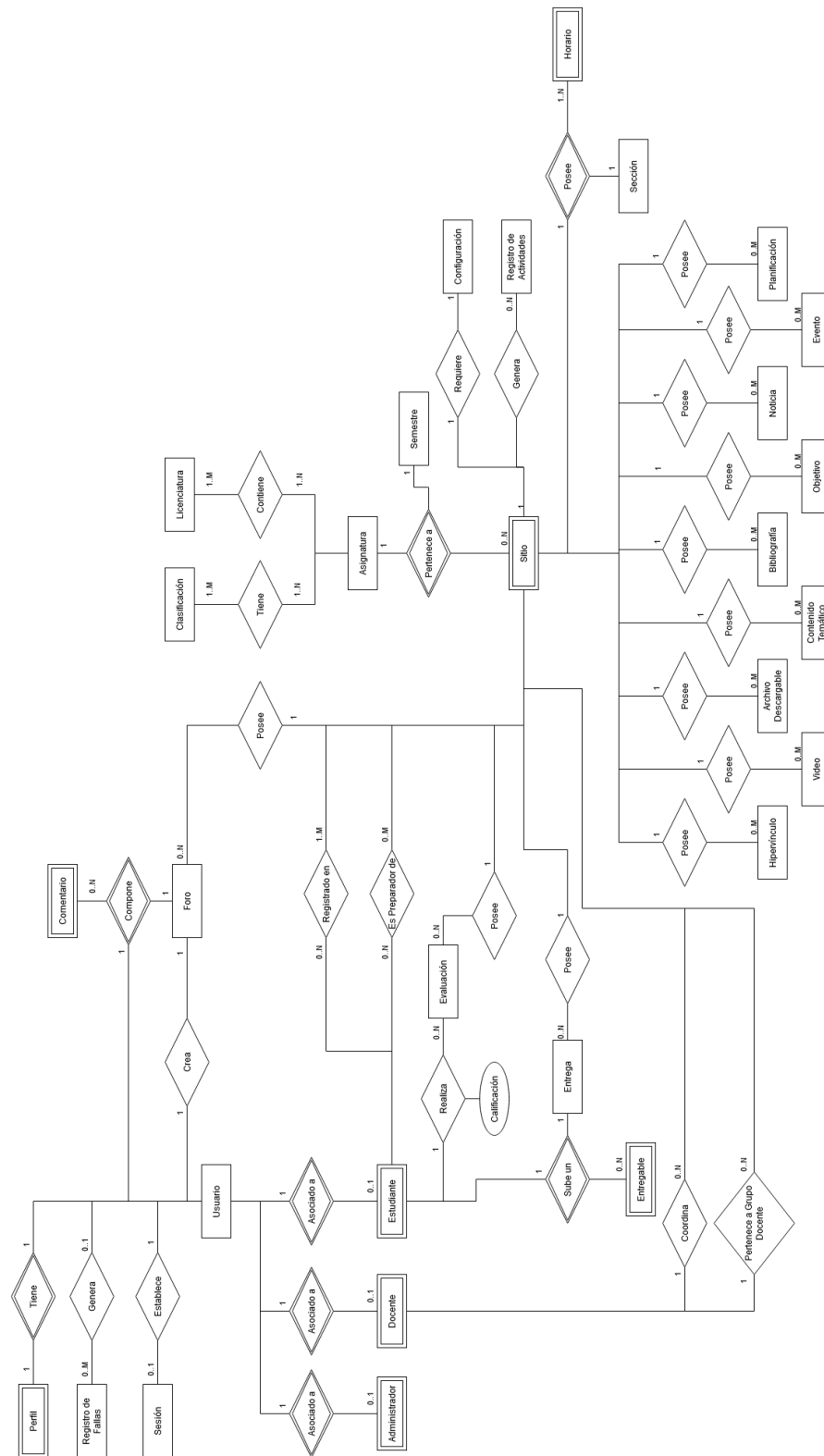
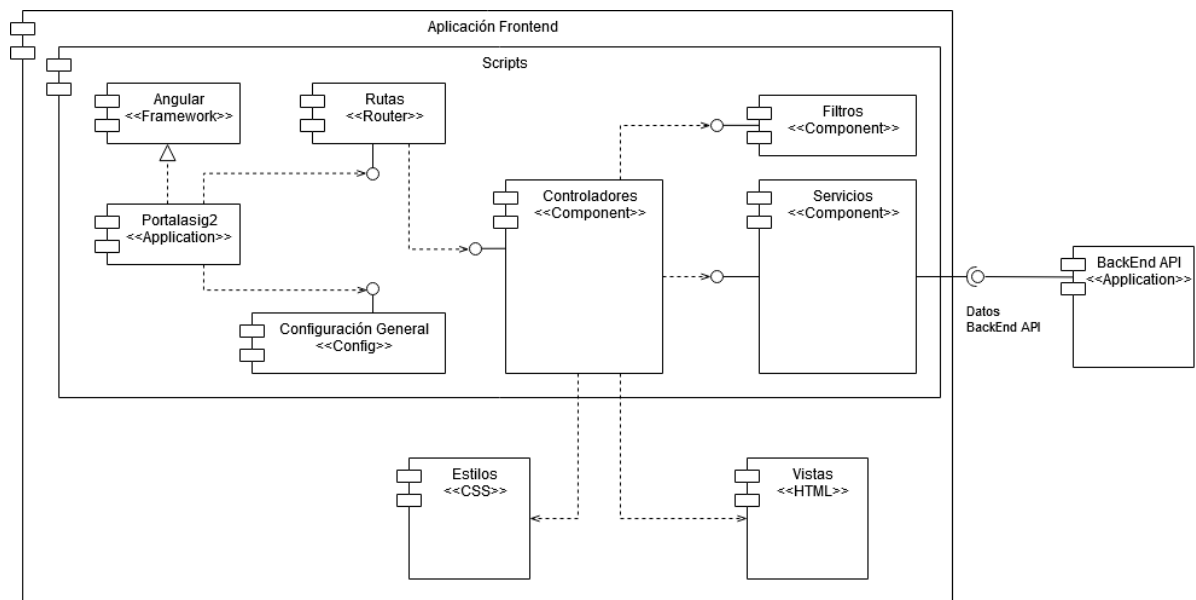
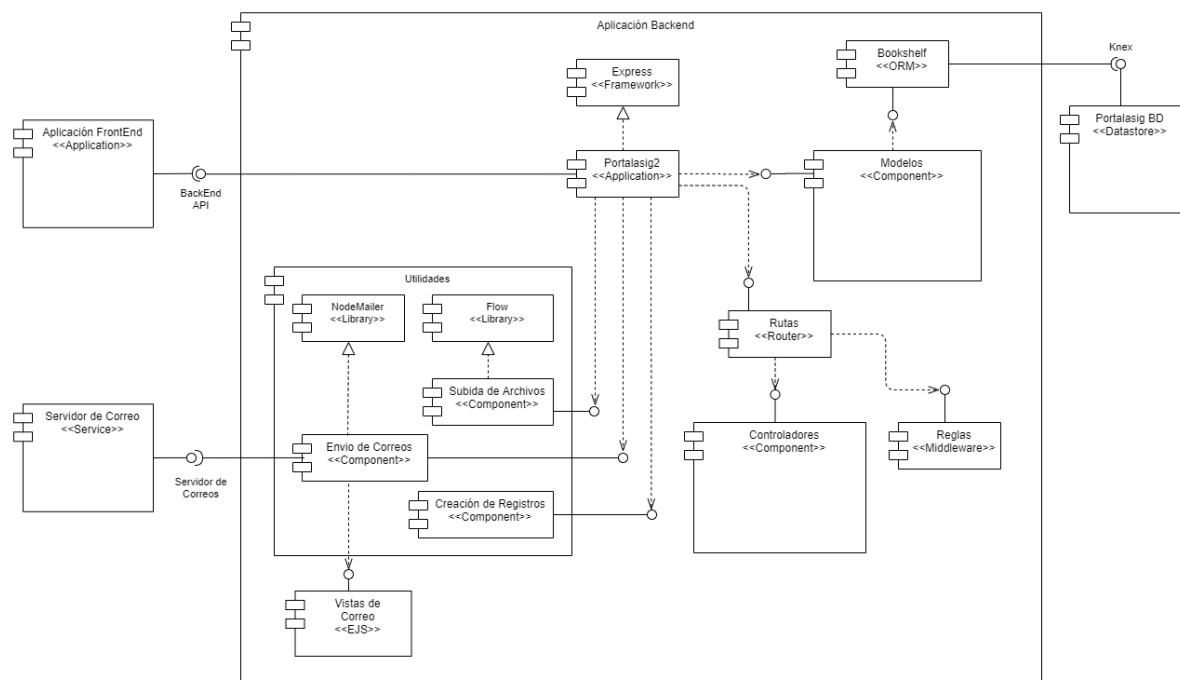


Figura A.8: Diagrama Entidad Relación Actualizado

Figura A.9: Diagrama de Componentes de Aplicación *FrontEnd* ActualizadoFigura A.10: Diagrama de Componentes de Aplicación *Backend* Actualizado

Diagramas de *PortalAsig2* Post Mantenimiento

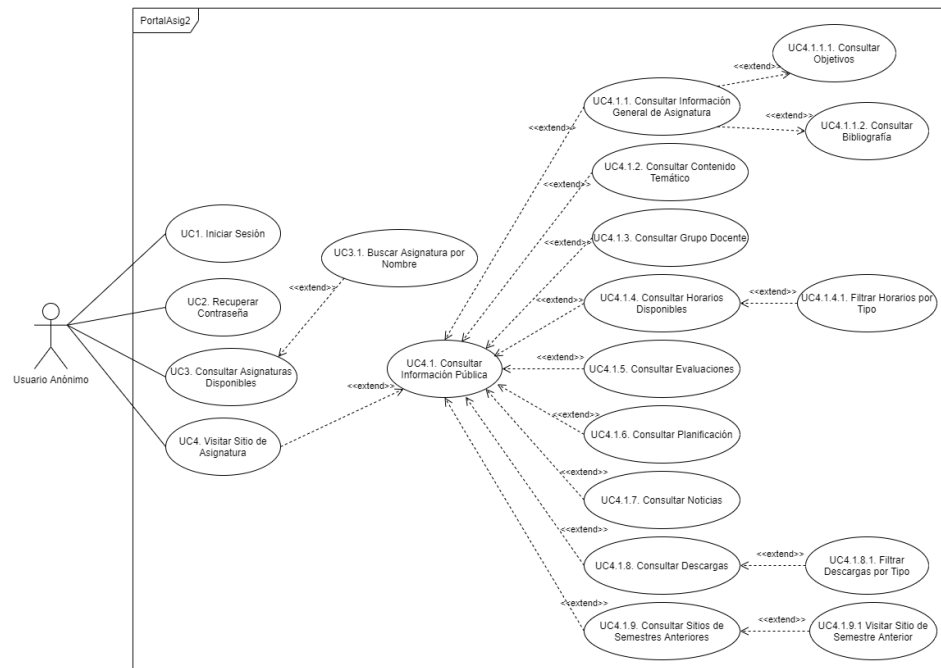


Figura A.11: Diagrama de Casos de Uso- Usuario Anónimo

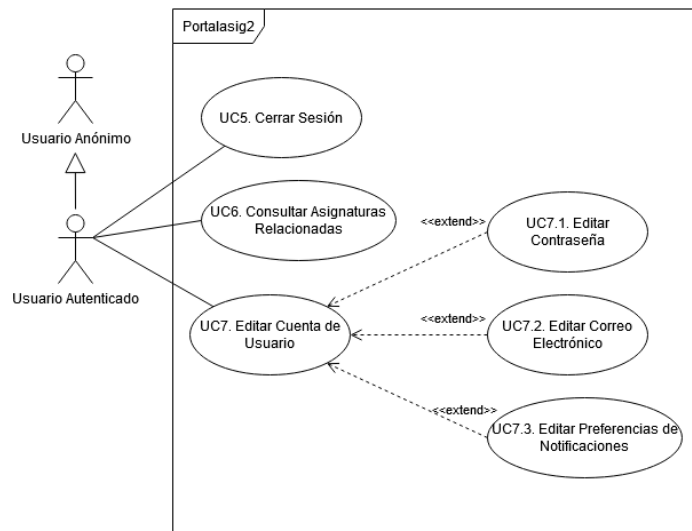


Figura A.12: Diagrama de Casos de Uso - Usuario Autenticado

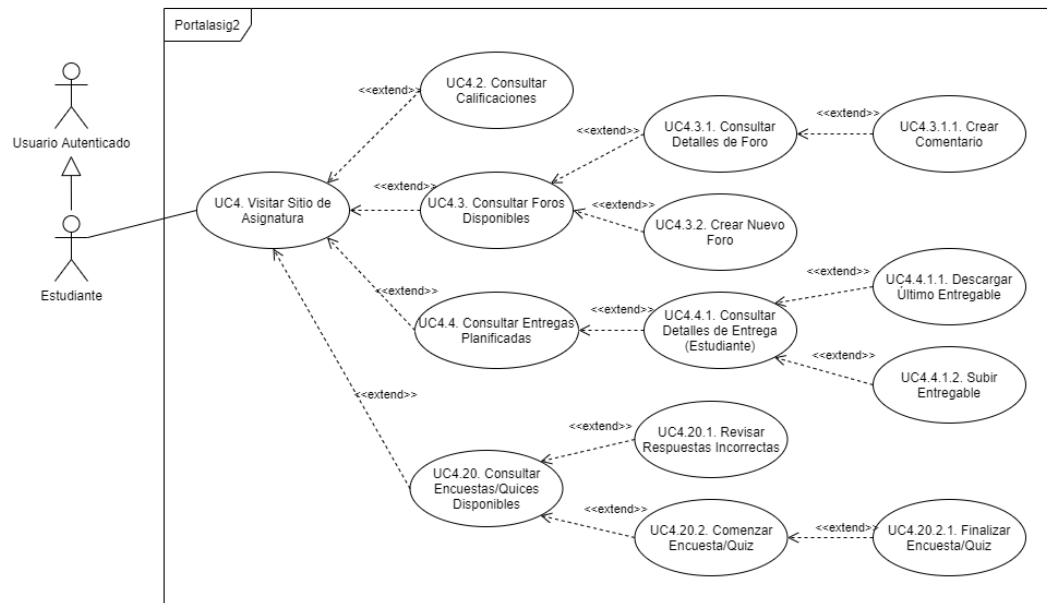


Figura A.13: Diagrama de Casos de Uso - Estudiante

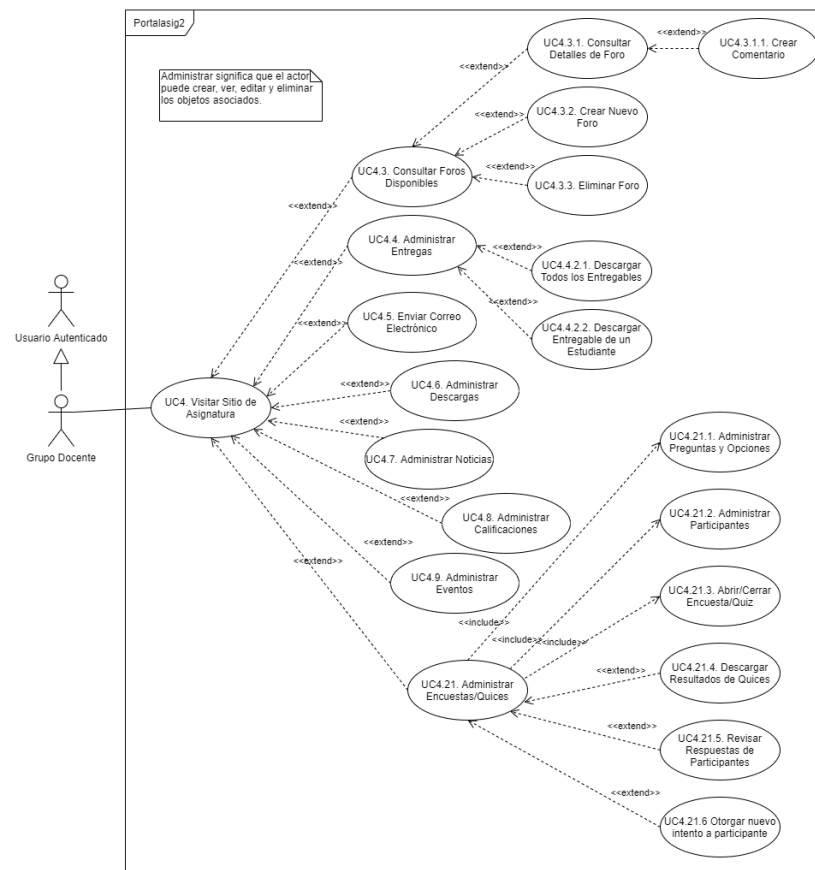


Figura A.14: Diagrama de Casos de Uso - Grupo Docente

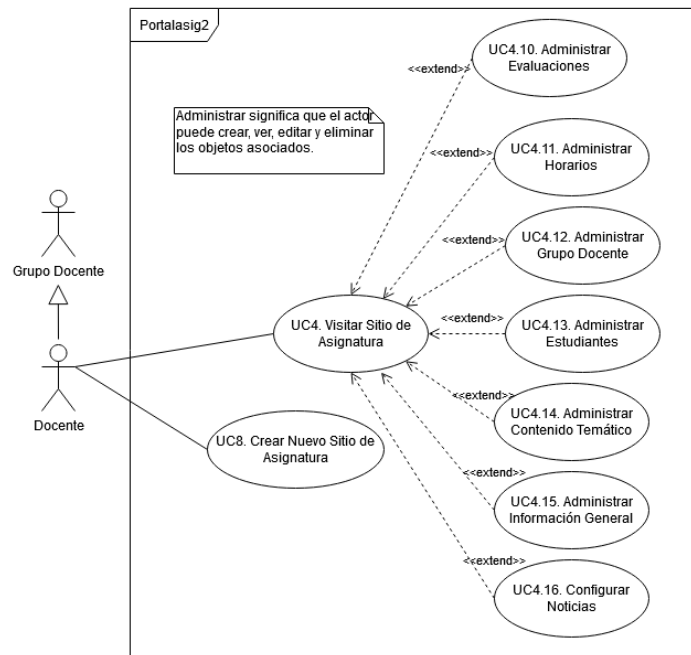


Figura A.15: Diagrama de Casos de Uso - Docente

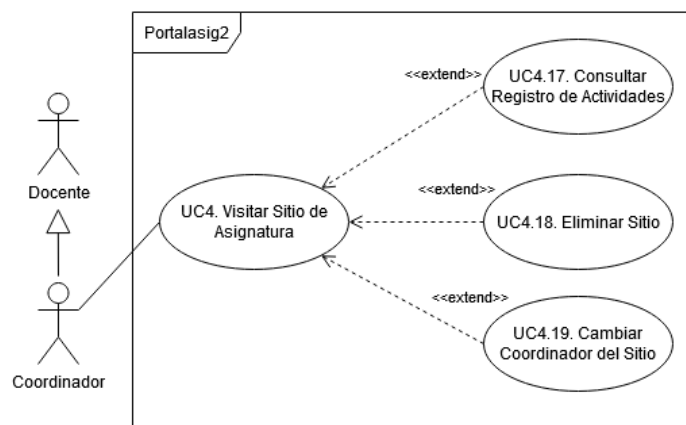


Figura A.16: Diagrama de Casos de Uso - Coordinador

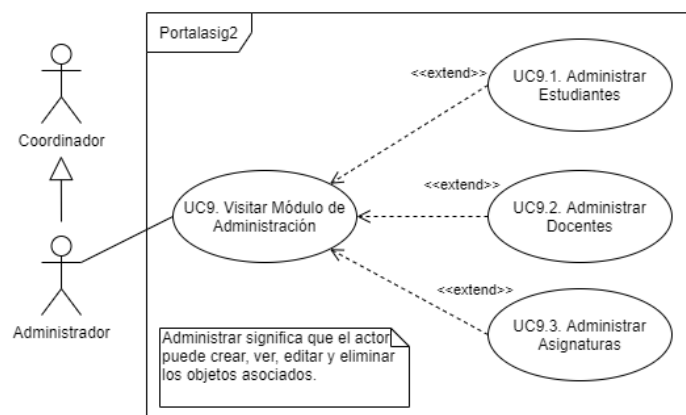
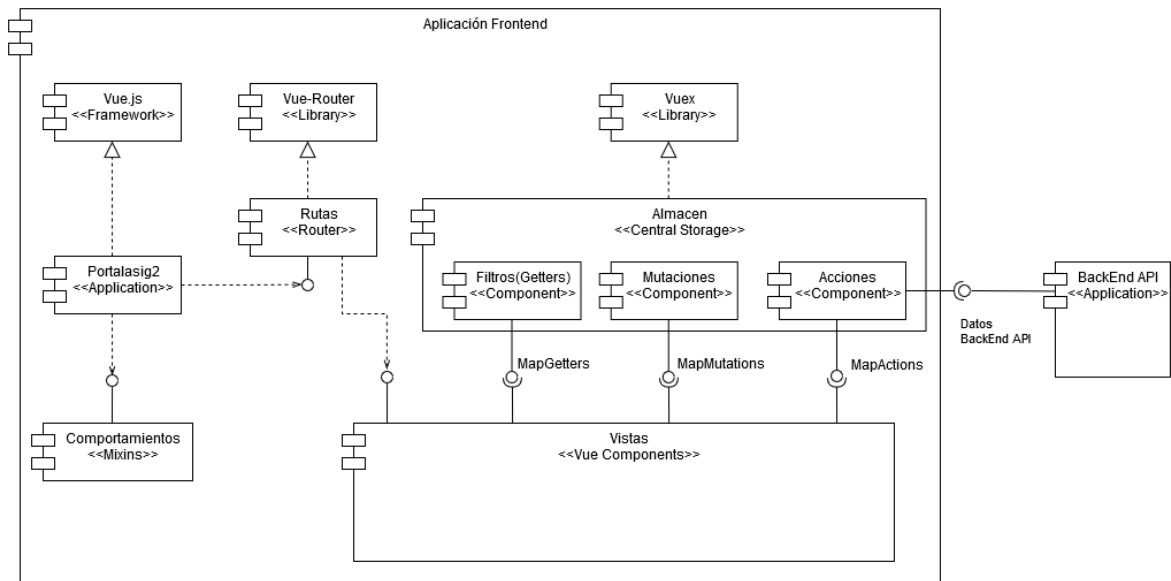
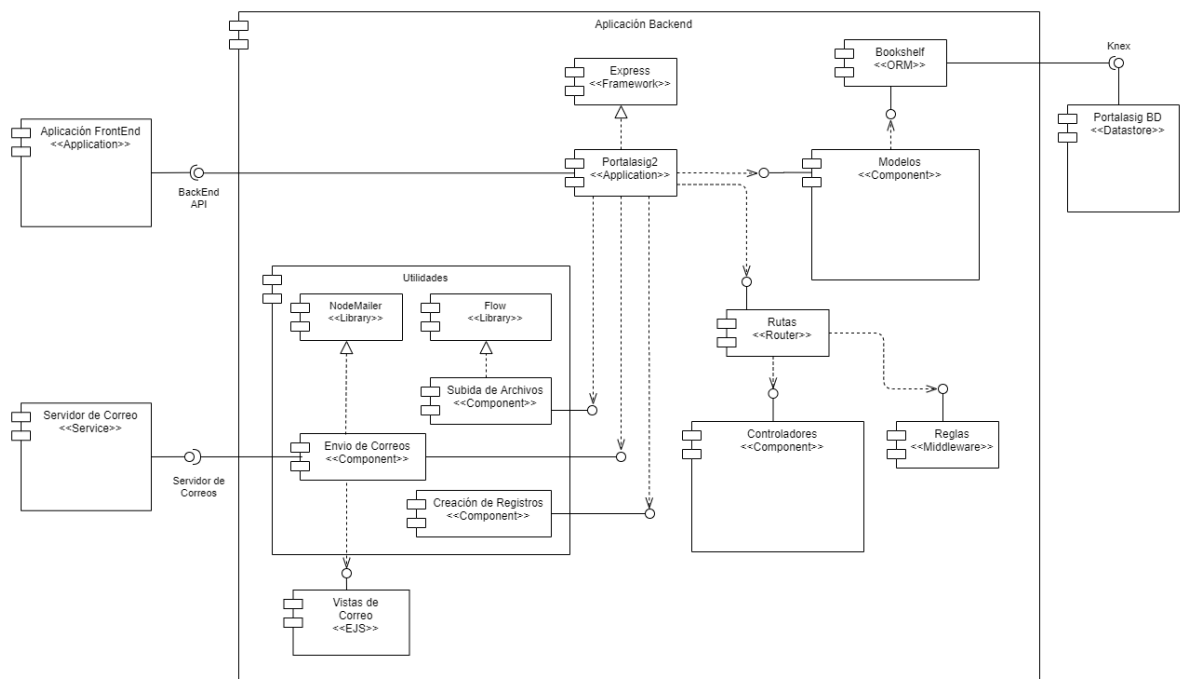


Figura A.17: Diagrama de Casos de Uso - Administrador



Figura A.19: Diagrama de Componentes de Aplicación *FrontEnd*Figura A.20: Diagrama de Componentes de Aplicación *BackEnd*

Bibliografía

- [1] O. Díaz and M. Villavicencio, *T.E.G. Portal generador de sitios web de asignaturas para la Facultad de Ciencias de la Universidad Central de Venezuela*. Caracas, Venezuela, Octubre 2013.
- [2] J. Sánchez, *T.E.G. Desarrollo de una nueva versión del portal generador de sitios web de asignaturas (Portalsig) para la Facultad de Ciencias de la Universidad Central de Venezuela*. Caracas, Venezuela, Febrero 2016.
- [3] I. Sommerville, *Software Engineering*. Addison-Wesley, 9th ed., 2011.
- [4] A. Doza, *T.E.G. Propuesta Metodológica para el Mantenimiento de Software basado en la Modelación Ágil. Caso de Estudio Rubricarte*. Caracas, Venezuela, Abril 2015.
- [5] P. Grubb and A. A. Takang, *Software Maintenance: Concepts and Practice*. World Scientific Publishing, 2nd ed., 2003.
- [6] M. Fowler, K. Beck, J. Brant, W. Opdyke, and D. Roberts, *Refactoring - Improving the Design of Existing Code*. Addison-Wesley Professional, 1999.
- [7] R. S. Pressman, *Software Engineering: A Practitioner's Approach*. McGraw-Hill, 5th ed., 2006.
- [8] L. H. Rosenberg, "Software re-engineering," tech. rep., NASA, Goddard Space Flight Center, 1997.
- [9] A. Alam and T. Padenga, *Application Software Reengineering*. Pearson, 2010.
- [10] "Vue v2 Official Guide." Sitio Web, 2020. <https://vuejs.org/v2/guide>.
- [11] "Vuex Official Guide." Sitio Web, 2020. <https://vuex.vuejs.org>.
- [12] "Vue Router Official Website." Sitio Web, 2020. <https://router.vuejs.org>.
- [13] "Bootstrap 4 Official Website." Sitio Web, 2020. <https://getbootstrap.com>.
- [14] "BootstrapVue Official Website." Sitio Web, 2020. <https://bootstrap-vue.org>.
- [15] "Node Official Website." Sitio Web, 2020. <https://nodejs.org/en/about>.
- [16] "Node Package Manager Official Website." Sitio Web, 2020. <https://www.npmjs.com/about>.

- [17] “Express.js Official Website.” Sitio Web, 2020. <https://expressjs.com>.
- [18] “Knex Official Documentation.” Sitio Web, 2020. <https://knexjs.org>.
- [19] “Bookshelf Official Documentation.” Sitio Web, 2020. <https://bookshelfjs.org>.
- [20] M. Seidl, M. Scholz, C. Huemer, and G. Kappel, *UML @ Classroom: An Introduction to Object-Oriented Modeling*. Springer International Publishing, 2015.
- [21] M. Kaproń and B. Pańczyk, “Modern technologies for creating graphical user interface in web applications,” *Journal of Computer Sciences Institute*, no. 15, pp. 139–142, 2020.
- [22] R. Garrote Jurado, T. Pettersson, A. Regueiro-Gómez, and M. Scheja, “Classification of the Features in Learning Management Systems,” in *Conference: XVII Scientific Convention on Engineering and Architecture*, 11 2014.
- [23] ESComplex, “Software complexity analysis of javascript abstract syntax trees.” Repositorio, 2017. <https://github.com/escomplex/complexity-report>.